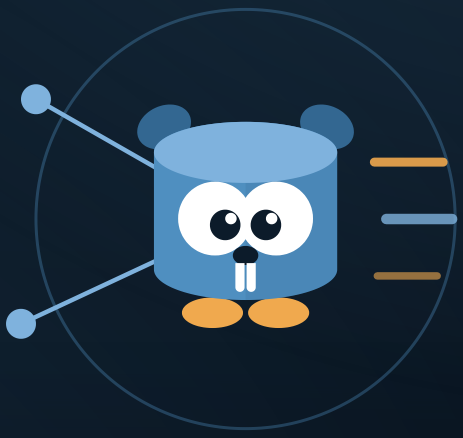
FIRST EDITION

pg

The Guide to the pg PostgreSQL Library for Go



WRITTEN BY

Gerasimos Maropoulos

github.com/kataras/pg

hellenic.dev

- Creator of pg
- Creator of the Iris Web Framework
- Founder of Hellenic Development

About this book

A complete, hands-on guide to github.com/kataras/pg, a PostgreSQL library for Go built on pgx/v5. It covers declaring a schema with `pg` struct tags, connecting and configuring a pool, the generic Repository[T] for CRUD, the Conditions/Where query builder, pagination, writing data with upserts and conflict handling, bulk loading and streaming with COPY, transactions and retry, LISTEN/NOTIFY, schema management and migrations, introspection and code generation, and the security, observability and testing practices a production deployment needs, every example verified against the library source.

Edition	First Edition
Author	Gerasimos Maropoulos
Website	https://github.com/kataras/pg
Source	https://github.com/kataras/pg
License	MIT License
Built	August 2026

> Runnable programs that pair with this book live in the library's own `_examples` directory: https://github.com/kataras/pg/tree/main/_examples.

Contents

Preface 7

1 Getting Started 9

Background 9
Installation 10
What pg Is 10
Your First Program 11
The Pieces 12
Project Layout 13
Summary 13
Further Reading 14

2 Schema and Struct Tags 15

The pg Struct Tag 15
Tag Option Reference 16
Primary Keys and Generated Defaults 17
References and Foreign Keys 18
Indexes and Uniqueness 18
Struct Embedding 19
Column Naming and Name Mappers 19
The Schema Registry 20
Views and Presenters 21
Data Types 21
Identifier Validation 24
Summary 25
Further Reading 25

3 Connections and Configuration 26

Open and OpenPool 26
Connection Strings 27
Connection Options 27
Pool Settings 29
Search Path 29
SSL Modes 30
PgBouncer and Transaction Pooling 31
Closing a DB 32
Health Checks and Pool Statistics 32
Summary 33
Further Reading 33

4 Repositories and CRUD 34

Repository[T]: The Type-Safe Entry Point 34
Reading Rows 35
Inserting Rows 35
Upserting Rows 36
Updating Rows 37
Deleting and Duplicating Rows 38
Counting Rows 38
The Non-Generic *DB Mirror 39
Table-Name CRUD on *DB 40
Updating JSONB Columns 42
Summary 42
Further Reading 43

5 Querying and Scanning 44

Background 44
Raw SQL on DB and Repository 45
The Generic Query Helpers 46
Scanning Into Registered Structs 47
Ad Hoc Read Models: QueryStructs, QueryStruct, ScanStructs 48
Column Name Precedence in LooseTable 49

What Gets JSON-Decoded Automatically, and What Does Not 49
Scanning a Join with to_jsonb 50
Summary 51
Further Reading 52

6 Filtering and Pagination 53
Background 53
The Conditions Builder 54
Building Fragments: Where, And, AndIf 54
Array and Search Helpers 55
Zero-Gated Range and Equality Filters 56
Local Numbering and Build 56
One Filter Set, Two Queries 57
Sorting: Why ORDER BY Cannot Be a Bind Parameter 57
OrderBy Validation and the Fallback Chain 58
Pagination with PageOptions and SelectPaginated 59
SelectWithTotal and the Window Column 60
Summary 61
Further Reading 61

7 Writing Data 63
Background 63
How Inserts Are Built 64
Zero Values, Defaults, and the UUID Primary Key Rule 64
RETURNING and idPtr 65
Single vs Many: InsertMany, UpsertMany, and the Batching Cap 65
Upsert and the Tag-Driven Conflict Target 66
Explicit Conflict Specs: desc.OnConflict 67
Repository-Level Conflict Methods 68
UpdateOrInsert: Check-Then-Act Writes 69
Updates: Update, UpdateOnlyColumns, UpdateExceptColumns 69
UpdateJSONB 71
Deletes 71
Password Columns 71
Summary 72
Further Reading 73

8 Bulk Loading and Streaming 74
Background 74
The COPY Protocol and CopyPlan 75
The All-or-Nothing Default Rule 75
DB.CopyFrom and Repository.CopyFrom 76
ErrCopyPassword and the Empty-Column Error 77
CopyFrom versus InsertMany 77
Streaming Reads: SelectIter and QueryIter 78
Single-Use Iterators and Connection Lifetime 79
The Server-Side Cursor Pattern 80
Summary 80
Further Reading 81

9 Transactions 82
Background 82
DB.InTransaction 83
Why Commit Errors Must Propagate 84
Nesting: Join Versus Savepoint 85
Manual Control: Begin, Commit, Rollback, IsTransaction 86
Repository InTransaction 87
pg.InTransaction and Typed Wrappers 87
Concurrent Access to One Transaction 88
Retrying Transient Failures 89
RetryOptions 90
InTransactionRetry 91
Full Jitter Backoff 91
Nested Retry Calls Run Once 92
ExecMany 93
SetConstraintsDeferred 93
Summary 94
Further Reading 95

10 Errors 96
Background 96
ErrNoRows 97

The Classifiers 97
ConstraintKind 98
ConstraintError 99
AsConstraintError Is Extraction-Only 100
Mapping Errors to an HTTP Response 100
Other Exported Sentinels 101
SQLSTATE Lookup Table 102
Summary 103
Further Reading 103

11 Schema Management and Migrations 104

Background 104
CreateSchema 105
CreateSchemaDumpSQL 105
Extensions Created on Demand 106
The set_timestamp Trigger Convention 106
CheckSchema 107
DeleteSchema 108
DB.Migrate 108
The Advisory Lock 109
The schema_migrations Ledger 110
Embedding Migrations 110
What Migrate Deliberately Does Not Do 111
When to Reach for a Dedicated Migration Tool 111
Summary 112
Further Reading 112

12 LISTEN and NOTIFY 114

Background 114
DB.Listen and Notification Delivery 115
DB.Notify 115
DB.Unlisten 116
The Listener Type 116
Notification, ErrEmptyPayload and UnmarshalNotification 117
Quoted Channel Identifiers 118
The Table-Change Layer 118
ListenTableOptions 119
The Trigger and Function pg Installs 119
TableNotification and Change Kinds 120
Repository ListenTable 121
The Identifier Restriction 121
Operational Caveats 122
Summary 123
Further Reading 123

13 Introspection and Code Generation 125

Background 125
Listing Tables 125
Filtering Tables 126
Listing Columns 127
Constraints and Unique Indexes 128
Triggers 128
Table and Database Size 128
Server Version 129
Autovacuum Controls 129
Reverse-Engineering Structs From a Database 130
File Layout Strategies 131
Formatting Generated Code 132
Typed Column-Name Constants 132
Safety Guards in the Generator 133
Summary 133
Further Reading 134

14 Security 135

Background 135
Values Are Bind Parameters 136
Identifiers: Validate, Then Quote 136
Why Quoting Alone Is Not Enough 137
The Trust Boundary 138
Developer-Authored SQL in Tags and Builders 139
Passwords 139
Secrets in Logs 141
TLS 141
Least-Privilege Database Roles 142
Error Messages Without Leaking Schema 142
Summary 143

15 Observability and Operations 145

Readiness and Liveness 145
Pool Statistics 146
Logging 147
Query Tracers and OpenTelemetry 147
The WithLogger/WithQueryTracer Ordering Caveat 148
Connection Pool Sizing 148
Exec Modes and Prepared-Statement Caching 149
PgBouncer and Transaction Pooling 149
Vacuum and Table Sizes 150
Timeouts 151
Retrying Transient Failures 151
Summary 152
Further Reading 153

16 Testing 154

Background 154
Why a Real Database, Not a Mock 155
The pgtest Package 155
ConnString and Skipping Cleanly 155
New: an Ephemeral Schema per Test 156
The Shared-Schema Constraint 156
A Full Test 157
Table-Driven Tests Over a Repository 158
Testing Transaction Rollback 159
Asserting on Typed Errors 160
Running the Suite in CI 161
Summary 161
Further Reading 162

Epilogue 163

Preface

pg is a PostgreSQL library for Go, built on top of pgx/v5. It maps Go structs to database tables through `pg:"..."` struct tags, generates and verifies the schema those tags describe, and exposes a generic `Repository[T]` for the CRUD operations most applications write by hand against `database/sql`. This book is its guide: a path from an empty `go.mod` to a tested, production-shaped data layer, written for the library as it exists today and verified against its source code.

Who this book is for

You should be comfortable with Go (structs, generics, interfaces, the standard library basics) and with relational databases in general: what a primary key is, what a transaction guarantees, roughly how an index speeds up a query. No prior experience with pg, or with pgx, is assumed. If you have used `database/sql` directly, or another Go database library, you will recognize the shape of the problems pg solves; the book takes care to explain not just what an API does but why it is shaped the way it is, since that is usually the part a reference alone leaves out.

How the book is organized

The chapters build on one another and are best read in order the first time; afterwards each stands alone as a reference.

- **Chapters 1–3** are the foundation: installing pg, declaring a schema with struct tags, and opening and configuring a connection pool.
- **Chapters 4–7** cover everyday data access: the repository pattern for CRUD, querying and scanning rows, filtering and pagination with the `Where` builder, and writing data with inserts, updates and upserts.
- **Chapters 8–10** cover higher-throughput and failure-handling paths: bulk loading and streaming with `COPY`, transactions, and the error types pg returns so you can branch on a

constraint violation instead of matching an error string.

- **Chapters 11–13** cover the database itself: schema management and migrations, LISTEN/NOTIFY, and introspecting a live database or generating Go code from one.
- **Chapters 14–16** are about running pg in production: security, observability and operations, and testing with the `pgtest` package. An epilogue closes the book with what to build next and where to go for updates.

Conventions

Code examples use the full import path (`github.com/kataras/pg`) and are complete enough to adapt directly into a real program; where an example belongs to a larger program, the chapter says so. Shell commands are written for a POSIX shell and work in PowerShell unless noted. The examples target Go 1.26, the version pg's own go.mod requires, and are written against the library as published in this book's edition; every identifier in every listing was checked against the source before the chapter shipped.

Each chapter closes with a **Further Reading** list of canonical references: the PostgreSQL documentation, the pgx documentation, and relevant parts of go.dev, for going deeper than a single chapter can. Some chapters open with a **Background** section on the underlying concept (what a connection pool does, what `LISTEN` / `NOTIFY` guarantees) for readers new to that piece of PostgreSQL specifically; where a chapter has one, its introduction says where to skip to if you already know the fundamentals.

About pg

pg has been developed by Gerasimos Maropoulos (@kataras), the author of the Iris web framework, and is released under the MIT license. The library lives at github.com/kataras/pg, where issues, discussions and contributions are welcome; runnable programs beyond the ones in this book live in its `_examples` directory. This book is maintained separately from the library, and corrections to it are welcome the same way.

CHAPTER 1

Getting Started

`github.com/kataras/pg` is a Go library for working with PostgreSQL. It sits directly on top of `jackc/pgx/v5`, the PostgreSQL driver for Go, and adds a thin, type-safe mapping layer on top of it: you describe a table once, as a Go struct carrying `pg:"..."` field tags, and the library generates the SQL needed to create that table, verify it against a live database, and read and write rows through a generic `Repository[T]`. The module path is `github.com/kataras/pg` and it requires Go 1.26 or newer. This chapter installs the module, builds a first program end to end (a struct, a schema, a connection, an insert and a read), and names the pieces you will meet again in every later chapter. Every symbol and example in this chapter is checked against the library's source before being written down.

Background

If you have used Go's `database/sql` package, or another PostgreSQL client for Go, before, skip to Installation.

What a driver does. `database/sql` defines a small set of interfaces (a connection, a statement, a set of result rows) and delegates the actual wire protocol to a driver package registered under a name such as "postgres". The driver is the part that speaks PostgreSQL's binary protocol: it opens a TCP connection, authenticates, encodes query parameters and decodes result rows into Go values. pg does not implement this layer itself. It is built directly on `jackc/pgx/v5`, a PostgreSQL driver that exposes its own richer API in addition to a `database/sql`-compatible one, and pg uses that native API rather than going through `database/sql`. That is why pg's own `Rows` and `Row` types are aliases for `pgx.Rows` and `pgx.Row`, not `sql.Rows` and `sql.Row`.

What a connection pool does. Opening a TCP connection to Postgres, negotiating TLS and authenticating costs real time, typically several milliseconds, so no serious program opens a fresh connection for every query. A connection pool opens a bounded number of connections up front or on demand, hands one out per query or transaction, and returns it to the pool when

the caller is done with it. pgx's pool type is `pgxpool.Pool`, and a pg `*DB` wraps exactly one `*pgxpool.Pool`: every query pg issues acquires a connection from that pool and releases it afterward. Chapter 3 covers how the pool is sized and configured.

What a struct-to-table mapper does. Without one, every read means hand-writing `rows.Scan(&a, &b, &c)` in the exact column order of the query, and every write means hand-writing placeholders and an argument list that has to stay in sync with the struct by hand. A mapper closes that gap in both directions: it reads a Go struct's fields (here, through `pg:"..."` tags) to know what column each field represents, then uses that same information to match result columns to struct fields by name when scanning, and to build `INSERT`, `UPDATE` and `CREATE TABLE` statements for the struct. pg's mapper lives in the `desc` subpackage and is the subject of the whole of Chapter 2.

Installation

You need Go 1.26+. Create a module and add pg to it:

```sh
mkdir myapp && cd myapp
go mod init myapp
go get github.com/kataras/pg@latest
```

pg depends on `github.com/jackc/pgx/v5` (the driver and connection pool), `github.com/gertd/go-pluralize` (used by the naming helpers that turn `Customer` into `customers`) and `golang.org/x/mod`. All three are ordinary transitive dependencies pulled in by `go get`; there is no code generator step required to start using the library, though the `gen` subpackage can optionally generate Go structs from an existing database (or column-constant files from an already-registered `Schema`) when you prefer to start from SQL instead of from Go.

What pg Is

pg is not a full ORM in the sense of lazy-loaded associations or a query DSL that abstracts SQL away from you. You still write SQL for anything beyond simple CRUD: `Select` and `SelectSingle` on a `Repository[T]` both take a query string and arguments and scan the result into your struct type. What pg removes is the boilerplate around that SQL: connecting, building `INSERT` / `UPDATE` / `DELETE` statements for a struct's own primary key, scanning rows into structs by column name, and generating (and later verifying) the DDL for a table straight from the struct

that describes it. The trade-off is explicit: you accept a `pg:"..."` tag vocabulary and a registration step, in exchange for never hand-writing a `Scan` call list again for the common cases.

Your First Program

A pg program has a fixed shape: define one or more structs with `pg` tags, register them in a `Schema`, open a `*DB`, then read and write through a `Repository[T]`. Here it is end to end.

```go
// main.go
package main

import (
    "context"
    "log"
    "time"

    "github.com/kataras/pg"
)

// Customer maps to the "customers" table. The `pg` tag on each field
// tells the library the column's name (when given explicitly),
// PostgreSQL type, and any constraints.
type Customer struct {
    ID        string    `pg:"type=uuid,primary"`
    CreatedAt time.Time `pg:"type=timestamp,default=clock_timestamp()"`
    Firstname string    `pg:"type=varchar(255)"`
}

func main() {
    ctx := context.Background()

    // A Schema is a registry: it maps table names to the struct
    // types that describe them.
    schema := pg.NewSchema()
    schema.MustRegister("customers", Customer{})

    // Open parses the connection string, builds a pgxpool.Pool and
    // pings it once. Prefer sslmode=verify-full in production; see
    // Chapter 3 for what each sslmode actually guarantees.
    connString := "postgres://postgres:admin!123@localhost:5432/" +
        "test_db?sslmode=disable"
    db, err := pg.Open(ctx, schema, connString)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    // CreateSchema issues CREATE TABLE (and any extensions,
    // foreign keys and triggers the schema needs) for every
    // registered table that does not exist yet.
    if err = db.CreateSchema(ctx); err != nil {
        log.Fatal(err)
    }
```

```
    // A Repository[T] is the type-safe entry point for CRUD on one
    // table. NewRepository panics if T was never registered, so a
    // typo here is caught at startup, not at the first query.
    customers := pg.NewRepository[Customer](db)

    newCustomer := Customer{Firstname: "Ada"}

    // InsertSingle's second argument, when non-nil, receives the
    // row's primary key after insert. ID has no explicit default in
    // the tag above, so the library filled one in automatically:
    // see the note below.
    err = customers.InsertSingle(ctx, newCustomer, &newCustomer.ID)
    if err != nil {
        log.Fatal(err)
    }

    existing, err := customers.SelectByID(ctx, newCustomer.ID)
    if err != nil {
        log.Fatal(err)
    }

    log.Printf("inserted customer: %#v\n", existing)
}
```

Run it:

```sh
go run main.go
```

Two details are worth pulling out. First, `Customer.ID` is tagged `pg:"type=uuid,primary"` with no `default=` option, yet the row gets an ID. A primary key column of type `uuid` with no explicit default and no nullable flag is given the default `gen_random_uuid()` automatically; this is also why `CreateSchema` emits `CREATE EXTENSION IF NOT EXISTS pgcrypto` for any schema that uses a `uuid` column, since `gen_random_uuid()` lives in that extension. Second, `InsertSingle`'s `idPtr` argument is what makes the generated ID visible to your program: pass a pointer to the field, and the generated `INSERT` statement gets a `RETURNING` clause scanned back into it; pass `nil` and the row is still inserted, but the ID is not read back.

The Pieces

The example above touches every major type in the library. Later chapters go deep on each one; this is the map.

Type	Role
`Schema`	A registry of table name to Go struct type, built with `NewSchema` and populated with `Register` / `MustRegister`.

`*DB`	A connection to one PostgreSQL database, wrapping a `*pgxpool.Pool`, opened with `Open` or `OpenPool`. Also exposes non-generic CRUD, transactions, `LISTEN` / `NOTIFY` and schema DDL.
`Repository[T]`	A type-safe, table-scoped view over `*DB` for one registered struct type `T`. Built with `NewRepository[T](db)`.
`desc` (subpackage)	Parses `pg` struct tags into `Table` / `Column` descriptors and builds the SQL (`CREATE TABLE`, `INSERT`, `UPDATE`, `DELETE`, scanning) behind both `*DB` and `Repository[T]`. Used directly only for advanced cases.
`gen` (subpackage)	Generates Go struct source from a live database's schema, or column-constant files from an already-registered `Schema`, for teams that prefer to start from SQL.
`pgtest` (subpackage)	Test helpers for spinning up a throwaway PostgreSQL instance (or reusing one) in Go tests.

`Repository[T]` is deliberately generic per table rather than one untyped `DB.Query` call per operation: `pg.NewRepository[Customer](db)` gives you `Select`, `Insert`, `Update` and the rest already scoped to the `customers` table and to the `Customer` type, so a mismatched struct type is a compile error, not a runtime scan failure. Chapter 4 covers every method it exposes, along with the non-generic mirror on `*DB` itself for code that does not want a type parameter.

Project Layout

A single `main.go` is a fine layout while you are learning the library. As a program grows, a common split keeps struct definitions, the schema registration and the repositories in their own package, separate from the code that uses them:

```
myapp/
├── go.mod
├── main.go          # wiring: Open, CreateSchema, run
├── store/           # domain package: structs, schema, repositories
│   ├── schema.go    # struct definitions and Schema registration
│   └── customers.go # CustomerRepository built on pg.Repository[Customer]
└── api/             # whatever consumes store (HTTP handlers, CLI, ...)
```

The rule that keeps this healthy is the same one that keeps any layered Go program healthy: `store` knows nothing about `api`, and `api` depends on `store`, never the other way around. That is what lets you test `store` with a real (or `pgtest`-provisioned) database and no HTTP server in the loop at all.

Summary

- pg is a type-safe mapping layer on top of `jackc/pgx/v5`, not a full ORM; you still write SQL for anything past simple CRUD.
- Install with `go get github.com/kataras/pg@latest`; no code generation step is required to start.
- A program's shape is fixed: define structs with `pg:"..."` tags, register them with `schema.MustRegister(tableName, StructValue{})`, open a `*DB` with `pg.Open`, then read and write through `pg.NewRepository[T](db)`.
- `db.CreateSchema(ctx)` issues the DDL for every registered table; a `uuid` primary key with no explicit default gets `gen_random_uuid()` automatically, and pulls in the `pgcrypto` extension.
- `InsertSingle(ctx, value, &value.ID)` inserts a row and reads its generated primary key back through a `RETURNING` clause; passing `nil` instead skips that read.
- `Schema`, `*DB`, `Repository[T]`, and the `desc`, `gen` and `pgtest` subpackages are the whole surface area; every later chapter is about one of them.

Further Reading

- A Tour of Go: the interactive introduction to the Go language this book assumes.
- Managing dependencies (go.dev): the official guide to Go modules, `go get` and `go.mod`.
- pgx godoc: the driver pg is built on; useful whenever you reach past pg's own API for `Rows`, `Row` or pool configuration.
- PostgreSQL: gen_random_uuid(): the pgcrypto function pg uses as the default for an untagged `uuid` primary key.
- pkg.go.dev/github.com/kataras/pg: the generated godoc reference for every exported symbol in this book.

CHAPTER 2

Schema and Struct Tags

Every table pg knows about starts life as a Go struct and a `pg` struct tag on each of its fields. This chapter is the reference for that tag: every option it accepts, what each one changes about the generated `CREATE TABLE` and about how a struct scans and writes, and the `Schema` registry that turns a set of tagged structs into a live database. It also covers column naming, views and presenters (tables that are read-only, or that exist only to shape a custom `SELECT`), the full data type catalog, and a validation rule added recently that rejects unsafe table, column and index names outright. Every option and function name below is read directly out of `desc/struct_table.go`, `desc/column.go`, `desc/naming.go`, `desc/data_type.go` and `schema.go`.

The pg Struct Tag

A field participates in a table only if it carries a non-empty `pg` tag; a field with no tag, or with `pg:"-"`, is skipped entirely before anything else runs. The tag's default name is `pg`, changeable with the package-level `SetDefaultTag` function if that name collides with another tag you already use.

The tag body is a comma-separated list of options. Each option is either bare (`primary`, `unique`) or a `key=value` pair (`type=uuid`, `default=now()`). A bare option that is not one of the recognized boolean flags is treated as a naming shorthand: `pg:"table_name"` on its own is equivalent to `pg:"name=table_name"`, which is exactly the pattern the presenter example later in this chapter uses for a struct whose fields map onto raw `information_schema` column names.

```go
type Customer struct {
    ID        string    `pg:"type=uuid,primary"`
    Email     string    `pg:"type=varchar(255),unique"`
    CreatedAt time.Time `pg:"type=timestamp,default=clock_timestamp()"`
}
```

Tag Option Reference

The table below lists every option `convertStructFieldToColumnDefinion` (`desc/struct_table.go`) recognizes, in the order the switch statement handles them.

Option	Value	Effect
`name`	string	Overrides the column name (default: derived from the field name; see Column Naming). A bare tag with no `=` and no recognized keyword, e.g. `pg:"id"`, is shorthand for `name=id`.
`type`	a data type name, optionally with `(argument)`	Sets the column's PostgreSQL type; see Data Types. `varchar(255)` splits into type `CharacterVarying` and type argument `255`. Optional: a field's Go type already maps to a default (see Data Types); write `type=` explicitly whenever that default is not what you want, or is ambiguous (`jsonb` vs `json`, an array element type, a `uuid`-typed string).
`primary`, `pk`	bool	Marks the column as the table's primary key. See Primary Keys.
`identity`	bool	Marks the column `GENERATED ALWAYS AS IDENTITY`. Also sets `AutoGenerated`, so it is skipped from `INSERT` / `UPDATE` argument lists.
`default`	a SQL expression	Sets a `DEFAULT` clause, written verbatim into the `CREATE TABLE` (see the security note below). `default=null` is shorthand for `nullable`.
`unique`	bool	Adds a single-column `UNIQUE` constraint. Mutually exclusive with `unique_index` on the same column; combining both is a registration error.
`conflict`	a SQL `ON CONFLICT` action	Names the `DO UPDATE SET ...` (or similar) clause `Repository.Upsert` / `UpsertSingle` use when no explicit conflict target is given. Exactly one column per table may set this.
`username`	bool	Marks the column as the username half of `SelectByUsernameAndPassword`.
`password`	bool	Marks the column as the password half of `SelectByUsernameAndPassword`; PostgreSQL does the encryption/decryption via `crypt()`, and `Schema.HandlePassword` must be set for the plain-text round trip to work. A `password` field with no explicit `type=` defaults to `Text`.
`nullable`, `null`	bool	`nullable` (or `null`) sets `DEFAULT NULL` and marks the column nullable; explicitly setting it to `false` clears a previously-set `default=null`.
`ref`, `reference`, `references`	`table(column [action] [deferrable])`	Adds a foreign key. See References.

`index`	an index type, or bare for `btree`	Adds a plain (non-unique) index of the given access method: `btree`, `hash`, `gist`, `spgist`, `gin` or `brin`.
`unique_index`	an index name	Adds the column to a named multi-column unique index; several columns sharing the same `unique_index` value form one composite unique constraint.
`check`	a SQL boolean expression	Adds a `CHECK(...)` constraint, written verbatim (see the security note below).
`generated`	a SQL expression	Makes the column `GENERATED ALWAYS AS (...) STORED`. Also sets `AutoGenerated`.
`auto`	bool	Marks the column auto-generated without a `GENERATED` clause of its own (e.g. a value a trigger fills in): skipped from `INSERT` / `UPDATE`.
`presenter`	bool	Excludes the column from `CREATE TABLE`, `INSERT`, `UPDATE` and `Duplicate` entirely; the column exists only for `SELECT` decoding. See Views and Presenters.
`unscannable`	bool	Excludes the column from row scanning (`Select` / `SelectSingle` leave the field untouched). Set automatically for `tsvector` columns, since their wire format is not meaningful to scan into a Go string.

Security note on the trusted-SQL options. `default`, `check`, `generated` and `conflict` are developer-authored SQL fragments, written into generated DDL/DML with no escaping: a `DEFAULT` clause, a `CHECK(...)` constraint, a `GENERATED ALWAYS AS (...) STORED` expression and an `ON CONFLICT ... DO UPDATE` clause, respectively. They must never be built from end-user input. Every *identifier* pg derives from a tag, in contrast, the table name, every column name and every unique index name, goes through the identifier check described in Identifier Validation before it can reach a query builder.

Primary Keys and Generated Defaults

`primary` (or its shorthand `pk`) marks the column PostgreSQL's primary key. One special case fires automatically: a primary key column of type `UUID` with no explicit `default=`, not marked `nullable`, and with no `ref=` (a referencing primary key, common in one-to-one table splits, is left alone) gets the default `gen_random_uuid()` filled in for you:

```go
ID string `pg:"type=uuid,primary"` // gets default=gen_random_uuid()
```

`Column.IsGeneratedPrimaryUUID()` and `Column.IsGeneratedTimestamp()` (true for a time-typed column defaulting to `clock_timestamp()` or `now()`) both feed `Column.IsGenerated()`, which is what excludes a column from `INSERT` / `UPDATE` argument lists regardless of whether it was

marked `auto` or `generated` explicitly. `CreateSchema` inspects the registered schema for any `UUID` column (via `Schema.HasColumnType`) and emits `CREATE EXTENSION IF NOT EXISTS pgcrypto` when it finds one, since `gen_random_uuid()` lives in that extension.

References and Foreign Keys

The `ref` / `reference` / `references` options add a foreign key. Its value is either a bare column name (assumed to reference the same table the field belongs to) or the full `table(column [action] [deferrable])` form:

```go
type BlogPost struct {
    BaseEntity

    BlogID string `pg:"type=uuid,index,ref=blogs(id cascade deferrable)"`
}
```

The optional action after the column name is one of `NO ACTION`, `CASCADE`, `RESTRICT`, `SET NULL` or `SET DEFAULT` (case-insensitive); it defaults to `CASCADE` when omitted. The optional trailing `deferrable` keyword makes the constraint checkable at the end of a transaction rather than immediately; it cannot be combined with `RESTRICT`, and `ConvertStructToTable` rejects that combination at registration time rather than letting PostgreSQL reject the DDL later.

Indexes and Uniqueness

Three tag options put a constraint or an index on a column, and they compose differently:

- `unique` adds a plain single-column `UNIQUE` constraint (PostgreSQL creates a backing index for it automatically).
- `unique_index=name` adds the column to a named, possibly multi-column unique index: every column that shares the same name becomes part of one composite `UNIQUE` constraint. `unique` and `unique_index` are mutually exclusive on the same column.
- `index` (bare, defaulting to `btree`) or `index=<type>` adds a plain, non-unique index of the given access method (`btree` / `hash` / `gist` / `spgist` / `gin` / `brin`), for columns you query by frequently but that do not need to be unique.

```go
type Customer struct {
    BaseEntity

    CognitoUserID string `pg:"type=uuid,unique_index=customer_unique_idx"`
```

```
    Email string `pg:"type=varchar(255),unique_index=customer_unique_idx"`
    Name  string `pg:"type=varchar(255),index=btree"`
}
```

Here `CognitoUserID` and `Email` together form one composite unique constraint named `customer_unique_idx`, the conflict target `Repository.UpsertSingle` can name explicitly, while `Name` gets an ordinary lookup index with no uniqueness implication.

Struct Embedding

An embedded struct field is flattened into the parent table rather than becoming a column of its own, provided at least one of its own fields carries a `pg` tag; a `time.Time` field, and any field whose tag contains `type=json` / `type=jsonb`, is the exception and is kept as a single column instead. This is how the common “base entity” pattern works:

```go
type BaseEntity struct {
    ID        string    `pg:"type=uuid,primary"`
    CreatedAt time.Time `pg:"type=timestamp,default=clock_timestamp()"`
    UpdatedAt time.Time `pg:"type=timestamp,default=clock_timestamp()"`
}

type Customer struct {
    BaseEntity // flattened: contributes id, created_at, updated_at

    Firstname string `pg:"type=varchar(255)"`
}
```

`Customer` ends up with four columns (`id`, `created_at`, `updated_at`, `firstname`), not one `base_entity` column. A field whose tag is exactly `pg:"-"` is skipped even when it is itself an embedded struct, which lets you opt an embedded type out of flattening entirely.

Column Naming and Name Mappers

When a field’s tag has no explicit `name=`, the column name comes from `desc.ToColumnName`, a package-level function variable. Its default converts the Go field name to snake_case (`ProviderAPIKey` becomes `provider_api_key`; a run of capitals like `ID` or `URL` is treated as one word rather than exploded letter by letter). `SetDefaultColumnNameMapper` replaces it:

```go
// Use the struct field name as-is for column names.
pg.SetDefaultColumnNameMapper(pg.NoColumnNameMapper)
```

```
// Derive column names from each field's json tag, falling back to
// the field name when the json tag is missing or "-".
pg.SetDefaultColumnNameMapper(pg.JSONColumnNameMapper)

// Restore the default snake_case mapper.
pg.SetDefaultColumnNameMapper(nil)
```

The mapper runs once per field, at `Schema.Register` time, before an explicit `name=` tag option (if present) overrides its result. There is a matching pair on the struct/field-name side for tooling that goes the other direction, `desc.ToStructName` and `desc.ToStructFieldName`, which the `gen` subpackage uses when generating Go source from a live database.

The Schema Registry

`Schema` is the registry that connects table names to struct types. `NewSchema()` returns an empty one, ready for `Register` / `MustRegister` calls:

```go
schema := pg.NewSchema()
schema.MustRegister("customers", Customer{})
schema.MustRegister("blogs", Blog{})
```

`Register(tableName, emptyStructValue, opts...)` converts the struct type to a `*desc.Table` (returning a descriptive error on any tag problem, including the identifier-validation failures covered below) and stores it, keyed both by `reflect.Type` and by table name. `MustRegister` is the same call with the error turned into a panic, so a broken tag surfaces at process startup rather than at the first query. Both are safe to call concurrently with each other and with the read methods below, though a `TableFilterFunc` passed as `opts` runs while `Register` holds its internal write lock and must not call back into the same `Schema`.

Once registered, tables are retrieved through:

Method	Signature	Purpose
`Get`	`(typ reflect.Type) (*desc.Table, error)`	Look up by the struct's `reflect.Type`; used internally by `NewRepository[T]` and the non-generic `*DB` CRUD methods.
`GetByTableName`	`(tableName string) (*desc.Table, error)`	Look up by registered table name, a direct map lookup; used by the table-name CRUD covered in Chapter 4.
`Tables`	`(types ...desc.TableType) []*desc.Table`	All registered tables, optionally filtered by type (base table, view, materialized view, presenter), sorted by registration order.

Views and Presenters

Two `TableFilterFunc` values, passed as the third argument to `Register` / `MustRegister`, mark a struct as something other than an ordinary, writable table:

```go
// View: a read-only table backed by a real database VIEW you create
// yourself (e.g. through db.ExecFiles or a migration).
schema.MustRegister("blog_master", BlogMaster{}, pg.View)

// Presenter: not a table or a view at all, just a shape to decode a
// custom SELECT's result columns into.
schema.MustRegister("table_info_presenters", TableInfo{}, pg.Presenter)
```

`View` sets the table's `Type` to `desc.TableTypeView`; `Presenter` sets it to `desc.TableTypePresenter`. Both make `Table.IsReadOnly()` report `true`, which `CreateSchema` and every write path on `Repository[T]` and `*DB` (`Insert`, `Update`, `Delete`, and their table-name-based counterparts) check before doing any work, returning `ErrIsReadOnly` instead of attempting a write against something that is not a real, mutable table:

```go
type TableInfo struct {
    TableName string `pg:"table_name"`
    TableType string `pg:"table_type"`
}

schema.MustRegister("table_info_presenters", TableInfo{}, pg.Presenter)

repo := pg.NewRepository[TableInfo](db)
tables, err := repo.Select(ctx,
    `SELECT table_name, table_type FROM information_schema.tables
     WHERE table_schema = $1;`, db.SearchPath())
```

`TableInfo`'s fields use the bare-tag naming shorthand (`pg:"table_name"` instead of `pg:"name=table_name"`) introduced earlier in this chapter, since a presenter's only job is to line up Go field names with the result columns of whatever `SELECT` you run against it.

Data Types

`desc.DataType` enumerates every PostgreSQL type the tag's `type=` option accepts, matched case-insensitively and by any of its listed aliases (`ParseDataType`). The full catalog, from `desc/data_type.go`:

DataType	Accepted `type=` names
BigInt	`bigint`, `int8`
BigIntArray	`bigint[]`, `int8[]`
BigSerial	`bigserial`, `serial8`
Bit	`bit`
BitVarying	`varbit`, `bit varying`
Boolean	`boolean`, `bool`
Box	`box`
Bytea	`bytea`
Character	`character`, `char`
CharacterArray	`character[]`, `char[]`
CharacterVarying	`varchar`, `character varying`
CharacterVaryingArray	`varchar[]`, `character varying[]`
Cidr	`cidr`
Circle	`circle`
Date	`date`
DoublePrecision	`float8`, `double precision`
Inet	`inet`
Integer	`int`, `int4`, `integer`
IntegerArray	`int[]`, `int4[]`, `integer[]`
IntegerDoubleArray	`int[][]`, `int4[][]`, `integer[][]`
Array	`array`
Interval	`interval`
JSON	`json`
JSONB	`jsonb`
Line	`line`
Lseg	`lseg`
MACAddr	`macaddr`
MACAddr8	`macaddr8`

Money	`money`
Numeric	`numeric`, `decimal`
Path	`path`
PgLSN	`pg_lsn`
Point	`point`
Polygon	`polygon`
Real	`real`, `float4`
SmallInt	`smallint`, `int2`
SmallSerial	`smallserial`, `serial2`
Serial	`serial4`
Text	`text`
TextArray	`text[]`
TextDoubleArray	`text[][]`
Time	`time`, `time without time zone`
TimeTZ	`timetz`, `time with time zone`
Timestamp	`timestamp`, `timestamp without time zone`
TimestampTZ	`timestamptz`, `timestamp with time zone`
TsQuery	`tsquery`
TsVector	`tsvector`
TxIDSnapshot	`txid_snapshot`
UUID	`uuid`
UUIDArray	`uuid[]`
XML	`xml`
Int4Range / Int4MultiRange	`int4range` / `int4multirange`
Int8Range / Int8MultiRange	`int8range` / `int8multirange`
NumRange / NumMultiRange	`numrange` / `nummultirange`
TsRange / TsMultirange	`tsrange` / `tsmultirange`
TsTzRange / TsTzMultiRange	`tstzrange` / `tstzmultirange`
DateRange / DateMultiRange	`daterange` / `datemultirange`

CIText	`citext` (requires the `citext` extension)
HStore	`hstore` (requires the `hstore` extension)

`RegisterDataType` lets you extend this catalog with your own type names; it panics if the `DataType` value or any of the given names already exists, so it is meant to be called once, at package init time.

`type=` is not strictly mandatory: `goTypeToDataType` gives every column a default based on the Go field's type before the tag is even parsed (`string` to `Text`, `int` / `int32` / `int64` to `Integer`, `float32` / `float64` to `Numeric`, `bool` to `Boolean`, `time.Time` to `Timestamp`, and so on), and a `type=` in the tag simply overrides that default. In practice you still write `type=` explicitly for anything the default gets wrong for your schema: a `uuid`-holding `string` field, a `varchar(255)` with an explicit length, a `jsonb` column, or any array or range type. `CIText` and `HStore` also need their PostgreSQL extension created before `CreateSchema` runs (`CREATE EXTENSION IF NOT EXISTS citext` / `hstore`), which `CreateSchema` does automatically whenever a registered column uses one of them.

Identifier Validation

Every table name, resolved column name and unique index name passed through `Schema.Register` is checked against `^[A-Za-z_][A-Za-z0-9_$]*$` before it is stored on the `Table`, and registration fails with a descriptive error the moment an unsafe name is found:

```go
schema.MustRegister("bad;table", SomeStruct{})
// panics: table name: desc: invalid identifier: "bad;table": must
// match ^[A-Za-z_][A-Za-z0-9_$]*$
```

This exists because table names, column names and unique index names are structural parts of a SQL statement, not values: they cannot be passed as `$1`-style bind parameters the way an inserted string can, so the query builders in `desc` write them directly into generated SQL, wrapped in double quotes. A name containing an embedded double quote (`bad"table`), a statement separator (`bad;table`), whitespace, a backtick, or non-ASCII characters could otherwise break out of that quoted position, whether by an honest mistake (a column name pasted with a stray character) or a name built from untrusted input. Rejecting anything outside the bare-identifier charset at registration time, before any query is ever built, closes that class of bug for every query builder at once instead of requiring each one to defend itself individually. Valid names are ordinary: they must start with a letter or underscore and continue with letters, digits, underscores or dollar signs, for example `users`, `_private`, `col_1`, or `MyColumn$2`.

Summary

- A field participates in a table only if it carries a non-empty `pg` tag; `pg:"-"` or no tag skips it entirely.
- The full option set is `name`, `type`, `primary` / `pk`, `identity`, `default`, `unique`, `conflict`, `username`, `password`, `nullable` / `null`, `ref` / `reference` / `references`, `index`, `unique_index`, `check`, `generated`, `auto`, `presenter` and `unscannable`; `default`, `check`, `generated` and `conflict` are raw, developer-authored SQL, never end-user input.
- A `uuid` primary key with no explicit default gets `gen_random_uuid()` automatically; `CreateSchema` adds the `pgcrypto` extension whenever that (or `password`) applies.
- Embedded structs flatten into the parent table (except `time.Time` and JSON-tagged fields); `SetDefaultColumnNameMapper`, `NoColumnNameMapper` and `JSONColumnNameMapper` control how field names become column names.
- `Schema.Register` / `MustRegister` build the table descriptor; `Get`, `GetByTableName` and `Tables` read it back.
- `pg.View` and `pg.Presenter` mark a registered struct read-only; every write path returns `ErrIsReadOnly` for it.
- Table, column and unique index names must match `^[A-Za-z_][A-Za-z0-9_$]*$`, since they are written directly into generated SQL rather than bound as parameters.

Further Reading

- PostgreSQL: Data Types: the canonical reference for every type in the catalog above.
- PostgreSQL: Constraints: `CHECK`, `UNIQUE`, primary keys and foreign keys, straight from the source the `check`, `unique`, `primary` and `ref` tags target.
- PostgreSQL: Generated Columns: the `GENERATED ALWAYS AS (...) STORED` syntax behind the `generated` tag option.
- pgcrypto: the extension providing `gen_random_uuid()`.
- PostgreSQL: Indexes: the access methods (`btree`, `hash`, `gist`, `spgist`, `gin`, `brin`) the `index` tag option chooses between.

CHAPTER 3

Connections and Configuration

A pg `*DB` is a thin wrapper around exactly one `*pgxpool.Pool`, and almost everything about how that pool behaves, how it authenticates, how many connections it keeps open, whether it encrypts traffic, is decided once, at `Open` time. This chapter covers both entry points (`Open` and `OpenPool`), the connection string format pg delegates to pgx to parse, every `ConnectionOption` the library defines, the pool settings you reach through connection string parameters rather than Go code, `search_path`, `sslmode` (what each mode actually protects against, and what it does not), running behind PgBouncer in transaction-pooling mode, and the health and pool-statistics calls you would wire into a readiness endpoint. Every function signature and default value below is checked against `db.go`, `db_options.go`, `db_health.go`, `db_stat.go` and the installed `pgx/v5` / `pgxpool` source.

Open and OpenPool

```go
func Open(ctx context.Context, schema *Schema, connString string,
    opts ...ConnectionOption) (*DB, error)

func OpenPool(schema *Schema, pool *pgxpool.Pool) *DB
```

`Open` parses `connString` with `pgxpool.ParseConfig`, applies every `ConnectionOption` in order, builds a `*pgxpool.Pool` with `pgxpool.NewWithConfig`, and pings it once before returning; a failed ping (or a failed config parse, or a failed option) returns an error and no `*DB`. `OpenPool` skips all of that and wraps a `*pgxpool.Pool` you already built and configured yourself, copying its connection config and resolving `search_path` the same way `Open` does. Reach for `OpenPool` when you need pool construction options `ConnectionOption` does not expose, or when you are sharing one pool across more than one `Schema`.

Both return the same `*DB`:

```go
type DB struct {
```

```
    Pool              *pgxpool.Pool
    ConnectionOptions *pgx.ConnConfig
    // unexported: searchPath, tx, schema, ...
}
```

`ConnectionOptions` is a copy of the pool's `pgx.ConnConfig` as it stood right after `Open` / `OpenPool` ran; it should not be mutated, and changing it after the fact has no effect on the live pool.

Connection Strings

pg does not parse connection strings itself. `Open` hands the string straight to `pgxpool.ParseConfig`, so both forms pgx accepts work unchanged:

```go
// Keyword/value (DSN) form.
connString := "host=localhost port=5432 user=postgres " +
    "password=admin!123 dbname=test_db sslmode=verify-full " +
    "search_path=public pool_max_conns=10"

// URL form.
connString := "postgres://postgres:admin!123@localhost:5432/test_db" +
    "?sslmode=verify-full&search_path=public&pool_max_conns=10"

db, err := pg.Open(context.Background(), schema, connString)
```

Both forms carry the same parameters; which one you reach for is a matter of taste (the URL form composes naturally from a single `DATABASE_URL` environment variable, the keyword form reads well when you are building the string field by field). Both accept every standard `libpq`-style parameter (`host`, `port`, `user`, `password`, `dbname`, `sslmode` and the rest), plus the pg-specific ones pgxpool adds on top for pool sizing (covered below) and the ones covered under Connection Options (`default_query_exec_mode`, `statement_cache_capacity`, `description_cache_capacity`).

Connection Options

A `ConnectionOption` is `func(*pgxpool.Config) error`, applied by `Open` in the order given, after the connection string is parsed and before the pool is built:

```go
type ConnectionOption func(*pgxpool.Config) error
```

Function	Signature	What it sets
`WithLogger`	`(logger tracelog.Logger) ConnectionOption`	Installs a `tracelog.TraceLog` tracer hardcoded to `tracelog.LogLevelTrace`, the most verbose level.
`WithLoggerLevel`	`(logger tracelog.Logger, level tracelog.LogLevel) ConnectionOption`	Same as `WithLogger`, with the level you choose instead of a hardcoded `LogLevelTrace`.
`WithQueryTracer`	`(tracers ...pgx.QueryTracer) ConnectionOption`	Appends one or more `pgx.QueryTracer` implementations (an OpenTelemetry tracer, custom metrics), composing with any tracer already installed via pgx's `multitracer` package.
`WithDefaultQueryExecMode`	`(mode pgx.QueryExecMode) ConnectionOption`	Sets pgx's default query execution mode. Equivalent to `default_query_exec_mode` in the connection string. See PgBouncer.
`WithStatementCacheCapacity`	`(n int) ConnectionOption`	Sets the automatic prepared-statement cache size (`0` disables it). Equivalent to `statement_cache_capacity` in the connection string.
`WithDescriptionCacheCapacity`	`(n int) ConnectionOption`	Sets the statement-description cache size used by pgx's describe exec mode. Equivalent to `description_cache_capacity` in the connection string.

`WithLogger` logs every SQL statement pgx executes together with all of its bind arguments, plaintext passwords passed to `SelectByUsernameAndPassword` included. It exists for local development; do not point it at a production logger. Use `WithLoggerLevel` with a lower level (`tracelog.LogLevelWarn` or `tracelog.LogLevelError`) to reduce verbosity, or `tracelog.LogLevelNone` to disable pgx's tracer logging outright. Note that pgx still logs the statement and its bind arguments for a *failed* query at every level down to and including `tracelog.LogLevelError`; only `LogLevelNone` guarantees sensitive bind arguments never reach the logger.

Two ordering caveats matter when combining options in one `Open` call. `WithLogger` / `WithLoggerLevel` overwrite the pool's tracer slot outright rather than composing with whatever is already there, so pass them *before* `WithQueryTracer` in the option list, the reverse order silently drops the tracer(s) `WithQueryTracer` installed. And `WithQueryTracer` with zero tracers is a no-op that never clears an already-installed tracer.

```go
db, err := pg.Open(ctx, schema, connString,
    pg.WithLoggerLevel(myLogger, tracelog.LogLevelWarn),
    pg.WithQueryTracer(otelTracer),
```

```
    pg.WithStatementCacheCapacity(0), // see PgBouncer, below
)
```

Pool Settings

Pool sizing and lifecycle are not `ConnectionOption` values; they are connection string parameters that `pgxpool.ParseConfig` reads directly (both DSN and URL forms accept them):

Parameter	Meaning	Default
`pool_max_conns`	Maximum number of pool connections.	`4`, or `runtime.NumCPU()` if larger
`pool_min_conns`	Minimum number of connections the pool tries to keep open.	`0`
`pool_min_idle_conns`	Minimum number of idle connections kept ready.	`0`
`pool_max_conn_lifetime`	Maximum age of a connection before it is closed and replaced.	`1h`
`pool_max_conn_lifetime_jitter`	Random jitter subtracted from `pool_max_conn_lifetime`, so connections do not all expire at once.	`0`
`pool_max_conn_idle_time`	Maximum time a connection may sit idle before being closed.	`30m`
`pool_health_check_period`	How often the pool checks idle connections.	`1m`

GO
```
connString := "postgres://postgres:admin!123@localhost:5432/test_db" +
    "?sslmode=verify-full" +
    "&pool_max_conns=25&pool_min_conns=5" +
    "&pool_max_conn_lifetime=1h30m&pool_max_conn_idle_time=15m"
```

Size `pool_max_conns` against what your PostgreSQL server (or PgBouncer in front of it) actually allows, not against how many goroutines your program might run concurrently: PostgreSQL's own `max_connections` is a hard, shared ceiling across every client connected to it, and an oversized pool from one service can starve every other client of the server.

Search Path

`OpenPool` (and therefore `Open`, which delegates to it) resolves the schema's search path from the connection config's `search_path` runtime parameter; if that is empty or unset, it falls back to `desc.DefaultSearchPath`, `"public"` by default (`SetDefaultSearchPath` changes the package-wide default before you call `Open`):

```go
pg.SetDefaultSearchPath("app") // before Open, if you never pass
                               // search_path in the connection string
```

`db.SearchPath()` returns whatever was resolved, and it is what `CreateSchema` uses for its `CREATE SCHEMA IF NOT EXISTS <path>;` statement and what every generated query qualifies table names with. Passing `search_path=` explicitly in the connection string is preferable to relying on the package-wide default whenever more than one `*DB` in the same process might target different schemas.

SSL Modes

`sslmode` is an ordinary connection string parameter, handled entirely by pgx (specifically `pgconn`'s TLS configuration), not by pg. pgx's default, when `sslmode` is omitted, is `prefer`, matching `libpq`'s own default; pg does not override it. Every example in this book (and in `Open`'s own godoc) uses `sslmode=disable` for a local database with no TLS listener at all; that is a development convenience, not a recommendation.

Mode	What it actually guarantees
`disable`	No encryption. Traffic, including the password in the initial handshake, is plaintext on the wire.
`allow`	Tries a plaintext connection first, only upgrading to TLS if the server demands it.
`prefer` (default)	Tries TLS first, falling back to plaintext if the server does not offer it. Encrypted when possible, but silently not when the server (or a network intermediary) does not cooperate.
`require`	Always encrypts, but does not verify the server's certificate against a root CA, so it protects against passive eavesdropping only, not against a man-in-the-middle presenting any certificate. (pgx follows `libpq` here: if `sslrootcert` is also set, `require` behaves like `verify-ca` instead.)
`verify-ca`	Encrypts and verifies the server's certificate chain against a trusted root CA, but does not check that the certificate's hostname matches the server you connected to.
`verify-full`	Encrypts, verifies the certificate chain, and verifies the hostname. The only mode that defends against a man-in-the-middle presenting a validly-signed certificate for a different host.

For a production connection, prefer `sslmode=verify-full` (pointing `sslrootcert` at the certificate authority that signed your database server's certificate, if it is not one your operating system already trusts), or `sslmode=require` at an absolute minimum when `verify-full` is not achievable and you accept the man-in-the-middle exposure that implies. `Open`'s own doc comment carries the same recommendation.

PgBouncer and Transaction Pooling

pgx defaults to `QueryExecModeCacheStatement` (`default_query_exec_mode=cache_statement`), which prepares and caches statements server-side, keyed by SQL text, and reuses that prepared statement on the same physical connection. That assumption breaks under PgBouncer's transaction pooling mode, where a client's logical connection can be handed a *different* physical server connection between transactions: a statement prepared on one backend connection may simply not exist on the next one PgBouncer hands you.

`WithDefaultQueryExecMode` (or `default_query_exec_mode` in the connection string) is how you tell pgx not to rely on server-side prepared statements surviving across queries, and it is the setting that actually fixes the connection: with it set, pgx never populates either its statement cache or its description cache during normal operation.

```go
db, err := pg.Open(ctx, schema, connString,
    pg.WithDefaultQueryExecMode(pgx.QueryExecModeSimpleProtocol),
    pg.WithStatementCacheCapacity(0),
    pg.WithDescriptionCacheCapacity(0),
)
```

`default_query_exec_mode` accepts `cache_statement` (the default), `cache_describe`, `describe_exec`, `exec` or `simple_protocol` as a connection string value. `QueryExecModeSimpleProtocol` (or `QueryExecModeExec`) avoids server-side prepared statements altogether, at the cost of client-side parameter interpolation for the simple protocol case. `WithStatementCacheCapacity(0)` and `WithDescriptionCacheCapacity(0)` are not required for that default behavior, but `*DB`'s `Query` / `Exec` / `QueryRow` pass their `args` straight through to pgx, which recognizes a `pgx.QueryExecMode` value among those args as a per-call override. Setting both capacities to zero (or `statement_cache_capacity=0` / `description_cache_capacity=0` in the connection string) means a call that explicitly opts back into `QueryExecModeCacheStatement` or `QueryExecModeCacheDescribe` fails immediately with an explicit error instead of silently caching a statement handle, which is exactly as unsafe to reuse across a pooled connection swap as a server-prepared one. Set all three together.

Closing a DB

```go
func (db *DB) Close()
```

`Close` closes the underlying `*pgxpool.Pool` (and, transitively, every connection it holds open); call it once, typically with `defer` right after a successful `Open`, for the lifetime of your program or service. A `*DB` returned from `Begin` / `InTransaction` (transaction scoped) shares the same pool and does not need, and should not receive, its own `Close` call.

Health Checks and Pool Statistics

```go
func (db *DB) Ping(ctx context.Context) error
func (db *DB) Health(ctx context.Context) (Health, error)
func (db *DB) PoolStat() PoolStat
```

`Ping` acquires a connection (if necessary) and verifies the database is reachable; it always goes through `db.Pool`, even when `db` is transaction-scoped, since a liveness check is about the database as a whole, not the specific connection a given transaction happens to be pinned to, and it never touches or invalidates an in-flight transaction.

`Health` builds on `Ping`: it pings, then reads the server version (`DB.GetVersion`) and a `PoolStat` snapshot, returning both in one `Health` value, or the ping/version error if the database is unreachable:

```go
type Health struct {
    ServerVersion string   `json:"server_version"`
    Pool          PoolStat `json:"pool"`
}
```

`PoolStat` mirrors `pgxpool`'s own `*pgxpool.Stat`, translated into a plain, JSON-serializable struct: `AcquireCount`, `AcquireDuration`, `AcquiredConns`, `CanceledAcquireCount`, `ConstructingConns`, `EmptyAcquireCount`, `IdleConns`, `MaxConns` and `TotalConns`. Wiring `Health` (or `PoolStat` alone) into a `/healthz` or `/readyz` endpoint gives you both the database's reachability and, over time, an early signal that `pool_max_conns` is undersized (`EmptyAcquireCount` climbing, `AcquiredConns` pinned at `MaxConns`) before requests start timing out waiting for a connection.

Summary

- `Open(ctx, schema, connString, opts...)` parses a connection string with `pgxpool.ParseConfig`, applies every `ConnectionOption`, builds a pool and pings it; `OpenPool(schema, pool)` wraps a pool you already built yourself.
- Both DSN (`key=value ...`) and URL (`postgres://...`) connection string forms work; pg does not parse them itself.
- `WithLogger` / `WithLoggerLevel` install a tracer, `WithQueryTracer` composes additional `pgx.QueryTracer`s, and `WithDefaultQueryExecMode` / `WithStatementCacheCapacity` / `WithDescriptionCacheCapacity` mirror connection string parameters of the same purpose.
- `pool_max_conns` (default 4, or `NumCPU()` if larger), `pool_min_conns`, `pool_min_idle_conns`, `pool_max_conn_lifetime` (`_jitter`), `pool_max_conn_idle_time` and `pool_health_check_period` are connection string parameters, not Go options.
- `search_path` resolves from the connection string, falling back to `desc.DefaultSearchPath` (`"public"`); `db.SearchPath()` reads it back.
- Prefer `sslmode=verify-full` in production; `require` still encrypts but does not defend against a man-in-the-middle, and `disable` / `allow` / `prefer` may hand you an unencrypted connection outright.
- Behind PgBouncer's transaction pooling, pair `WithDefaultQueryExecMode(pgx.QueryExecModeSimpleProtocol)` (or `QueryExecModeExec`) with `WithStatementCacheCapacity(0)`.
- `db.Close()` closes the pool; `db.Ping`, `db.Health` and `db.PoolStat()` are the building blocks for a readiness endpoint.

Further Reading

- pgxpool godoc: `ParseConfig`'s full parameter list, including every pool setting in this chapter.
- PostgreSQL: Connection Strings: the canonical DSN/URL syntax pgx implements.
- PostgreSQL: SSL Support: the authoritative description of what each `sslmode` value protects against.
- PgBouncer: Pooling Modes: the transaction pooling behavior that makes server-side prepared statements unsafe.
- pgx QueryExecMode godoc: the exec modes `WithDefaultQueryExecMode` chooses between.

CHAPTER 4

Repositories and CRUD

`Repository[T]` is where most application code meets pg day to day: one generic type, bound to one registered struct, exposing reading, inserting, upserting, updating, deleting and counting as plain Go methods with no query string required for the common cases. `*DB` itself mirrors nearly all of it without the type parameter, for code that does not know `T` at compile time, plus a table-name-based CRUD surface added alongside it that validates every table and column name against the registered `Schema` before any SQL is built. This chapter documents every method on both, and the safety property that makes the table-name API sound. Every signature below is read directly out of `repository.go`, `db_repository.go` and `db_crud.go`.

Repository[T]: The Type-Safe Entry Point

```go
func NewRepository[T any](db *DB) *Repository[T]
```

`NewRepository[T]` looks up `T`'s table definition in `db`'s `Schema` (the same `Schema.Get` lookup by `reflect.Type` that Chapter 2 covers) and caches it on the returned `*Repository[T]`. It panics if `T` was never registered: this is deliberate, the same way `MustRegister` panics on a bad tag. A typo between the struct you registered and the struct you build a repository for is a startup-time failure, not a runtime error the first time a handler runs:

```go
customers := pg.NewRepository[Customer](db) // panics if Customer
                                            // was never registered
```

`repo.DB()` returns the underlying `*DB`, and `repo.Table()` returns the cached `*desc.Table` (read-only; do not mutate it). `repo.IsReadOnly()` reports whether the underlying table is a view or presenter (see Views and Presenters), which every write method below checks up front, returning `ErrIsReadOnly` instead of attempting SQL against something that is not a writable table. `repo.InTransaction(ctx, fn)` runs `fn` with a transaction-scoped repository, joining an already-open transaction instead of nesting one, exactly as `DB.InTransaction` does.

Reading Rows

Method	Signature	Behavior
`Select`	`(ctx, query string, args ...any) ([]T, error)`	Runs `query`, scans every result row into a `T` by column name.
`SelectSingle`	`(ctx, query string, args ...any) (T, error)`	Same, for a query expected to return one row; returns `ErrNoRows` for zero.
`SelectByID`	`(ctx, id any) (T, error)`	`SELECT * FROM <table> WHERE <primary key> = $1 LIMIT 1`, built from the table's registered primary key column; no query string needed.
`SelectByUsernameAndPassword`	`(ctx, username, plainPassword string) (T, error)`	Matches the columns tagged `username` / `password`; the password side is compared with PostgreSQL's `crypt()`, reusing the stored value's own salt.
`Exists`	`(ctx, value T) (bool, error)`	Reports whether a row matches `value`'s non-zero fields.

`Select` / `SelectSingle` take caller-supplied SQL, exactly like `database/sql`'s `Query` / `QueryRow`, with one difference: the scan step is automatic. Result columns are matched to `T`'s fields by column name (case-insensitively), not by position, so the query need not select columns in struct-declaration order, and can select a subset of them:

```go
recent, err := customers.Select(ctx,
    `SELECT * FROM customers WHERE created_at > $1
     ORDER BY created_at DESC;`, since)

one, err := customers.SelectSingle(ctx,
    `SELECT * FROM customers WHERE email = $1;`, email)

byID, err := customers.SelectByID(ctx, customerID)
```

`SelectByUsernameAndPassword` only works once a `username`-tagged and a `password`-tagged column exist on `T` (see Chapter 2) and, for encrypted passwords, once `Schema.HandlePassword` is configured.

Inserting Rows

Method	Signature	Behavior
`Insert`	`(ctx, values ...T) error`	Zero values: no-op. One value: delegates to `InsertSingle`. Multiple: delegates to `InsertMany`.
`InsertSingle`	`(ctx, value T, idPtr any) error`	Inserts one row. A non-nil `idPtr` adds a `RETURNING <primary key>` and scans it back.
`InsertMany`	`(ctx, values ...T) error`	Bulk-inserts via multi-row `VALUES` statements, batched, inside one transaction.

`InsertMany` batches rows into multi-row `INSERT` statements of up to `desc.DefaultInsertBatchSize` (500) rows per statement, rather than one round trip per row, which is what makes inserting a few thousand rows fast instead of a multi-second (or multi-minute) operation. The whole call runs inside a single `InTransaction`, so a failure in a later batch rolls back every earlier batch too. Because PostgreSQL caps a single statement at 65535 bind parameters, a wide table (many insertable columns) shrinks the effective batch size below 500 to stay under that ceiling: the exact formula is `min(500, 65535 / Table.NumInsertableColumns())`. Per-row semantics match `InsertSingle`: a zero-valued field on a column that carries a database default still emits the SQL `DEFAULT` keyword instead of a bind parameter, so `clock_timestamp()`, `gen_random_uuid()` and the like still fire per row exactly as they would for a single insert.

```go
err := customers.InsertSingle(ctx, newCustomer, &newCustomer.ID)

err := customers.Insert(ctx, c1, c2, c3) // -> InsertMany
```

Upserting Rows

Method	Signature	Behavior
`Upsert`	`(ctx, forceOnConflictExpr string, values ...T) error`	Zero: no-op. One: delegates to `UpsertSingle`. Multiple: delegates to `UpsertMany`.
`UpsertSingle`	`(ctx, forceOnConflictExpr string, value T, idPtr any) error`	Inserts or updates one row.
`UpsertMany`	`(ctx, forceOnConflictExpr string, values ...T) error`	Bulk `INSERT ... ON CONFLICT DO UPDATE`, batched the same way `InsertMany` is.

`forceOnConflictExpr` names the `ON CONFLICT` target: an empty string uses the struct's own declared conflict target (its `unique_index` or `unique` tag), a non-empty string usually names a specific unique index or column expression to target for a full `DO UPDATE SET` instead. The

exported constant `pg.DoNothing` (`"DO NOTHING"`) is a special case of that non-empty form: it keeps the same tag-derived target the empty-string case would use, but forces `DO NOTHING` instead of `DO UPDATE`, for the common "insert if absent, otherwise leave the existing row alone" case. When the struct declares no conflict-target tag at all, the result is a target-less `ON CONFLICT DO NOTHING`, valid PostgreSQL that fires against a conflict on any unique constraint on the table:

```go
err := customers.UpsertSingle(ctx, "customer_unique_idx",
    customerToUpsert, &customerToUpsert.ID)

err := customers.Upsert(ctx, pg.DoNothing, c1, c2, c3)
```

A `DO NOTHING` action, whether reached via `pg.DoNothing` or the `conflict=DO NOTHING` struct tag (see Chapter 7), still appends `RETURNING` on the single-row path, so `UpsertSingle` with a non-nil `idPtr` behaves the way you would expect: an insert that did not conflict populates `idPtr`, and a row skipped by the conflict returns `ErrNoRows`. Those two outcomes are distinguishable, which is the point. The bulk path (`UpsertMany`) never appends `RETURNING`, so it reports no identifiers either way. See Chapter 7 for `Repository.InsertSingleOnConflict` with `OnConflict{DoNothing: true}`, which gives the same contract with an explicit conflict target.

`UpsertMany` shares `InsertMany`'s transaction, batching and per-row `DEFAULT` semantics; see Inserting Rows above for the details.

Updating Rows

Method	Signature	Behavior
`Update`	`(ctx, values ...T) (int64, error)`	Updates every column except the primary key, by primary key value. Equivalent to `UpdateOnlyColumns(ctx, nil, values...)`.
`UpdateOnlyColumns`	`(ctx, columnsToUpdate []string, values ...T) (int64, error)`	Updates only the named columns.
`UpdateExceptColumns`	`(ctx, columnsToExcept []string, values ...T) (int64, error)`	Updates every column except the named ones.
`UpdateOnlyColumnsReportNoRows`	`(ctx, columnsToUpdate []string, values ...T) (bool, error)`	Same as `UpdateOnlyColumns`, but returns `(false, nil)` instead of `(0, nil)`-with-no-error when no row matched, by returning `ErrNoRows` internally and swallowing it into a boolean.

All four match rows by primary key and return the number of rows affected (or, for the last one, whether exactly one row was). Passing more than one value runs every update inside one transaction, and the returned count is the sum across all of them:

```go
updated, err := customers.UpdateOnlyColumns(ctx,
    []string{"cognito_user_id"}, customerWithNewID)

updated, err := customers.UpdateExceptColumns(ctx,
    []string{"created_at"}, customerWithEverythingElseChanged)
```

`UpdateOnlyColumns` is also how you write a column back to its Go zero value on purpose, something a naive "skip zero fields" update strategy could never do: passing `[]string{"username"}` with `Username: ""` on the struct genuinely sets the column to `''`, because the column list, not the struct's zero-ness, decides what gets written.

Deleting and Duplicating Rows

Method	Signature	Behavior
`Delete`	`(ctx, values ...T) (int64, error)`	Deletes rows matching the given values' primary keys; returns the count removed.
`DeleteByID`	`(ctx, id any) (bool, error)`	Deletes one row by primary key value directly, without needing a whole `T`.
`Duplicate`	`(ctx, id any, newIDPtr any) error`	Duplicates the row with primary key `id` via `INSERT ... SELECT`; `newIDPtr`, if non-nil, receives the new row's primary key.

```go
deleted, err := customers.Delete(ctx, existingCustomer)
removed, err := customers.DeleteByID(ctx, customerID)

var newID string
err := customers.Duplicate(ctx, customerID, &newID)
```

`Duplicate` builds its `INSERT` from a `SELECT` of the existing row rather than reading the row into Go and re-inserting it, so it never round-trips the row's data through your process.

Counting Rows

```go
func (repo *Repository[T]) Count(ctx context.Context,
    query string, args ...any) (int64, error)
```

`Count` runs a caller-supplied query, typically a `COUNT(*)` or other aggregate, and scans the single resulting value into an `int64`. A query that yields zero rows (a `COUNT` wrapped in a `GROUP BY` that matched nothing, for instance) counts as zero: `Count` swallows `ErrNoRows` and returns `(0, nil)` rather than forcing every caller to special-case it.

```go
n, err := customers.Count(ctx,
    `SELECT COUNT(*) FROM customers WHERE created_at > $1;`, since)
```

The Non-Generic *DB Mirror

Every `Repository[T]` method above has a namesake on `*DB` itself, for code paths that do not carry a compile-time `T`, generic helper functions, code generation, or handlers that dispatch on a runtime type. The `*DB` versions resolve the table definition from the argument's own `reflect.Type` via `Schema.Get`, the exact mechanism `Repository[T]` uses internally, just without a cached lookup:

Method	Signature
`SelectByID`	`(ctx, destPtr any, id any) error`
`SelectByUsernameAndPassword`	`(ctx, destPtr any, username, plainPassword string) error`
`Exists`	`(ctx, value any) (bool, error)`
`Insert`	`(ctx, values ...any) error`
`InsertSingle`	`(ctx, value any, idPtr any) error`
`Upsert`	`(ctx, forceOnConflictExpr string, values ...any) error`
`UpsertSingle`	`(ctx, forceOnConflictExpr string, value any, idPtr any) error`
`Update`	`(ctx, values ...any) (int64, error)`
`UpdateOnlyColumns`	`(ctx, columnsToUpdate []string, values ...any) (int64, error)`
`UpdateExceptColumns`	`(ctx, columnsToExcept []string, values ...any) (int64, error)`
`Delete`	`(ctx, values ...any) (int64, error)`
`Duplicate`	`(ctx, value any, idPtr any) error`
`Mutate`	`(ctx, query string, args ...any) (int64, error)`
`MutateSingle`	`(ctx, query string, args ...any) (bool, error)`

Across the whole conflict-handling family the conflict specification comes first, immediately after `ctx`: `Upsert`, `UpsertMany`, `UpsertSingle`, `InsertOnConflict` and `InsertSingleOnConflict` all read the same way on `*DB` and on `Repository[T]`. The variadic methods have no choice, since `values ...T` must be last, and the single-row methods follow them so that one rule covers every case.

One method is worth reading carefully rather than assuming it mirrors its `Repository[T]` counterpart exactly.

`DB.Select` is not `Repository[T].Select` without the type parameter, it takes a scanner callback instead of returning `[]T`, because `*DB` has no `T` to scan into automatically:

```go
func (db *DB) Select(ctx context.Context,
    scannerFunc func(Rows) error, query string, args ...any) error
```

```go
var names []string
err := db.Select(ctx, func(rows pg.Rows) error {
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
            return err
        }
        names = append(names, name)
    }
    return nil
}, `SELECT firstname FROM customers;`)
```

For the common “scan a single column into a slice” or “scan one ad hoc row” shapes, the package-level generic helpers `QuerySlice[T]`, `QueryMap[K, V]` and `QueryFunc[T]` (in `common.go`) usually read better than a hand-written `DB.Select` callback; they are covered in Chapter 5.

Table-Name CRUD on *DB

`db_crud.go` adds a second, table-name-based CRUD surface on `*DB`: `DeleteByID`, `DeleteBy`, `ExistsBy` and `CountBy` all take the target table by its registered *name* (a string) rather than a typed Go value, which is convenient for generic, table-agnostic code, an admin tool, a generic REST layer, code driven by configuration, at the cost of losing the compile-time type safety `Repository[T]` gives you.

Method	Signature
`DeleteByID`	`(ctx, tableName string, id any) (bool, error)`

`DeleteBy`	`(ctx, tableName string, colValPairs ...any) (int64, error)`
`ExistsBy`	`(ctx, tableName string, colValPairs ...any) (bool, error)`
`CountBy`	`(ctx, tableName string, colValPairs ...any) (int64, error)`
`SelectSingle`	`(ctx, destPtr any, query string, args ...any) error`

`colValPairs` is a flat `"col1", v1, "col2", v2, ...` argument list, ANDed together into a `WHERE` clause; passing none deletes/matches every row of the table. This shape reopens exactly the risk an earlier, removed version of this API had: a table or column name coming straight from a caller-controlled string, concatenated into SQL, is an injection vector. Every method here closes that hole the same way, and it is worth naming precisely, since it is what makes this API safe to expose to less-trusted callers than the rest of the library:

1. `tableName` is resolved through `Schema.GetByTableName`. An unknown table name returns a descriptive error before any SQL is built at all.
2. Every column name in `colValPairs` is resolved through the returned `*desc.Table` (`GetColumnByName`). An unknown column likewise returns a descriptive error instead of reaching a query.
3. Only names that survive both checks, meaning names that already exist in the registered `Schema`, are quoted with `QuoteIdentifier` and interpolated into the generated SQL; values are always passed as `$N` bind parameters, never interpolated.
4. `DeleteByID` and `DeleteBy` additionally return `ErrIsReadOnly` for a view, materialized view or presenter table, matching `Repository.Delete`'s behavior for read-only tables.

```go
removed, err := db.DeleteByID(ctx, "customers", customerID)

deleted, err := db.DeleteBy(ctx, "customers",
    "cognito_user_id", oldCognitoID)

exists, err := db.ExistsBy(ctx, "customers", "email", email)

n, err := db.CountBy(ctx, "customers", "name", "Ada")
```

`SelectSingle` on `*DB` is a different shape from the four above: it resolves its table not from a `tableName` string but from `destPtr`'s own registered Go type (the same `Schema.Get(reflect.TypeOf(destPtr))` mechanism `SelectByID` uses), and its `query` is caller-supplied SQL, not built from a table name at all. It exists to let table-agnostic code scan one row into any registered struct type with the same column-by-name convenience `Repository[T].SelectSingle` gives a typed repository, but, unlike the other four methods in this section, it does not parse or validate the query string itself, keeping it safe is the caller's responsibility, exactly as with `Repository[T].SelectSingle`:

```go
var c Customer
err := db.SelectSingle(ctx, &c,
    `SELECT * FROM customers WHERE email = $1;`, email)
```

Updating JSONB Columns

```go
func (db *DB) UpdateJSONB(ctx context.Context,
    tableName, columnName, rowID string,
    values map[string]any, fieldsToUpdate []string) (int64, error)
```

`UpdateJSONB` updates a single `jsonb` column, in full or in part, without requiring you to read the row, unmarshal the column, modify it in Go and write the whole document back. `tableName` and `columnName` are resolved against the schema exactly like the table-name CRUD above (an unknown table or column returns a descriptive error before any SQL is built, and the resolved names are quoted), and the table's primary key column (required) is used to locate the row by `rowID`.

When `fieldsToUpdate` is empty, `values` replaces the column outright (`SET col = $1`). When `fieldsToUpdate` is non-empty, every key in `values` is required to also appear in `fieldsToUpdate` (a mismatch is a descriptive error, not a silent partial write), and the update becomes a shallow JSONB merge, `SET col = col || $1`, so keys not present in `values` are left untouched in the stored document:

```go
// Full replace.
n, err := db.UpdateJSONB(ctx, "customers", "preferences",
    customerID, allPreferences, nil)

// Partial merge: only "theme" and "locale" change.
n, err := db.UpdateJSONB(ctx, "customers", "preferences", customerID,
    map[string]any{"theme": "dark", "locale": "en"},
    []string{"theme", "locale"})
```

Summary

- `pg.NewRepository[T](db)` panics on an unregistered `T`, catching a schema/repository mismatch at startup rather than at query time.
- Reading: `Select`, `SelectSingle`, `SelectByID`, `SelectByUsernameAndPassword`, `Exists`. Writing: `Insert` / `InsertSingle` / `InsertMany`, `Upsert` / `UpsertSingle` / `UpsertMany` (batched,

one transaction, capped by `desc.DefaultInsertBatchSize` and PostgreSQL's 65535-bind-parameter limit).

- Updating: `Update`, `UpdateOnlyColumns`, `UpdateExceptColumns`, `UpdateOnlyColumnsReportNoRows`, all matched by primary key. Deleting/duplicating: `Delete`, `DeleteByID`, `Duplicate`. Counting: `Count` (treats `ErrNoRows` as zero).
- `*DB` mirrors nearly every `Repository[T]` method without the type parameter, resolving the table from the argument's own registered type; the conflict specification is always the first argument after `ctx` on both, and `DB.Select` takes a scanner callback rather than returning a slice.
- `DeleteByID`, `DeleteBy`, `ExistsBy`, `CountBy` and `SelectSingle` on `*DB` take a table by name; the first four validate the table (via `Schema.GetByTableName`) and every column (via `GetColumnByName`) before building SQL, quoting resolved names and binding values as `$N` parameters, which is what makes them safe against a caller-controlled table or column name.
- `UpdateJSONB` updates a `jsonb` column in full or, given `fieldsToUpdate`, as a shallow merge (`col || $1`) that leaves untouched keys alone.

Further Reading

- PostgreSQL: INSERT: the `ON CONFLICT` clause `Upsert` / `UpsertSingle` / `UpsertMany` generate.
- PostgreSQL: The JSONB Type: the `||` concatenation operator `UpdateJSONB`'s partial-update path relies on.
- pgx godoc: Rows: the interface behind pg's `Rows` alias, relevant to `DB.Select`'s scanner callback.
- pkg.go.dev/github.com/kataras/pg: the generated godoc reference, useful for cross-checking a method signature against the currently installed version.

CHAPTER 5

Querying and Scanning

Raw SQL is first-class in pg. A `*DB` and a `*Repository[T]` both expose `Query`, `QueryRow`, `QueryBoolean`, `Exec`, `Mutate`, `MutateSingle` and `Count` directly, so you never have to fight the library to write the query you actually want. On top of that, a small set of generic helpers in `common.go` (`QuerySlice`, `QueryTwoSlices`, `QueryMap`, `QuerySingle`, `QueryFunc`) remove the boilerplate of looping over `pgx.Rows` for the common shapes: a single column, two columns, or a hand-written scan function. When the result shape is a struct, `Repository[T].Select` scans through the table's registered descriptor, and `QueryStructs` / `QueryStruct` / `ScanStructs` do the same for a struct that was never registered at all, including one that embeds a JSON-decoded field from a `to_jsonb(x.*)` projection. Every signature and behavior described here was verified against the library source in `C:/github/pg`, primarily `common.go`, `scan.go`, `desc/scanner.go` and `desc/loose_table.go`.

Background

If you have used `database/sql` before, skip to Raw SQL on DB and Repository; the shapes below will already be familiar.

pg is built directly on `github.com/jackc/pgx/v5`, not on `database/sql`. `Row` and `Rows` in this library are type aliases for `pgx.Row` and `pgx.Rows`, and every query method ultimately calls `pgx.Pool.Query` / `QueryRow` / `Exec` (or the transaction's equivalent, when the `*DB` is transactional). This matters for one practical reason: a `pgx.Row` returned by `QueryRow` defers its error until you call `Scan`, exactly like `database/sql`, and a `pgx.Rows` returned by `Query` must be closed, either explicitly with `rows.Close()` or by letting one of the scanning helpers in this chapter close it for you. Every helper described below already does that housekeeping; you only need to manage `rows.Close()` yourself when you call `db.Query` or `repo.Query` directly and scan the result by hand.

Raw SQL on DB and Repository

A repository does not hide SQL from you. `Repository[T]` wraps a `*DB` and a cached `*desc.Table`, and every one of its raw-SQL methods is a one-line delegation to the same method on `*DB`:

Method	On `*DB`	On `Repository[T]`	Returns
`Query`	yes	yes	`(Rows, error)`
`QueryRow`	yes	yes	`Row`
`QueryBoolean`	yes	yes	`(bool, error)`
`Exec`	yes	yes	`(pgconn.CommandTag, error)`
`Mutate`	yes	yes	`(int64, error)`
`MutateSingle`	yes	yes	`(bool, error)`
`Count`	yes	yes	`(int64, error)`
`Select`	yes (callback form)	no (use `Select(ctx, query, args...) ([]T, error)` instead)	see below

`Query` and `QueryRow` mirror `database/sql`'s naming: `Query` returns `Rows` for a result set of any size, `QueryRow` returns a `Row` whose `Scan` reports `ErrNoRows` if the query produced nothing. `Exec` runs a statement that returns no rows (an `UPDATE`, `DELETE`, `INSERT` without `RETURNING`, or DDL) and hands back a `pgconn.CommandTag`, pgx's wrapper around the server's command tag, from which you can read `RowsAffected()` yourself. `Mutate` and `MutateSingle` save you that one extra call: `Mutate` runs `Exec` and returns `tag.RowsAffected()` directly as an `int64`, and `MutateSingle` wraps `Mutate` and returns `rowsAffected > 0` as a `bool` for the common "did exactly one row change" check. `QueryBoolean` is the same idea for a query whose entire result is a single boolean column, typically a `SELECT EXISTS (...)`. `Count` is built for a single numeric result such as `COUNT(*)`, with one deliberate accommodation: a query that legitimately produces no row at all (for example, a `COUNT(*) ... GROUP BY` over an empty table, which unlike a bare `COUNT(*)` yields zero rows rather than one row with value zero) still returns `(0, nil)` instead of forcing you to special-case `ErrNoRows` at every call site.

```go
n, err := repo.Count(ctx, "SELECT COUNT(*) FROM customers")

exists, err := repo.QueryBoolean(ctx,
    "SELECT EXISTS (SELECT 1 FROM customers WHERE email = $1)",
    email)
```

```
affected, err := repo.Mutate(ctx,
    "UPDATE customers SET name = $1 WHERE id = $2", name, id)
```

`Repository[T]` adds one method `*DB` does not have in this raw-SQL group: `Select(ctx, query, args...) ([]T, error)`, which scans every row into `T` using the repository's own table descriptor. `*DB` has a `Select` too, but with a different, callback-based signature: `Select( ctx, scannerFunc func(Rows) error, query string, args ...any) error`. It runs `query`, hands the resulting `Rows` to `scannerFunc`, and closes them afterward regardless of what `scannerFunc` returns. Reach for `DB.Select` when you want to drive `rows.Next()` / `rows.Scan()` yourself without worrying about closing the rows on every exit path; reach for `QuerySlice`, `QueryFunc` or `QueryStructs` (below) when the result fits one of their shapes, since they save you from writing that loop at all.

The Generic Query Helpers

`common.go` **exists because most result sets are shaped like a list of one thing, not a list of structs.** Five package-level generic functions cover the common non-struct shapes, all sharing the same convention: a query yielding no rows returns an empty, non-nil result and a nil error, never `ErrNoRows`.

`QuerySlice[T any](ctx, db, query, args...) ([]T, error)` scans a single-column result directly into a `[]T`, where `T` is any type `rows.Scan` accepts on its own (`string`, `int64`, `time.Time`, a `uuid.UUID`, and so on):

```
names, err := pg.QuerySlice[string](ctx, db,
    "SELECT name FROM customers WHERE active;")
```
GO

When `T` is `string`, `QuerySlice` silently drops every empty-string result from the returned list; this is a documented quirk, not a general "skip zero values" rule, and it does not apply to `QueryIter` (Chapter 8) or any other helper.

`QueryTwoSlices[T, V any](ctx, db, query, args...) ([]T, []V, error)` is the same idea for a two-column query, returning two parallel slices instead of a slice of pairs.

`QueryMap[K comparable, V any](ctx, db, query, args...) (map[K]V, error)` scans a two-column query into a map keyed by the first column. A later duplicate key overwrites an earlier one, so if you need a specific row to win, order the query accordingly (an `ORDER BY` that places the winning row last). A no-rows query returns an empty, non-nil map, matching the rest of the family.

`QuerySingle[T any](ctx, db, query, args...) (T, error)` is the single-row, single-column case, built on `QueryRow(...).Scan(&entry)`. It does surface `ErrNoRows` (there is no list to fall back to empty), so check for it with `pg.IsErrNoRows(err)` when the caller needs to tell "no row" apart from a real failure.

The last member of the family handles everything QuerySlice cannot express: a handful of columns combined into an ad hoc type. `ScanFunc[T any]` is `func(rows Rows) (T, error)`, and `QueryFunc[T any](ctx, db, scan ScanFunc[T], query, args...) ([]T, error)` calls it once per row:

```go
type nameAndCount struct {
    Name  string
    Count int64
}

rows, err := pg.QueryFunc(ctx, db, func(rows pg.Rows) (nameAndCount, error) {
    var nc nameAndCount
    err := rows.Scan(&nc.Name, &nc.Count)
    return nc, err
}, "SELECT name, COUNT(*) FROM customers GROUP BY name ORDER BY name;")
```

Every one of these five functions is a thin loop over `db.Query` followed by `rows.Next()` /your scan step/ `rows.Err()`, with the `rows.Close()` handled for you via `defer`. Pick `QueryFunc` only when the row genuinely does not fit a registered struct or a `QueryStructs` / `ScanStructs` ad hoc type (see below); those cover the struct case with less code than a hand-written `ScanFunc`.

Scanning Into Registered Structs

When `T` was registered with `Schema.Register` / `MustRegister`, `Repository[T].Select` and `Repository[T].SelectSingle` scan by matching each result column's name, case-insensitively, against the struct's descriptor:

```go
customers, err := repo.Select(ctx,
    "SELECT * FROM customers WHERE created_at > $1", since)

customer, err := repo.SelectSingle(ctx,
    "SELECT * FROM customers WHERE id = $1", id)
```

Under the hood both delegate to `desc.RowsToStruct[T]` and `desc.RowToStruct[T]`, which take the repository's cached `*desc.Table` and a `pgx.Rows`. A third exported function, `desc.ConvertRowsToStruct (td *desc.Table, rows pgx.Rows, valuePtr any) error`, scans

exactly one already-positioned row (after a successful `rows.Next()`) into an existing pointer, and is what `DB.SelectByID` and `Repository[T].SelectIter` (see Chapter 8) build on when a `[]T` or a `T` return value is not what the caller needs.

Column resolution is O(1) per column: `desc.RowsToStruct` builds a case-insensitive lookup of the table's columns once, before the row loop, rather than re-scanning `td.Columns` for every column of every row. A column with no matching struct field is routed to a no-op scanner and silently ignored, unless the table descriptor was marked `Strict`, in which case an unmapped column is an error. A registered table also gets type-aware scan targets you do not have to think about: a nullable `UUID` / `Text` / `CharacterVarying` column tolerates an unexpected `NULL` into a plain (non-pointer) string field, a nullable `JSONB` / `JSON` column that is not itself a `sql.Scanner` is decoded with `encoding/json` automatically, and a `password` column (see Chapter 7) is routed through the table's `PasswordHandler` if one is configured.

Ad Hoc Read Models: QueryStructs, QueryStruct, ScanStructs

`Repository[T]` requires `T` to be registered; `NewRepository[T]` panics otherwise. Three package-level functions in `scan.go` exist for exactly the opposite situation: scanning a query's result into a struct that was never registered at all, typically a join result or a presenter shape assembled just for one query.

```go
type OrderWithCustomer struct {
    ID       int64
    Total    float64
    Customer *Customer // populated from to_jsonb(c.*) AS customer.
}

rows, err := pg.QueryStructs[OrderWithCustomer](ctx, db, `
    SELECT o.id, o.total, to_jsonb(c.*) AS customer
    FROM orders o JOIN customers c ON c.id = o.customer_id`)
```

`QueryStructs[T any](ctx, db, query, args...) ([]T, error)` runs `query` and scans every row into `T`. `QueryStruct[T any](ctx, db, query, args...) (T, error)` is the single-row counterpart and reports `ErrNoRows` when the query yields nothing. `ScanStructs[T any](rows Rows) ([]T, error)` scans an already-open `Rows` value (from `db.Query`, `repo.Query`, or a transaction's `Query`) the same way, and closes it before returning.

All three resolve `T`'s descriptor the same way: `db`'s `Schema` is consulted first (the exact registered descriptor Repository[T] would use, including password handling), and only when `T` is not registered does the resolution fall back to `desc.LooseTable`, a schema-independent descriptor built purely from `T`'s field tags and names via reflection. `ScanStructs` has no `*DB` to

consult, so it always takes the `LooseTable` path, even for a type that happens to be registered elsewhere; prefer `QueryStructs` / `QueryStruct` (or `Repository[T].Select`) when a `*DB` is available and `T` is registered. `LooseTable`'s result is cached per `reflect.Type`, so repeated calls for the same `T` pay the reflection cost only once.

A query yielding no rows returns an empty, non-nil slice and a nil error from `QueryStructs` / `ScanStructs`, matching `Repository[T] .Select`'s convention.

Column Name Precedence in LooseTable

`desc.LooseTable` does not require a `pg` struct tag on every field the way the registered-table path does; a field with no tag at all is still scanned. For each exported field, the column name is resolved in this order:

1. The `pg` tag's `name=` option, e.g. `pg:"name=food_id"` maps to `food_id`. Every other option in the tag is ignored by `LooseTable`, including a bare, comma-less tag such as `pg:"id"` (a naming shorthand the registered-table path understands, but `LooseTable` does not); such a field falls through to its `json` tag or snake-cased field name instead.
2. The `json` tag's name, e.g. `json:"foodId,omitempty"` maps to `foodId`. A bare `json:"-"` skips the field entirely, following `encoding/json`'s own convention; `json:"-,"` (with the trailing comma) is instead read as the literal column name `-`.
3. `SnakeCase(field name)`, e.g. `FoodID` maps to `food_id`.

A field tagged `pg:"-"` is skipped outright, checked before the `json` tag. Unexported fields are always skipped. Embedded (anonymous) struct fields are treated as one ordinary field, not flattened: an embedded struct becomes exactly one column of its own (JSON-wrapped, per the rules in the next section), rather than having its own fields promoted to the top level the way a registered table's tagged nested struct is flattened. A type that needs promoted embedded fields must be registered with `Schema.Register` instead.

Every column `LooseTable` produces is marked nullable, since it has no schema to consult for a field's real nullability; this is why a plain (non-pointer) text-like field on an ad hoc type tolerates an unexpected `NULL` the same way a registered table's nullable column does. The resulting descriptor is also non-strict, so a result column with no matching field is ignored rather than causing an error, which is what lets a join or presenter query freely select extra columns the Go type does not care about.

What Gets JSON-Decoded Automatically, and What Does Not

A struct, map or slice field on an ad hoc type is JSON-decoded without a manual `json.Unmarshal` call, whether the source column is a genuine `jsonb` / `json` column or a `to_jsonb(...)` / `row_to_json(...)` projection in the `SELECT` list. Specifically: a field whose type, after removing at most one pointer indirection, has kind struct, map, or slice is marked as a JSON column, except for three deliberate carve-outs:

- `time.Time` (and a pointer to it) is never JSON-wrapped; it scans through pgx's native timestamp handling.
- `[]byte` (and `*[]byte`) is never JSON-wrapped; it is `bytea`, not JSON.
- Any type that implements `sql.Scanner` (or whose pointer does) is left alone, since it already knows how to scan itself; wrapping it in JSON would hand it a payload it does not expect.
- Any type defined in `github.com/jackc/pgx/v5/pgtype` (for example `pgtype.Range[T]`, `pgtype.Array[T]`) is excluded as well: pgx's own driver already scans directly into these, and none of them implement `sql.Scanner`, so the check above alone would not catch them. Marking one of these JSONB would hand pgx's native text representation (for example `[1,10)` for a range) to the JSON decoder and fail with a syntax error.

A non-pointer JSON-marked field is scanned through the same `jsonScanner` a registered table's JSONB column uses, which both decodes with `encoding/json` and tolerates a SQL `NULL` as a no-op. A pointer field (such as `Customer *Customer` above) takes a different, also pre-existing path: pgx's own `pgtype.JSONCodec` already decodes generically into any Go pointer-to-pointer destination, allocating the pointee on a non-null value and leaving it `nil` on `NULL`. Either way, no extra configuration is required on your part.

Scanning a Join with to_jsonb

Put the last two sections together and the earlier `OrderWithCustomer` example is the general pattern for reading a join without writing a custom struct-scanning function: project the "many" side's own columns normally, and fold the "one" side into a single column with `to_jsonb(alias.*)`.

```go
type adHocItem struct {
    ID     int64
    Name   string
    Parent *scanTestParent // decoded from to_jsonb(p.*) AS parent.
}

const joinQuery = `
    SELECT c.id, c.name, to_jsonb(p.*) AS parent, c.parent_id
    FROM child c JOIN parent p ON p.id = c.parent_id`

items, err := pg.QueryStructs[adHocItem](ctx, db, joinQuery)
```

Three things happen at once here, all covered above: `c.id` and `c.name` populate `adHocItem.ID` / `Name` by ordinary column-name matching; `to_jsonb(p.*) AS parent` populates `Parent` through the pointer-field JSON path; and `c.parent_id`, which `adHocItem` has no field for, is silently ignored rather than causing an error, because `LooseTable`'s descriptor is always non-strict. None of this requires `adHocItem` to be registered with a `Schema`, and none of it requires a hand-written `rows.Scan` call.

Summary

- `Query`, `QueryRow`, `QueryBoolean`, `Exec`, `Mutate`, `MutateSingle` and `Count` exist on both `*DB` and `Repository[T]`; the repository versions delegate straight through. `DB.Select` is the callback form (`func(Rows) error`); `Repository[T].Select` / `SelectSingle` return a `[]T` / `T` directly.
- `QuerySlice`, `QueryTwoSlices`, `QueryMap` and `QuerySingle` cover one- and two-column result shapes without a hand-written scan loop; `QueryFunc` plus a `ScanFunc[T]` covers everything else that is not a registered struct.
- A registered `T` scans through `desc.RowsToStruct` / `RowToStruct` (or `ConvertRowsToStruct` for a single already-positioned row), matching columns case-insensitively via a lookup built once per query, not once per row.
- `QueryStructs`, `QueryStruct` and `ScanStructs` scan into an unregistered, ad hoc struct via `desc.LooseTable`; `QueryStructs` / `QueryStruct` first check whether `T` happens to be registered in `db`'s `Schema` and use that descriptor instead when it is.
- `LooseTable` resolves a column name from the `pg` tag's `name=` option, then the `json` tag, then `SnakeCase(field name)`; it never flattens embedded structs, marks every column nullable, and ignores unknown result columns.
- A struct, map or slice field JSON-decodes automatically, except `time.Time`, `[]byte`, `sql.Scanner` implementors and pgx's own `pgtype` types; this is what makes `to_jsonb(x.*)`

`AS field` scan straight into a struct field with no manual unmarshaling.

Further Reading

- pgx documentation: the driver `Row`, `Rows`, `CommandTag` and `pgtype` types this chapter builds on.
- PostgreSQL: JSON Functions and Operators: `to_jsonb`, `row_to_json` and the rest of the JSON function family.
- Go: encoding/json: the decoder every JSON-marked field goes through.
- database/sql.Scanner: the interface `LooseTable` and the registered-table scanner both check for before deciding to JSON-wrap a field.

CHAPTER 6

Filtering and Pagination

An endpoint that lists rows almost never wants every row. It wants a filtered, sorted, paged slice, plus a total count for the client's pager. Building that by hand means juggling three interlocking concerns at once: a `WHERE` clause whose optional filters come and go depending on what the caller passed, an `ORDER BY` column that must come from a trusted list rather than raw user input, and a `LIMIT` / `OFFSET` pair with its own bind parameters appended after whatever the filter already used. pg gives each concern its own tool: `Conditions` (in `where.go`) builds the `WHERE` fragment and renumbers its bind parameters so one filter set can drive both the page query and its `COUNT` twin; `desc.Table.OrderBy` / `Repository[T].OrderBy` validate a caller-supplied sort column against the table's real columns before quoting it; and `PageOptions` / `SelectPaginated` / `SelectWithTotal` (in `pagination.go`) turn a plain `SELECT` into a paged one. This chapter covers all three, verified against the current source.

Background

If you are comfortable with parameterized SQL and know why string concatenation is unsafe, skip to The Conditions Builder.

A parameterized query separates the SQL text from the values it operates on: `WHERE email = $1` is sent to PostgreSQL once, as a fixed string, and `$1`'s value is sent alongside it as data, never substituted into the text. This is what makes parameterized queries immune to SQL injection: a value cannot "break out" into SQL syntax, because it was never part of the SQL text in the first place. The awkward part is building a query whose filter set varies at runtime. If a caller only supplies some of ten possible filters, the naive approach concatenates SQL fragments and separately tracks which `$N` belongs to which fragment, a bookkeeping problem that gets worse the moment the same filter set needs to drive two different queries (a page of rows, and a count of matching rows) with the placeholders renumbered consistently in both. `Conditions` exists to remove that bookkeeping entirely.

The Conditions Builder

`Conditions` **builds a** `WHERE` **clause from raw SQL fragments whose bind parameters are local to each fragment, then renumbers all of them to consecutive global positions when you render the final clause.** Inside any fragment passed to `Where` or one of the `And*` methods, `$1` .. `$n` refer to that call's own `args`, and may repeat within the same fragment; `Build` rewrites every occurrence to the correct global `$N` afterward.

```go
c := pg.Where("archived = $1", false).
    And("price BETWEEN $1 AND $2", 10, 100)

clause, args := c.Build(1)
// clause: (archived = $1) AND (price BETWEEN $2 AND $3)
// args:   []any{false, 10, 100}
```

`Conditions` is deliberately SQL-transparent: it does not parse or understand the fragments you give it, it only renumbers placeholders and joins fragments with `AND`. Column names, type names and match expressions passed to the helper methods below are developer-authored SQL literals, not sanitized in any way; never build them from unvalidated user input. User input belongs in the `args` alongside a fragment, or, for a dynamic sort column, in `Repository[T].OrderBy` / `desc.Table.OrderBy` (covered later in this chapter). The zero value of `Conditions` is not ready to use; start with `Where`.

Building Fragments: Where, And, AndIf

`Where(fragment string, args ...any) *Conditions` starts a builder with an optional first fragment. Passing `""` starts an empty builder with nothing appended, so code that builds a filter set conditionally can always start from `Where("")` and grow it with `And` / `And*` calls without a special first-fragment case.

`And(fragment string, args ...any) *Conditions` appends a fragment joined with `AND` to whatever is already accumulated; an empty fragment is a no-op, which is what makes `Where("")` safe to grow unconditionally. `AndIf(cond bool, fragment string, args ...any) *Conditions` appends `fragment` only when `cond` is true; when it is false, `args` are discarded entirely and the builder is returned unchanged, which is the common shape for an optional filter whose presence a caller already decided:

```go
c := pg.Where("").
```

```
    AndIf(status != "", "status = $1", status).
    AndIf(minAge > 0, "age >= $1", minAge)
```

Array and Search Helpers

Four methods cover recurring filter shapes that would otherwise be hand-written every time.

`AndAnyOf(column, elemType string, values any) *Conditions` appends an optional array filter that passes when `values` is `NULL` or empty and otherwise requires `column` to equal one of its elements:

```sql
($1::<elemType>[] IS NULL OR CARDINALITY($1::<elemType>[]) = 0
 OR <column> = ANY($1::<elemType>[]))
```

`values` is bound once as `$1` even though the rendered fragment references it three times; `Build` gives all three occurrences the same global position, so the underlying argument is not duplicated. `elemType` must match `^[A-Za-z_][A-Za-z0-9_ ]*$` (a bare or space-separated PostgreSQL type name, such as `smallint` or `double precision`); `AndAnyOf` panics on a violation, since `elemType` is developer-authored SQL, not user input, and a malformed value is a coding mistake to catch immediately.

`AndMatchAnyOf(match, elemType string, values any) *Conditions` is the same optional-array gate around a caller-supplied `match` expression instead of a fixed `column = ANY(...)` comparison, for `EXISTS` / `NOT EXISTS` / `IN` -subquery shapes `AndAnyOf` cannot express; `match` must itself reference `$1` as the array parameter.

`AndNameMatchAnyOf(matchExpr string, names []string) *Conditions` appends a multi-name search that passes when `names` is `NULL` or empty and otherwise requires at least one non-blank name for which `matchExpr` holds, using `unnest` internally so `matchExpr` can reference the unnested element as `t`:

```go
c.AndNameMatchAnyOf(`name ILIKE ('%' || btrim(t) || '%')`,
    []string{"Ann", "Bob"})
```

`AndSearch(term string, substring bool, tsvExpr, textExpr string) *Conditions` appends a text-search filter that always passes when `term` is empty and otherwise requires a match: an `ILIKE '%term%'` comparison against `textExpr` when `substring` is true, or a full-text `tsvExpr @@ plainto_tsquery($1)` comparison when it is false. Only one of `tsvExpr` / `textExpr` is used, depending on `substring`; the unused one is ignored.

Zero-Gated Range and Equality Filters

Three more methods gate a comparison on whether the value is the Go zero value for its kind, so an unset filter (left at its zero value by the caller) simply does not filter.

`AndOptionalEq(column string, value any) *Conditions` picks its gate from `value`'s `reflect.Kind`: a string value gates on `$1 = ''`, an integer/unsigned/float value gates on `$1 = 0`, and any other kind (bool, `time.Time`, a slice) applies the equality unconditionally, since there is no defined "disabled" zero value for those; pair `AndIf` with such a value instead if it needs to be optional.

`AndMin(column string, value any) *Conditions` and `AndMax(column string, value any) *Conditions` are lower- and upper-bound filters, ignored when `value` is not strictly positive:

```sql
-- AndMin("price", 10)
($1 <= 0 OR price >= $1)
-- AndMax("price", 100)
($1 <= 0 OR price <= $1)
```

Local Numbering and Build

`Build(startIndex int) (clause string, args []any)` renders every accumulated fragment as one parenthesized, `AND`-joined clause, with each fragment's local `$1`..`$n` placeholders renumbered to consecutive global positions starting at `startIndex`, and returns that clause together with the flattened arguments in matching order. Use `startIndex` 1 for a standalone query, or a higher value to append the clause after other, already-numbered parameters.

An empty builder, whether from `Where("")` with nothing appended or from one where every `And*` call was skipped, renders as the literal clause `TRUE` with nil args, so `WHERE` + clause is always valid SQL and never accidentally filters out every row.

`Args() []any` returns the accumulated arguments in fragment order without paying for the renumbering pass, useful when you only need the arguments and already know the clause is unused (rare in practice). `NextIndex(startIndex int) int` returns `startIndex` plus the number of accumulated arguments: the first free `$N` after a `Build(startIndex)` call, so a caller can append its own trailing parameters (`LIMIT $N` / `OFFSET $N+1`) without recounting placeholders by hand. `String()` implements `fmt.Stringer` by rendering `Build(1)`'s clause, handy for logging and tests where the exact starting index does not matter.

One Filter Set, Two Queries

The reason `Conditions` renumbers placeholders instead of letting you write them once is that a filter set almost always needs to drive two queries: the page of rows, and the total count of matching rows. Calling `Build` with the same `startIndex` twice on the same `Conditions` value yields byte-identical clauses and arguments both times, which is exactly what lets you reuse one filter set for both queries instead of hand-writing (and hand-renumbering) the `WHERE` clause a second time:

```go
filters := pg.Where("archived = $1", false).
    AndSearch(search, true, "search_vector", "full_name").
    AndAnyOf("category", "smallint", categoryIDs).
    AndMin("price", minPrice).
    AndMax("price", maxPrice)

whereClause, args := filters.Build(1)

total, err := repo.Count(ctx,
    "SELECT COUNT(*) FROM items WHERE "+whereClause, args...)
if err != nil {
    return nil, 0, err
}

orderBy, err := repo.OrderBy(sortColumn, sortDescending)
if err != nil {
    return nil, 0, err
}

pageQuery := fmt.Sprintf(
    "SELECT * FROM items WHERE %s ORDER BY %s LIMIT $%d OFFSET $%d",
    whereClause, orderBy, filters.NextIndex(1), filters.NextIndex(1)+1)

pageArgs := append(append([]any{}, args...), limit, offset)
items, err := repo.Select(ctx, pageQuery, pageArgs...)
```

Both queries see identical `$1` .. `$5` positions for the shared filter, because `Build(1)` is deterministic and side-effect free; only the page query goes on to append its own `LIMIT` / `OFFSET` parameters, picked up exactly where `filters.NextIndex(1)` leaves off. The `SelectPaginated` helper later in this chapter automates the `LIMIT` / `OFFSET` appending step for the common case; `Conditions` itself is independent of it and works equally well handed to `SelectPaginated`'s `query` argument or to a query you assemble yourself, as shown here.

Sorting: Why ORDER BY Cannot Be a Bind Parameter

A `WHERE` clause filters on values, and values can always be bind parameters. An `ORDER BY` clause names a column, and PostgreSQL only accepts a `$N` placeholder where a value is expected, never where an identifier is expected (this is documented behavior of the wire protocol pg's driver, pgx, implements; see jackc/pgx#885). A caller that lets an end user pick the sort column therefore has no parameterized way to pass that choice through. The unsafe shortcut, string-concatenating the user's column name straight into the query, reopens exactly the injection hole parameterized queries exist to close.

The safe alternative is to validate the requested column against an allowlist and only then quote it, never to concatenate raw input into SQL. `desc.Table.OrderBy` and its repository-scoped wrapper, `Repository[T].OrderBy`, do exactly that in one call.

OrderBy Validation and the Fallback Chain

```go
func (td *Table) OrderBy(column string, descending bool,
    extraColumns ...string) (string, error)

func (repo *Repository[T]) OrderBy(column string, descending bool,
    extraColumns ...string) (string, error)
```

`column` is matched against the table's own columns case-insensitively, the same rule `GetColumnByName` uses, or by exact, case-sensitive membership in `extraColumns` (for computed or aliased columns that have no `*Column` entry, such as an expression exposed under an alias in the `SELECT` list). On a match against a table column, the returned fragment quotes the descriptor's canonical column name, not whatever casing the caller passed; on a match against `extraColumns`, it quotes the caller-supplied name as given, since there is no descriptor entry to canonicalize against. Quoting uses `pgx.Identifier.Sanitize`, which double-quotes the identifier and doubles any embedded `"`.

Every entry in `extraColumns` must itself be a bare, unquoted identifier, the same shape the library already enforces for every table, column and unique index name. `OrderBy` validates all of them up front, before even looking at `column`, and returns a descriptive error naming the first offending entry if any fails; without this check, a schema-qualified or space-containing entry would still be accepted and quoted as one bogus identifier (`"t.name"` becomes the literal quoted identifier `"t.name"`, not the table-qualified column `t."name"`), surfacing later as an opaque "column does not exist" error from PostgreSQL rather than a clear one from `OrderBy` itself.

An unrecognized `column` returns a descriptive error naming just the offending column, not the full allowlist, so the error is safe to surface to a client without leaking the table's column names. An empty `column` does not error; it falls back, in order, to a column named `created_at`, then to one named `updated_at`, then to the table's primary key column. If the table has none of those three, `OrderBy` returns an error rather than guessing. `descending` selects the trailing `" DESC"` (true) or `" ASC"` (false) on the returned fragment.

```go
orderBy, err := repo.OrderBy("id", true)
// orderBy: `"id" DESC`

orderBy, err = repo.OrderBy("", false, "total_score")
// falls back to created_at/updated_at/primary key, in that order
```

A caller that aliases the table in its own query can prefix the returned fragment with the alias itself, e.g. `"f." + fragment`, since `OrderBy` has no way to know about a query-local alias on its own.

Pagination with PageOptions and SelectPaginated

`PageOptions` describes `LIMIT` / `OFFSET` pagination and ordering for `SelectPaginated`:

```go
type PageOptions struct {
    Limit        int64  // zero or negative adds no LIMIT.
    Offset       int64  // zero or negative adds no OFFSET.
    OrderBy      string // interpolated, never bound; see above.
    WithoutTotal bool   // skip the derived COUNT query.
}
```

`OrderBy` here is interpolated directly into the query, never bound as a parameter, for the same reason covered above; it must come from `Repository[T].OrderBy` / `desc.Table.OrderBy` or a trusted literal written by you, never directly from unvalidated user input.

```go
func (repo *Repository[T]) SelectPaginated(ctx context.Context,
    page PageOptions, query string, args ...any) ([]T, int64, error)
```

`query` is a `SELECT` without its own `ORDER BY`, `LIMIT`, `OFFSET` or a trailing semicolon; `SelectPaginated` defensively trims trailing whitespace and a trailing `;` before use, so a caller-supplied query ending in either does not produce invalid SQL once extended.

Unless `page.WithoutTotal` is set, the total row count is obtained from a query derived automatically by wrapping `query` in a subquery: `SELECT COUNT(*) FROM (query) AS _pg_total`, executed with the same `args` via `DB.Count`. This is a different mechanism from `Conditions` -

based reuse: it works whether or not you used `Conditions` to build `query`'s `WHERE` clause, because it counts the whole query, not just a filter fragment. A total of zero short-circuits: `SelectPaginated` returns `(nil, 0, nil)` immediately, without running the page query at all, since it would necessarily return no rows either. When `WithoutTotal` is set, the `COUNT` query is skipped entirely, total is reported as `-1`, and the page query still runs.

`page.Limit` and `page.Offset`, when positive, are appended to the query as extra bind parameters (never interpolated), numbered to continue correctly after whatever positional parameters `query` already uses. Row scanning for the page query is identical to `Repository[T].Select`: rows convert to `[]T` via `desc.RowsToStruct` against the repository's own table descriptor.

```go
orderBy, err := repo.OrderBy("created_at", true)
if err != nil {
    return nil, 0, err
}

page := pg.PageOptions{Limit: 20, Offset: 40, OrderBy: orderBy}
items, total, err := repo.SelectPaginated(ctx, page,
    "SELECT * FROM customers WHERE active")
```

SelectWithTotal and the Window Column

`SelectWithTotal` covers the opposite case: a query you have already written in full, including its own `ORDER BY`, that produces its total via a `COUNT(*) OVER()` window column rather than a derived subquery.

```go
func (repo *Repository[T]) SelectWithTotal(ctx context.Context,
    query string, args ...any) ([]T, int64, error)
```

`query` must include the window column aliased as `total_count`:

```go
query := `
    SELECT id, name, category, COUNT(*) OVER() AS total_count
    FROM items ORDER BY id`

items, total, err := repo.SelectWithTotal(ctx, query)
```

Internally this calls `desc.RowsToStructWithTotal[T](repo.Table(), rows, "total_count")`, which scans every other column into `T` exactly as `RowsToStruct` does and additionally captures the named window column out of band, into the returned `int64`, so `T` needs no artificial field for it. `total_count` is matched case-insensitively against each row's field descriptions, the same rule

every other column resolution in the library uses. `COUNT(*) OVER()` yields the same value on every row of the result set, so the last row scanned wins for the returned total (an unspecified total comes back only if the query does not uphold that guarantee).

Unlike `SelectPaginated`, `SelectWithTotal` does not derive a separate `COUNT` query, does not append `ORDER BY` / `LIMIT` / `OFFSET`, and does not trim `query`; you are responsible for the window column and any ordering/pagination clauses yourself. Zero rows returns `(empty, 0, nil)` from both methods.

Summary

- `Conditions`, started with `Where` and grown with `And` / `AndIf` / `AndAnyOf` / `AndMatchAnyOf` / `AndNameMatchAnyOf` / `AndSearch` / `AndOptionalEq` / `AndMin` / `AndMax`, builds a `WHERE` clause from raw SQL fragments whose bind parameters are call-local and repeatable.
- `Build(startIndex)` renumbers every fragment's placeholders to consecutive global positions and is deterministic: calling it twice with the same `startIndex` on the same `Conditions` value produces identical output, which is what lets one filter set drive both a page query and its `COUNT` twin. `Args` / `NextIndex` / `String` cover the remaining bookkeeping.
- Dynamic `ORDER BY` cannot be a bind parameter (see jackc/pgx#885); validate the column with `Repository[T].OrderBy` / `desc.Table.OrderBy`, which fall back through `created_at`, `updated_at`, then the primary key when no column is given, and accept an `extraColumns` allowlist for computed columns, itself identifier-validated.
- `SelectPaginated` wraps a plain `SELECT` with a derived `COUNT(*)` subquery (skippable via `PageOptions.WithoutTotal`) and appends `LIMIT` / `OFFSET` as bind parameters; a zero total short-circuits without running the page query.
- `SelectWithTotal` is for a query that already carries its own `COUNT(*) OVER() AS total_count` window column and its own ordering; it captures the total out of band via `desc.RowsToStructWithTotal`.

Further Reading

- jackc/pgx#885: the driver issue documenting why an identifier, such as an `ORDER BY` column, cannot be a bind parameter.
- PostgreSQL: Extended Query Protocol: how parameter binding works at the wire-protocol level.

- PostgreSQL: Window Functions: the `OVER()` clause behind `SelectWithTotal`'s `total_count` column.
- PostgreSQL: Full Text Search: `plainto_tsquery` and `tsvector`, used by `AndSearch`'s non-substring mode.
- OWASP: Query Parameterization Cheat Sheet: the general case for why values, and only values, belong in bind parameters.

CHAPTER 7

Writing Data

Every write in pg funnels through a small number of query builders in `desc/insert_query.go`, `desc/on_conflict.go` and `desc/update_query.go`, called from `Repository[T]` and `*DB`. This chapter follows a single insert from a Go struct to the `INSERT` statement PostgreSQL actually receives: which columns participate, how a zero-valued field interacts with a column's database default, and the one special rule that applies to a UUID primary key. From there it covers batching many rows into one statement, the tag-driven `ON CONFLICT` behavior `Upsert` gives you for free, the explicit `desc.OnConflict` specification for when the tag-driven behavior is not precise enough, the check-then-act `UpdateOrInsert` pattern, the three shapes of `UPDATE`, deletes, and how a `password`-tagged column is hashed, either by PostgreSQL itself or by a Go-side handler you supply. Every claim below was checked against the current source, mainly `desc/insert_query.go`, `desc/on_conflict.go`, `desc/argument.go`, `desc/struct_table.go`, `repository.go`, `conflict.go` and `db.go`.

Background

If you already know the difference between `INSERT ... ON CONFLICT DO NOTHING` and `DO UPDATE`, and why PostgreSQL's bind parameters are capped at 65535 per statement, skip to How Inserts Are Built.

Every value bound into a query, whether an `INSERT`'s column value or a `WHERE` clause's comparison, occupies one of PostgreSQL's numbered parameter slots (`$1`, `$2`, ...), and the wire protocol caps a single prepared statement at 65535 of them. A single-row `INSERT` rarely gets close to that limit, but a multi-row `INSERT ... VALUES (...), (...), ...` statement multiplies parameters per row by the number of columns, so a batch of a few hundred rows against a wide table can approach the ceiling. This is why pg's bulk-insert paths compute a batch size from the table's own column count rather than using one fixed number for every table, covered below.

`ON CONFLICT`, PostgreSQL's clause for handling a would-be duplicate row, has two independent parts: a conflict target (which unique constraint or index the database should watch for a violation on) and an action (`DO NOTHING`, silently skip the row, or `DO UPDATE SET ...`, overwrite specific columns of the existing row instead). Both parts appear throughout this chapter, first as something pg derives automatically from struct tags, then as something you specify explicitly.

How Inserts Are Built

`desc.BuildInsertQuery(td *desc.Table, structValue reflect.Value, idPtr any, forceOnConflictExpr string, upsert bool) (string, []any, error)` is the single-row builder every insert path in the library eventually calls. Two column categories never participate in the generated `INSERT` at all: a column marked `AutoGenerated` (the `identity` tag) and a column marked `Presenter` (a read-only, computed column that exists only for `SELECT`). Every other column contributes either a `$N` bind parameter or, per the next section, the literal keyword `DEFAULT`.

Zero Values, Defaults, and the UUID Primary Key Rule

A field left at its Go zero value on a column that carries a database default is not sent as that zero value at all; it is omitted from the statement so the database's own default fires. This is `extractArguments`'s central rule (`desc/argument.go`): when a column's `Default` tag option is non-empty and the corresponding struct field `isZero`, the field is skipped from the argument list entirely, and `buildInsertQuery` emits the literal `DEFAULT` keyword in its place. An empty string, a zero `time.Time`, a zero `int`, all trigger this the same way.

```go
type Product struct {
    ID        string    `pg:"type=uuid,primary"`
    CreatedAt time.Time `pg:"type=timestamp,default=clock_timestamp()"`
    Name      string    `pg:"type=varchar(255)"`
    Rank      int       `pg:"type=int,default=0"`
}
```

Inserting a `Product{Name: "Widget"}` (leaving `ID`, `CreatedAt` and `Rank` at their zero values) produces `INSERT INTO products (name) VALUES ($1)`, with `id`, `created_at` and `rank` all omitted so `gen_random_uuid()`, `clock_timestamp()` and the literal `0` default each fire on the server. If you do want to insert the literal zero value for a defaulted column, set the field to a value PostgreSQL would still consider non-default (there is no per-field override for this; the decision is made purely from the Go zero value).

The UUID primary key rule is a related but separate piece of `struct_table.go`'s tag-parsing logic, applied once at `Schema.Register` time, not per row: a field tagged `primary` whose resolved type is `UUID`, that is not nullable, carries no explicit `default=` option, and does not reference another table (no `ref=` option), automatically receives `Default = "gen_random_uuid()"`. That is why `pg:"type=uuid,primary"` alone, with no explicit `default=`, is enough to get a server-generated UUID primary key: the library fills in the default for you, and the zero-value-skip rule above then omits the column whenever the Go field is left at `""`.

RETURNING and idPtr

Every single-row insert path accepts an `idPtr` parameter. When it is non-nil, `BuildInsertQuery` resolves the table's primary key column and appends `RETURNING <primary key>` to the statement; the resulting value is then scanned into `idPtr` via `QueryRow(...).Scan(idPtr)` instead of a plain `Exec`. This is how you get a server-generated primary key back without a second round trip:

```go
customer := Customer{Email: "a@example.com", Name: "Ann"}
err := repo.InsertSingle(ctx, customer, &customer.ID)
// customer.ID is now populated from RETURNING id.
```

Passing `idPtr` as `nil` skips `RETURNING` and uses a plain `Exec`, which is marginally cheaper when you do not need the generated value back.

Single vs Many: InsertMany, UpsertMany, and the Batching Cap

`Repository[T].Insert(ctx, values ...T) error` dispatches on the number of values: zero does nothing, exactly one delegates to `InsertSingle`, and more than one delegates to `InsertMany`. `Upsert` follows the identical shape, delegating to `UpsertSingle` or `UpsertMany`.

`InsertMany` and `UpsertMany` do not issue one round trip per row; they build multi-row `INSERT ... VALUES (...), (...), ...` statements via `desc.BuildBulkInsertQuery`, batched at `desc.DefaultInsertBatchSize` (500) rows per statement, with the whole operation wrapped in one transaction via `Repository[T].InTransaction` so a failure in a later batch rolls back every earlier batch in the same call. Per-row semantics match the single-row path exactly: a zero-valued field on a defaulted column still emits `DEFAULT` rather than a parameter, row by row, so `clock_timestamp()` / `gen_random_uuid()` fire per row exactly as they would for individual `InsertSingle` calls.

The 500-row default batch size is deliberately conservative, and pg shrinks it further for a wide table. Before running, `InsertMany` and `UpsertMany` compute `desc.Table.NumInsertableColumns()`, the count of columns that occupy a position (a `$N` parameter or a `DEFAULT` keyword) in one row of the generated statement, and cap the batch size at `min(DefaultInsertBatchSize, 65535/NumInsertableColumns())`. A 30-column table, for instance, caps out at 2184 rows per statement (`65535 / 30`), well under the 500-row default, so no adjustment is needed there; a genuinely wide table (over 131 columns) would need the adjustment to avoid exceeding PostgreSQL's bind-parameter ceiling mid-batch. You never have to compute this yourself.

```go
// Any length; batched and wrapped in one transaction internally.
err := repo.InsertMany(ctx, products...)
```

Upsert and the Tag-Driven Conflict Target

`Upsert` / `UpsertSingle` / `UpsertMany` take a `forceOnConflictExpr string` parameter that controls which `ON CONFLICT` target is used, built by the same `tagConflictTarget` logic `BuildInsertQuery` and `BuildBulkInsertQuery` both call:

- An empty `forceOnConflictExpr` uses the struct's own tags: if any column carries a `conflict=` tag (declaring the table's single `ON CONFLICT` action), the target is every column tagged `unique`; otherwise the target is every column tagged with a matching `unique_index=` name. The action is `DO UPDATE SET col = EXCLUDED.col` over every other inserted column, unless the `conflict=` tag's value is itself `DO NOTHING` (matched case-insensitively), in which case the action is `DO NOTHING` instead. When neither tag is present anywhere on the struct, `Upsert` silently falls back to a plain `INSERT` with no `ON CONFLICT` clause at all, so a genuine duplicate surfaces PostgreSQL's own unique-violation error (SQLSTATE `23505`) rather than being handled.
- The exported constant `pg.DoNothing` (`"DO NOTHING"`) is a special-cased `forceOnConflictExpr` value, not a unique index name: it keeps the same tag-derived target the empty-string case above would use, but forces a `DO NOTHING` action regardless of what the struct's own `conflict=` tag says. When the struct declares no conflict-target tag at all, the result is a target-less `ON CONFLICT DO NOTHING`, valid PostgreSQL that fires against a conflict on any unique constraint on the table. The match against `pg.DoNothing` is case-insensitive, but the exact value `"DO NOTHING"` is always recognized.
- Any other non-empty `forceOnConflictExpr` names either a specific unique index (looked up by name against the table's declared `unique_index=` groups) or a specific tag-derived

conflicting column, and forces a full `DO UPDATE SET col = EXCLUDED.col` over every other inserted column, regardless of what the struct's own tags would otherwise have produced.

```go
// customer_unique_idx is a unique_index= name declared on Customer.
err := repo.UpsertSingle(ctx, "customer_unique_idx", customer, &customer.ID)
```

The conflict expression is always the first argument after `ctx`.
`Repository[T].UpsertSingle(ctx, forceOnConflictExpr string, value T, idPtr any)` and `DB.UpsertSingle(ctx, forceOnConflictExpr string, value any, idPtr any)` agree, as do `Upsert` and `UpsertMany` on both types. The variadic methods cannot put it anywhere else, since `values ...T` has to come last, and the single-row methods match them so that moving a struct between the repository path and the raw `*DB` path does not change how the call reads.

A `DO NOTHING` action, whether reached via `pg.DoNothing` or a `conflict=DO NOTHING` tag, still appends `RETURNING` on the single-row path. `UpsertSingle` / `DB.UpsertSingle` called with a non-nil `idPtr` therefore report the two outcomes distinguishably: a row that inserted without conflicting populates `idPtr`, and a row skipped by the conflict returns `ErrNoRows`. Read `ErrNoRows` here as "the row was already there", not as a failure. The bulk path never appends `RETURNING`, so `UpsertMany` reports no identifiers either way, and `InsertSingleOnConflict` with `OnConflict{DoNothing: true}` offers the same contract with an explicit conflict target, covered next.

Every tag-driven `DO UPDATE` this section describes updates every inserted column outside the conflict target, all-or-nothing; there is no tag-driven way to update only some columns or to attach a `WHERE` condition to the update. That precision is what `desc.OnConflict` exists for.

Explicit Conflict Specs: desc.OnConflict

```go
type OnConflict struct {
    Columns    []string
    Constraint string
    DoNothing  bool
    SetColumns []string
    SetWhere   string
}
```

`Columns` names the conflict-target columns explicitly (`ON CONFLICT (a, b)`); each name is validated against the table and quoted. `Constraint` targets a named constraint instead (`ON CONFLICT ON CONSTRAINT name`); it is quoted but not validated against the table, since table descriptors do not track constraint names. `Columns` and `Constraint` are mutually exclusive, and leaving both empty falls back to the same tag-derived target `Upsert` would use. `DoNothing`

emits `DO NOTHING` and cannot be combined with `SetColumns` or `SetWhere`. `SetColumns` lists exactly which columns the `DO UPDATE SET` clause touches; leaving it empty updates every inserted column outside the conflict target, matching `Upsert`'s default behavior but with full control over the target. `SetWhere` appends a raw SQL predicate after the `SET` list (`... WHERE <SetWhere>`); PostgreSQL lets that predicate reference the target table's own (unqualified, since no alias is added) columns for the existing row and `EXCLUDED.*` for the proposed new row, which is what lets you write a "only overwrite if newer" guard:

```go
oc := pg.OnConflict{
    Columns:    []string{"sku"},
    SetColumns: []string{"price", "updated_at"},
    SetWhere:   "items.updated_at < EXCLUDED.updated_at",
}
```

`SetWhere` is developer-authored SQL, written verbatim with no escaping; never build it from end-user input.

`desc.BuildInsertQueryOnConflict` and `desc.BuildBulkInsertQueryOnConflict` are the single-row and bulk builders for an explicit `OnConflict`, mirroring `BuildInsertQuery` / `BuildBulkInsertQuery` in every other respect (column selection, the zero-value-to-DEFAULT rule, password handling). One difference is worth calling out: `BuildInsertQueryOnConflict` always appends `RETURNING <primary key>` when `idPtr` is non-nil and a conflict target exists, even for a `DoNothing` action, where a plain `INSERT` would otherwise only return `RETURNING` for a `DO UPDATE`. A `DO NOTHING` that ends up skipping the row then yields zero returned rows, which is what lets `InsertSingleOnConflict` (next section) report that skip as `ErrNoRows` instead of leaving `idPtr` unset.

Repository-Level Conflict Methods

```go
func (repo *Repository[T]) InsertOnConflict(ctx context.Context,
    oc OnConflict, values ...T) error

func (repo *Repository[T]) InsertSingleOnConflict(ctx context.Context,
    oc OnConflict, value T, idPtr any) error
```

`InsertOnConflict` batches exactly like `InsertMany`: one transaction, the same `min(DefaultInsertBatchSize, 65535/NumInsertableColumns())` batch size, built via `desc.BuildBulkInsertQueryOnConflict`. It never appends `RETURNING`; use `InsertSingleOnConflict` when a primary key needs to come back.

`InsertSingleOnConflict` runs `Exec` when `idPtr` is `nil`. When `idPtr` is non-nil, it always scans a `RETURNING` value: for a `DO UPDATE` action this is the inserted-or-updated row's primary key, and for a `DoNothing` action that skipped an existing conflicting row, the query returns zero rows and the method returns `ErrNoRows` instead of leaving `idPtr` unset:

```go
err := repo.InsertSingleOnConflict(ctx, pg.OnConflict{
    Columns:   []string{"email"},
    DoNothing: true,
}, customer, &customer.ID)

if pg.IsErrNoRows(err) {
    // a customer with this email already existed; nothing was inserted.
}
```

UpdateOrInsert: Check-Then-Act Writes

Some upserts do not fit the `ON CONFLICT` model cleanly: the row you want to update is identified by a business key that is not necessarily the table's `ON CONFLICT` target, or the update itself needs logic richer than a `SET` list. `UpdateOrInsert` is the write half of that check-then-act pattern, built to still be correct against a writer racing the same check:

```go
func UpdateOrInsert[R any](ctx context.Context, db *DB,
    updateQuery, insertQuery string, args []any,
    insertExtraArgs ...any) (R, error)
```

It runs `updateQuery` (with `args`) via `db.QueryRow(...).Scan(&result)` first. If that matches a row, the scanned `R` is returned immediately. If it reports `ErrNoRows`, `UpdateOrInsert` runs `insertQuery` instead, with `args` followed by `insertExtraArgs` as its bind parameters, and scans its `RETURNING` value into `R` the same way. Any other error from either query is returned as-is. Both queries are developer-authored SQL, executed verbatim; `UpdateOrInsert` builds, validates or quotes none of it, so parameterize any user-supplied value in `args` / `insertExtraArgs`, never by concatenating it into the query text. The `insertQuery` should still carry its own `ON CONFLICT` clause, so that a concurrent insert racing this one's `UPDATE` (which found no row) is handled safely rather than raising a raw unique-violation error.

Updates: Update, UpdateOnlyColumns, UpdateExceptColumns

```go
func (repo *Repository[T]) Update(ctx context.Context,
    values ...T) (int64, error)
```

```
func (repo *Repository[T]) UpdateOnlyColumns(ctx context.Context,
    columnsToUpdate []string, values ...T) (int64, error)

func (repo *Repository[T]) UpdateExceptColumns(ctx context.Context,
    columnsToExcept []string, values ...T) (int64, error)
```

`Update` is `UpdateOnlyColumns(ctx, nil, values...)`: a full update of every column, matched by primary key. `UpdateExceptColumns` computes `columnsToUpdate` as every column name except the ones you list (`desc.Table.ListColumnNamesExcept`), then calls `UpdateOnlyColumns` with that explicit list.

A full update (`nil`/empty `columnsToUpdate`) and an explicit column list behave differently around two specific kinds of column, and this is a real trap for `UpdateExceptColumns`. A column tagged `identity` or `generated=` is excluded from every insert and update unconditionally, full or partial alike, because that exclusion happens earlier, in `extractArguments`'s own `AutoGenerated`/`Presenter` check, before any column list is even consulted; no special handling is needed for it here. A UUID primary key carrying its automatic `gen_random_uuid()` default, and a timestamp column whose default is `clock_timestamp()`/`now()`, are different: neither is `AutoGenerated`, so they are protected only on a full update, which additionally filters out anything PostgreSQL, not you, is responsible for maintaining. That is why a full `Update` never clobbers `created_at`. An explicit column list (whether from `UpdateOnlyColumns` directly or via `UpdateExceptColumns`) carries no such protection for these two: naming one of them explicitly, or leaving it un-excluded from `UpdateExceptColumns`'s exception list, sets it to whatever value the struct happens to hold, verbatim. If your table has a `created_at`/`updated_at` pair maintained by database defaults, exclude them explicitly:

```
updated, err := repo.UpdateExceptColumns(ctx,
    []string{"created_at", "updated_at"}, customer)
```
GO

This same explicit-list behavior is also what makes `UpdateOnlyColumns` useful for writing a zero value back to a defaulted column, something a full `Update` cannot do at all: naming the column bypasses the insert-only "skip zero when a default exists" rule entirely (that rule never applies to updates in the first place; it is specific to `extractArguments`'s insert path), so `UpdateOnlyColumns(ctx, []string{"username"}, Customer{Username: ""})` really does set `username` to the empty string.

`Repository[T]` also has `UpdateOnlyColumnsReportNoRows(ctx, columnsToUpdate, values...) (bool, error)`, which runs a `RETURNING` update per value and reports whether a row actually matched (returning `(false, nil)` on `ErrNoRows` rather than an error), useful when "no row matched" needs to be told apart from a real failure without a separate `Exists` check.

UpdateJSONB

`DB.UpdateJSONB(ctx, tableName, columnName, rowID string, values map[string]any, fieldsToUpdate []string) (int64, error)` updates a single JSONB column of one row, resolved by table name (not a Go struct type) through the schema, the same safety net the table-name CRUD methods use: an unknown table or column returns a descriptive error before any SQL is built, and every resolved name is quoted.

With an empty `fieldsToUpdate`, it replaces the column's entire contents: `SET "column" = $1` with the whole `values` map as the new JSONB document. With a non-empty `fieldsToUpdate`, it merges instead: `SET "column" = "column" || $1`, PostgreSQL's JSONB concatenation operator, which overwrites the named top-level keys and leaves every other existing key in the column untouched. Before merging, every key named in `fieldsToUpdate` must be present in `values` (a missing one is an error), and every key in `values` that is *not* named in `fieldsToUpdate` is deleted from the map in place before the query runs, since the merge only sends what remains. Passing `values` a caller intends to reuse afterward is therefore unsafe for the partial path; pass a fresh map, or copy it first.

Deletes

```go
func (repo *Repository[T]) Delete(ctx context.Context,
    values ...T) (int64, error)

func (repo *Repository[T]) DeleteByID(ctx context.Context,
    id any) (bool, error)
```

`Delete` removes rows by primary key, extracted from each value in `values`, in a single `DELETE FROM table WHERE pk = ANY($1)` statement (one round trip regardless of how many values you pass). `DeleteByID` is the convenience form that takes just the primary key value instead of a whole struct, and reports whether a row was actually removed. `*DB` has the same two methods, `Delete(ctx, values ...any) (int64, error)` and the table-name-based `DeleteByID` / `DeleteBy` (covered in Chapter 4 alongside the rest of the table-name CRUD surface); `DB.Delete` resolves the target table from the first value's type, so every value passed to one call must be the same struct type.

Password Columns

A field tagged `pg:"password"` (`Column.Password`) gets special handling on every write and read path, and the handling depends on whether the table's `Schema` was configured with a Go-side `desc.PasswordHandler`:

```go
schema.HandlePassword(pg.PasswordHandler{
    Encrypt: func(tableName, plainPassword string) (string, error) {
        // hash plainPassword yourself, e.g. with bcrypt.
    },
    Decrypt: func(tableName, encryptedPassword string) (string, error) {
        // verify/decrypt, or return "" to never populate plaintext on Select.
    },
})
```

With no `PasswordHandler.Encrypt` configured, PostgreSQL does the hashing: an insert or update of a password column is wrapped as `crypt($N, gen_salt('<PasswordAlg>'))`, using the `pgcrypto` extension's `crypt` / `gen_salt` functions. `PasswordAlg` is a package-level variable (default `"bf"`, blowfish-based) restricted to an allowlist of `"bf"`, `"md5"`, `"xdes"`, `"des"`; every query that would embed it validates it against that allowlist first, since it is interpolated directly into the generated SQL. Set `PasswordAlg` once, during initialization, never concurrently with query building. `Repository[T].SelectByUsernameAndPassword` / `DB.SelectByUsernameAndPassword` verify a login this same way, comparing the password column against itself (`crypt($2, password) = password`) so the stored hash's own salt is reused for the comparison.

With `PasswordHandler.Encrypt` configured, the library never sends a plaintext password to PostgreSQL at all: `Encrypt` runs in Go before the value is bound as an ordinary `$N` parameter. On the read side, `PasswordHandler.Decrypt`, if configured, runs per row through a dedicated scanner that replaces the stored value with `Decrypt`'s result; without a `Decrypt` func, a `SELECT` of a password column scans back whatever PostgreSQL actually stored (the `crypt()` hash, or whatever `Encrypt` produced), never a plaintext password you did not explicitly ask to reconstruct.

Which path you pick also determines whether the table can use the `COPY` protocol for bulk loading at all: a table whose password column is hashed server-side (no Go-side `Encrypt`) has no `COPY` equivalent, since `COPY` cannot invoke a SQL function per row. Chapter 8 covers this in full, under `ErrCopyPassword`.

Summary

- `extractArguments` skips a zero-valued field on a defaulted column and emits `DEFAULT` in its place; a `primary` UUID column with no explicit `default=` and no `ref=` gets

`gen_random_uuid()` automatically at registration time.

- `idPtr` non-nil adds `RETURNING <primary key>` and scans it back; `nil` uses a plain `Exec`.
- `Insert` / `Upsert` dispatch on argument count to the single-row or batched path; `InsertMany` / `UpsertMany` batch at `min(desc.DefaultInsertBatchSize, 65535/NumInsertableColumns())` rows per statement, all inside one transaction.
- `Upsert` 's tag-driven conflict target comes from `conflict=` / `unique` / `unique_index=` tags; `forceOnConflictExpr` names a specific index or column and forces a full `DO UPDATE`, except for the special value `pg.DoNothing`, which keeps the tag-derived target but forces `DO NOTHING` instead. On the single-row path it still returns the primary key, so `idPtr` is populated for an insert that did not conflict and `ErrNoRows` reports one that was skipped. Its parameter position differs between `*DB` and `Repository[T]`.
- `desc.OnConflict` (`Columns` / `Constraint`, `DoNothing`, `SetColumns`, `SetWhere`) gives full control over the conflict target and action; `InsertOnConflict` / `InsertSingleOnConflict` are its repository-level entry points, and only the latter supports `RETURNING`.
- `UpdateOrInsert[R]` runs an `UPDATE` first and falls back to an `INSERT` (with its own `ON CONFLICT`) on `ErrNoRows`, for check-then-act writes keyed on a business identity.
- A full `Update` skips generated columns automatically; an explicit column list (`UpdateOnlyColumns`, and therefore `UpdateExceptColumns` too) does not, and is also how you write a zero value back to a defaulted column.
- A `password` -tagged column is hashed server-side via `crypt()` / `gen_salt()` by default, or in Go via a `desc.PasswordHandler` installed with `Schema.HandlePassword`; the choice also decides whether the table can use `COPY`.

Further Reading

- PostgreSQL: INSERT ... ON CONFLICT: the full grammar behind every conflict target and action in this chapter.
- PostgreSQL: pgcrypto: `crypt()` and `gen_salt()`, the functions behind server-side password hashing.
- PostgreSQL: JSON Functions and Operators: the `||` concatenation operator `UpdateJSONB` 's partial-update path relies on.
- PostgreSQL: Limits: the 65535-parameter ceiling per statement that shapes the batching cap.
- Go: reflect package: `reflect.Value` / `IsZero`, the mechanism behind the zero-value-to-DEFAULT rule.

CHAPTER 8

Bulk Loading and Streaming

`InsertMany` (Chapter 7) is fast, but it is still an `INSERT`: every row costs bind parameters, every batch costs a round trip, and the statement still goes through the planner like any other `INSERT`. PostgreSQL's `COPY` protocol is a different, lower-level path built specifically for moving many rows in or out with minimal per-row overhead, and pg exposes it through `desc.CopyPlan` / `BuildCopyPlan`, `DB.CopyFrom` and `Repository[T].CopyFrom`. `COPY` is faster, and it gives up things `INSERT` has: no `ON CONFLICT`, no `RETURNING`, and critically, no per-row `DEFAULT`, which changes how a defaulted column behaves across a batch in a way worth understanding before you reach for it. The second half of this chapter is the opposite direction: reading many rows without materializing them all in memory first, via `Repository[T].SelectIter` and the package-level `QueryIter`, both built on Go's `iter.Seq2` range-over-func iterators. Every signature and behavior here was verified against `desc/copy_from.go`, `db_copy.go`, `repository_copy.go` and `repository_iter.go`.

Background

If you already know what makes `COPY` faster than `INSERT`, and you are comfortable with Go's range-over-func iterators (`iter.Seq2`, standard since Go 1.23), skip to The COPY Protocol and CopyPlan.

An `INSERT`, even a multi-row one, is still parsed, planned and bound like any other SQL statement, and every value it carries occupies one of the connection's numbered bind-parameter slots. `COPY` sidesteps all of that: the client and server agree once on a column list and a wire format, then the client streams rows of raw column data with no further per-row SQL parsing and no bind-parameter accounting at all. This is why `COPY` has no bind-parameter ceiling to batch around, the way `InsertMany` does, and why it is the standard tool for loading (or exporting) large amounts of data quickly.

A Go iterator, in the `iter.Seq` / `iter.Seq2` sense, is a function that takes a `yield` callback and calls it once per element, stopping early if `yield` returns `false`. `for x := range someSeq { ... }` is syntactic sugar the compiler turns into exactly that call pattern. `iter.Seq2[K, V]` is the two-value form, `func(yield func(K, V) bool)`, which `SelectIter` / `QueryIter` use to yield a `(value, error)` pair per row without allocating a slice for the whole result set first.

The COPY Protocol and CopyPlan

`desc.CopyPlan` holds the precomputed column list and per-row value extraction for a `COPY FROM` of a table's struct values, built once by `desc.BuildCopyPlan` and then replayed for every row:

```go
func BuildCopyPlan(td *Table, structValues []reflect.Value) (*CopyPlan, error)

type CopyPlan struct {
    // ColumnNames is the COPY column list, in Row's value order.
    ColumnNames []string
    // unexported fields.
}

func (p *CopyPlan) Row(structValue reflect.Value) ([]any, error)
```

`BuildCopyPlan` resolves the column list in three steps. First, a column marked `AutoGenerated` or `Presenter` never participates, mirroring `desc.Table.NumInsertableColumns` and the bulk `INSERT` builder's own column selection exactly. Second, a `password`-tagged column with no working Go-side `PasswordHandler.Encrypt` makes `BuildCopyPlan` return `ErrCopyPassword` immediately, before the third step runs for any column (covered below). Third, for every remaining column, `BuildCopyPlan` decides whether to include it at all, which is the one place `COPY` genuinely diverges from `InsertMany`.

The All-or-Nothing Default Rule

A `COPY` statement has no per-row `DEFAULT`: a column is either present in every row of the stream or entirely absent from it, and PostgreSQL only applies a column's database-side default to a column that is completely missing from the `COPY` column list, never to an individual row within an included column. `BuildCopyPlan` resolves this with one reflection pre-pass per `Default`-bearing column, over every row in the batch: if every row's value for that column would have triggered the `DEFAULT` keyword under `InsertMany` (the same zero-value check `extractArguments` uses, see Chapter 7), the column is omitted from `ColumnNames`

entirely, so `gen_random_uuid()`, `clock_timestamp()`, or whatever the column's default expression is, fires for every row. If even a single row's value is non-zero, the column is included instead, and every row's value for it is sent verbatim, including the zero-valued rows.

Concede the trade-off plainly: a column that carries a default and is zero in some rows but set in others is only representable under `COPY` by including the column and sending the literal zero (an all-zeros UUID, a zero `time.Time`) for the rows that would otherwise have been defaulted, never as `DEFAULT`. This is the sharpest divergence from `InsertMany`, which decides `DEFAULT`-vs-value independently for each row and can therefore never lose that per-row precision. If your batch genuinely needs the database default to fire for only some rows of a column that is not all-zero across the whole batch, use `InsertMany` for that load instead of `CopyFrom`.

```go
values := []Product{
    // Non-zero CreatedAt forces the column into the COPY column list.
    {Name: "explicit", CreatedAt: someTime},
    // CreatedAt left zero here, but now sent as the zero time, not DEFAULT.
    {Name: "zero"},
}
```

If every remaining column ends up omitted this way (every one of them carries a `Default` and is zero across the whole batch), `BuildCopyPlan` returns a descriptive error rather than handing pgx an empty column list, which would otherwise reach PostgreSQL as `copy "table" ( ) from stdin` and fail with a raw syntax error.

DB.CopyFrom and Repository.CopyFrom

```go
func (db *DB) CopyFrom(ctx context.Context, tableName Identifier,
    columnNames []string, rowSrc CopyFromSource) (int64, error)

func (repo *Repository[T]) CopyFrom(ctx context.Context,
    values []T) (int64, error)
```

`Identifier` is a type alias for `pgx.Identifier` (a qualified name such as `Identifier{"public", "customers"}`), and `CopyFromSource` is a type alias for `pgx.CopyFromSource`, the row-iterator interface `COPY` consumes. `DB.CopyFrom` is a thin wrapper over pgx's own `CopyFrom`, routed through the active transaction when there is one, the same pool-versus-transaction routing every other query method in the library uses. `Repository[T].CopyFrom` is what you reach for directly: it builds a `desc.CopyPlan` from `values`, wraps `plan.Row` in a `pgx.CopyFromSlice` source, and calls `DB.CopyFrom` with the repository's own table identifier and `plan.ColumnNames`.

```go
copied, err := repo.CopyFrom(ctx, products)
```

`CopyFrom` returns `ErrIsReadOnly` for a read-only repository (a view, materialized view or presenter table), and `(0, nil)` for an empty `values` slice without touching the database. Unlike `InsertMany`, it does not open a transaction of its own: it routes through the current transaction when there is one, but a caller that wants the copy to roll back together with other statements must wrap the call itself, via `Repository[T].InTransaction`. The whole operation is also all-or-nothing at the driver level: `COPY` either succeeds in full or pgx aborts it, and no rows are stored from a failed copy.

ErrCopyPassword and the Empty-Column Error

`desc.ErrCopyPassword` is returned by `BuildCopyPlan` (and therefore by `Repository[T].CopyFrom`) when a table has a password column that PostgreSQL itself hashes on write, via the same `crypt($N, gen_salt('<PasswordAlg>'))` fragment `INSERT` / `UPDATE` use (see Chapter 7). `COPY` streams raw column values straight into the table and cannot invoke a SQL function per row the way a regular `INSERT` can, so that database-side hashing path has no `COPY` equivalent at all. There are two ways out: fall back to `InsertMany` / `InsertSingle` for that table, which still support the database-side `crypt()` path, or install a Go-side `desc.PasswordHandler` via `Schema.HandlePassword` so `BuildCopyPlan` can encrypt the password itself, in Go, once per row, before handing it to `COPY`.

The other error worth naming explicitly is the one described in the previous section: every insertable column omitted because it carries a default and is zero across the whole batch. Both errors are returned before any network round trip, so a caller can detect and handle them without touching the database.

CopyFrom versus InsertMany

	`CopyFrom`	`InsertMany`
Throughput on a large plain load	Faster: no per-row bind-parameter accounting	Fast, but slower than `COPY`
Bind-parameter ceiling	None	Batched at `min(500, 65535/columns)` per statement (Chapter 7)
`ON CONFLICT`	Not supported	Supported (`Upsert` / `InsertOnConflict`)
`RETURNING`	Not supported	Supported, per row, via `InsertSingle`

Per-row `DEFAULT`	All-or-nothing per column, across the whole batch	True per row, independently decided for each row
Own transaction	No; routes through the current one if any	Yes; always wraps the whole call in one transaction
Password column hashed server-side (no Go `PasswordHandler`)	Not supported (`ErrCopyPassword`)	Supported

Reach for `CopyFrom` for a large, plain load where every row genuinely gets its own value for every defaulted column (or none do), and where you do not need conflict handling or a returned primary key. Reach for `InsertMany` whenever you need `ON CONFLICT`, `RETURNING`, or a database default to fire for individual zero-valued rows of a column that is not all-zero across the batch.

Streaming Reads: SelectIter and QueryIter

`Repository[T].SelectIter` and the package-level `QueryIter` are the read-side counterpart: a lazy, single-use iterator that decodes one row at a time, without ever materializing the whole result as a slice.

```go
func (repo *Repository[T]) SelectIter(ctx context.Context,
    query string, args ...any) iter.Seq2[T, error]

func QueryIter[T any](ctx context.Context, db *DB,
    query string, args ...any) iter.Seq2[T, error]
```

`SelectIter` scans each row into `T` using the same per-row struct scanning `Repository[T].Select` itself uses (`desc.ConvertRowsToStruct` against the repository's own cached table descriptor); `QueryIter` is the streaming analog of `QuerySlice`, scanning directly into a single scannable `T` such as `string` or `int64`. Use either instead of the slice-returning form for exports and large scans where a full `[]T` would not fit comfortably in memory; for anything that already fits, the slice form remains simpler to use and costs no more.

```go
for row, err := range repo.SelectIter(ctx, "SELECT * FROM big_table") {
    if err != nil {
        return err
    }

    // process row, one at a time.
}
```

Result semantics are identical between the two functions. A query error (the initial `Query` call itself failing) yields exactly one pair, `(zero T, that error)`, and then the iterator stops. A row-scan error, or a `rows.Err()` surfaced after the last row, is reported the same way: one terminal `(zero T, err)` yield, after which the iterator stops; every row already yielded before that point is unaffected, only rows after the failure are lost. A yielded error is always paired with the zero value of `T`, never a partially populated one, so you can check `err` first without also checking the value. `QueryIter` does not replicate `QuerySlice`'s "drop every empty string" quirk (see Chapter 5): it yields every row exactly as `SelectIter` / `Select` do, so switching an existing `QuerySlice[string]` call to `QueryIter[string]` can change what you observe if you were relying on that quirk.

Single-Use Iterators and Connection Lifetime

The returned `iter.Seq2` is a closure that runs the query fresh every time you range over it; ranging over the same returned value twice runs the query twice, not the same results again. Treat it as single-use anyway, for a more concrete reason than the name suggests: while unconsumed rows remain, `SelectIter` / `QueryIter` hold a connection checked out from the pool, and pgx connections execute statements serially. A second statement issued on the same `*DB` before the first iteration is fully drained, or broken out of, blocks or fails rather than running concurrently, the same "conn busy" constraint `BeginConcurrent` (see Chapter 9) exists to work around. On a plain, non-transactional `*DB` backed by a pool with more than one connection, an unrelated second query can still proceed fine on a different pooled connection while one iteration is in flight, since the constraint is per-connection, not per-`*DB`; on a transaction-scoped `*DB`, there is exactly one connection to contend over, so finishing (or breaking out of) the iteration first is not optional.

The connection is released deterministically, via a deferred `rows.Close()` inside the iterator closure, in exactly one of three ways: the loop runs to completion, the query or a row scan fails (see above), or your range loop exits early (`break`, `return`, or the loop body returning `false` to `yield`, which range-over-func delivers as `yield` itself returning `false`). In every case the closure returns, its deferred `rows.Close()` runs, and the connection is released back to the pool (or, on a transaction-scoped `*DB`, simply left free for the transaction's next statement) before control returns to your code. Breaking out of a `SelectIter` / `QueryIter` loop needs no special cleanup of your own; the next query on the same `*DB` is safe to issue immediately.

Server-side cursors are deliberately not part of this API. pgx already streams query results row by row over the wire as `rows.Next()` is called; it does not buffer the entire result set in memory before `SelectIter` (or a plain `Select`) ever sees the first row, which is the property

`SelectIter`'s own memory behavior relies on. There is no missing piece here for the common "stream a big `SELECT` without blowing up memory" case.

The Server-Side Cursor Pattern

A real, standalone SQL `CURSOR` is a narrower tool than `SelectIter` covers: explicit server-side control over how much of a possibly expensive query plan PostgreSQL materializes per fetch, or holding a query open across multiple round trips interleaved with other statements on the same transaction. A cursor's lifetime is tied to its transaction, so reach for it by hand, inside one, when you specifically need that:

```go
repo := pg.NewRepository[BigRow](db)

err := repo.InTransaction(ctx, func(tx *pg.Repository[BigRow]) error {
    const declare = "DECLARE export_cursor CURSOR FOR " +
        "SELECT * FROM big_table"
    if _, err := tx.Exec(ctx, declare); err != nil {
        return err
    }

    for {
        rows, err := tx.Query(ctx, "FETCH 1000 FROM export_cursor")
        if err != nil {
            return err
        }

        // RowsToStruct closes rows itself.
        batch, err := desc.RowsToStruct[BigRow](tx.Table(), rows)
        if err != nil {
            return err
        }
        if len(batch) == 0 {
            return nil // exhausted.
        }

        // ... consume batch ...
    }
})
```

Each `FETCH` round trip reuses the same underlying scanning machinery `Repository[T].Select` uses (`desc.RowsToStruct` against `tx.Table()`'s descriptor), so a batch of rows scans exactly the same way it would from any other query. Reach for `SelectIter` first; reach for a hand-declared cursor only when you specifically need the per-fetch server-side control, or the ability to interleave other statements with the same open result set, that `SelectIter` does not offer.

Summary

- `desc.BuildCopyPlan` computes a `COPY` column list from a table descriptor and a batch of struct values, excluding `AutoGenerated` / `Presenter` columns exactly as the bulk `INSERT` builder does, and rejecting a password column with no Go-side encryption (`ErrCopyPassword`).
- A `Default`-bearing column is omitted from the `COPY` column list only when it is zero in every row of the batch, letting the database default fire for all of them; otherwise it is included and every row's value, zero or not, is sent verbatim. This is `COPY`'s central trade-off against `InsertMany`'s true per-row `DEFAULT`.
- `DB.CopyFrom` is a thin, transaction-aware wrapper over pgx's own `CopyFrom`; `Repository[T].CopyFrom` builds the plan and the `pgx.CopyFromSlice` source for you. Neither opens its own transaction; neither supports `ON CONFLICT` or `RETURNING`.
- `SelectIter` / `QueryIter` return an `iter.Seq2[T, error]` that decodes one row at a time without materializing a `[]T`; a query or scan error yields exactly one terminal `(zero T, err)` pair.
- The iterator holds a pooled (or transaction) connection for as long as rows remain unconsumed; breaking out of the range loop closes the rows and releases the connection before your code regains control, with no manual cleanup required.
- A server-side `SQL CURSOR`, declared and fetched by hand inside a transaction, remains available for the narrower cases `SelectIter` does not cover: explicit per-fetch control, or interleaving other statements with an open result set.

Further Reading

- PostgreSQL: COPY: the full `COPY` statement grammar and its format options.
- PostgreSQL: Populating a Database: PostgreSQL's own guidance on `COPY` versus `INSERT` for bulk loads.
- pgx: CopyFrom: the driver-level API `DB.CopyFrom` wraps.
- Go: iter package: `Seq` / `Seq2` and range-over-func, the mechanism behind `SelectIter` / `QueryIter`.
- PostgreSQL: DECLARE: the `CURSOR` statement behind the hand-declared streaming pattern.

CHAPTER 9

Transactions

A transaction groups several statements so that either all of them take effect or none do. pg wraps pgx's transaction primitives with a closure-based API, `DB.InTransaction`, that handles begin, commit, rollback and panic recovery for you, plus a retry variant for the transient errors PostgreSQL's SERIALIZABLE isolation level produces under contention. This chapter covers the full contract of `InTransaction`: what happens when the function you pass it returns an error, returns `ErrIntentionalRollback`, panics, or succeeds but the commit itself fails. It also covers a real bug this library once had in that last case, because the fix teaches something true about how Go defers and named results interact. Then it covers nesting (joining an outer transaction versus opening a savepoint), the generic `pg.InTransaction[R]` helper, concurrent access to a single transaction, and the retry machinery for SQLSTATE 40001 and 40P01. Every signature below was checked against `tx.go`, `db.go`, `retry.go`, `concurrent_tx.go`, `db_exec.go` and `repository.go`.

Background

If you have used SQL transactions before (`BEGIN`, `COMMIT`, `ROLLBACK`) and know what SERIALIZABLE isolation buys you, skip to DB.InTransaction. PostgreSQL, like every SQL database, lets you bracket a sequence of statements between `BEGIN` and `COMMIT` so they either all become visible to other connections at once or, on `ROLLBACK`, none of them do. This matters the moment a unit of work touches more than one row or more than one table: moving money between two accounts is two `UPDATE` statements, and a crash between them must not leave the database with money that appeared or vanished. Isolation levels control what a transaction is allowed to see of other, concurrently running transactions. PostgreSQL's default, READ COMMITTED, lets a transaction see rows committed by others since it began, which is usually fine but permits a class of anomaly called write skew: two transactions each read a value, each decide it is still safe to act on, and both commit, producing a result neither would have permitted alone. SERIALIZABLE isolation closes this gap by guaranteeing the outcome is

equivalent to some serial (one-at-a-time) ordering of the concurrent transactions, and it does so by aborting one of the conflicting transactions rather than blocking, which is why retry logic (covered later in this chapter) exists at all.

DB.InTransaction

`DB.InTransaction(ctx context.Context, fn func(*DB) error) error` runs `fn` inside a transaction and decides what to do based on what `fn` returns:

```go
const debit = `UPDATE accounts SET balance = balance - $1 WHERE id = $2`
const credit = `UPDATE accounts SET balance = balance + $1 WHERE id = $2`

err := db.InTransaction(ctx, func(tx *pg.DB) error {
    if _, err := tx.Exec(ctx, debit, amount, from); err != nil {
        return err
    }

    _, err := tx.Exec(ctx, credit, amount, to)
    return err
})
```

The contract, exactly as implemented:

- `fn` returns `nil`: the transaction commits, and **any error the commit itself returns is passed back to the caller**. A successful `fn` does not guarantee a successful `InTransaction`.
- `fn` returns `pg.ErrIntentionalRollback`: the transaction rolls back, and `InTransaction` returns `nil` on a successful rollback, or the rollback error otherwise. This is the documented way to abort a transaction deliberately without treating it as a failure, useful for a dry-run code path that wants every side effect of `fn` undone.
- `fn` returns any other non-nil error: the transaction rolls back and that error is returned. If the rollback itself also fails, both errors are combined with `errors.Join`, so `errors.Is` and `errors.As` still reach either one.
- `fn` panics: the transaction rolls back and the panic is re-thrown after the rollback completes. `InTransaction` does not swallow panics; it only makes sure the connection is not left in an in-transaction state before the panic continues unwinding.

The context argument governs `Begin`, `Commit` and `Rollback` directly, so a context that is already canceled or expired when `InTransaction` is called returns promptly (from the failing `Begin`) without ever invoking `fn`.

Why Commit Errors Must Propagate

The first bullet above, that a successful `fn` can still fail overall, used to be broken in this library, and the fix is worth understanding because it is a general Go lesson, not a pg-specific quirk. Here is the shape of the bug, reconstructed:

```go
// The buggy version (illustrative, not the real signature).
func (db *DB) InTransaction(ctx context.Context, fn func(*DB) error) error {
    tx, err := db.Begin(ctx)
    if err != nil {
        return err
    }

    defer func() {
        if err != nil {
            _ = tx.Rollback(ctx)
        } else {
            err = tx.Commit(ctx) // writes to the WRONG "err"
        }
    }()

    err = fn(tx)
    return err // the return VALUE was already copied here
}
```

`return err` copies the current value of `err` into the function's unnamed result at the moment it executes. The deferred closure then runs, and it can still assign to the variable named `err` (the same one `fn`'s result was stored in), but that assignment happens after the copy was already made. If `fn` succeeded, `err` was `nil` at `return err` time, the (already-decided) return value became `nil`, and the later `err = tx.Commit(ctx)` inside the deferred closure changed a variable nobody was going to read again. A commit failure, including a serialization failure detected only at commit time (see Retrying Transient Failures), looked identical to success from the caller's point of view: the row changes were never actually persisted, and nothing said so.

The fix is the named result in the real signature:

```go
func (db *DB) InTransaction(
    ctx context.Context, fn func(*DB) error,
) (err error) {
```

With a named result, `err` inside the function body and `err` in the deferred closure are the identical variable that Go returns to the caller: there is no intermediate copy for `return err` to freeze. The deferred closure's `err = tx.Commit(ctx)` now writes directly to the value the caller receives. This is precisely why Go lets a `defer` mutate a named result in the first place: it is the

sanctioned pattern for exactly this kind of “clean up, and possibly override the outcome based on what cleanup found” logic. Any function that commits, releases a resource, or otherwise needs its deferred cleanup to be able to fail the call should use a named result for the same reason.

Nesting: Join Versus Savepoint

Two different pg operations both look like “start another transaction,” and they behave differently on purpose. Getting this distinction backwards either silently merges work you meant to keep separate, or fails to give you the independent rollback you needed.

Calling `InTransaction` again on a `*DB` that is already inside a transaction joins the existing transaction. No `SAVEPOINT` is taken, no new transaction begins. `fn` runs directly against the same `*DB`, and any error it returns (including `ErrIntentionalRollback`) propagates out to whichever `InTransaction` call is actually managing the transaction (opened the real `BEGIN`), rather than being contained to the inner call:

```go
err := db.InTransaction(ctx, func(tx *pg.DB) error {
    // tx.IsTransaction() is already true here.
    return tx.InTransaction(ctx, func(tx2 *pg.DB) error {
        // tx2 IS tx. No savepoint. An error here rolls back
        // everything the outer call did too.
        return doSomething(ctx, tx2)
    })
})
```

The check that makes this happen is the first line of `InTransaction`:

```go
func (db *DB) InTransaction(
    ctx context.Context, fn func(*DB) error,
) (err error) {
    if db.IsTransaction() {
        return fn(db)
    }
    // ... Begin / defer commit-or-rollback / fn(tx) ...
}
```

Calling `DB.Begin` on a `*DB` that is already inside a transaction opens a savepoint-backed subtransaction instead. `Begin` detects the same condition (`db.tx != nil`) but takes the opposite action: it calls `db.tx.Begin(ctx)`, which is pgx’s own nested-transaction support, implemented as `SAVEPOINT sp_N`. The returned `*DB` has its own `Commit` (`RELEASE SAVEPOINT sp_N`) and `Rollback` (`ROLLBACK TO SAVEPOINT sp_N`), independent of the outer transaction:

```go
tx, err := db.Begin(ctx)
if err != nil {
    return err
}
defer tx.Rollback(ctx) // no-op after a successful Commit below.

sub, err := tx.Begin(ctx) // SAVEPOINT sp_1
if err != nil {
    return err
}

if err := riskyStep(ctx, sub); err != nil {
    sub.Rollback(ctx) // ROLLBACK TO SAVEPOINT sp_1; tx is unaffected.
} else {
    sub.Commit(ctx) // RELEASE SAVEPOINT sp_1
}

return tx.Commit(ctx)
```

Reach for `Begin` (not `InTransaction`) when you specifically need an inner unit of work that can fail and roll back on its own without undoing everything the outer transaction already did, for example attempting several independent, best-effort side effects inside one larger transaction. `BeginConcurrent` (below) follows the identical join-versus-savepoint rule.

Manual Control: Begin, Commit, Rollback, IsTransaction

`InTransaction` is a closure over the pattern of Begin, run, then Commit-or-Rollback. When you need to hold a transaction open across multiple function calls instead of one closure, use the pieces directly:

- `DB.Begin(ctx context.Context) (*DB, error)`: starts a transaction (or, per the previous section, a savepoint if `db` is already transactional) and returns a new `*DB` bound to it. The default `pgx.TxOptions{}` is used, meaning PostgreSQL's default isolation level, read committed.
- `DB.Commit(ctx context.Context) error`: commits. Calling it a second time (or calling it on a `*DB` whose transaction already closed through a prior `Commit` or `Rollback`) returns `nil` without issuing SQL, rather than an error, because the transaction is already closed. Calling it on a `*DB` with no transaction at all returns an error: `commit outside of a transaction`.
- `DB.Rollback(ctx context.Context) error`: the same idempotent behavior, mirrored for rollback.
- `DB.IsTransaction() bool`: reports whether this `*DB` is bound to a transaction (`db.tx != nil`). Both `InTransaction` and `Begin` consult it to decide whether to join, start fresh, or open a savepoint.

The `defer tx.Rollback(ctx)` idiom used in the savepoint example above works identically at the top level: it is safe to defer immediately after a successful `Begin` precisely because `Rollback` is a no-op once the transaction is already closed, so a later, successful `Commit` leaves the deferred `Rollback` nothing to do.

Repository InTransaction

`Repository[T].InTransaction(ctx, fn func(*Repository[T]) error) error` mirrors `DB.InTransaction` at the typed level: it commits or rolls back based on `fn`'s return value using the exact same rules, and it applies the exact same nesting short-circuit, checked against the repository's own `*DB` before delegating:

```go
func (repo *Repository[T]) InTransaction(
    ctx context.Context, fn func(*Repository[T]) error,
) error {
    if repo.db.IsTransaction() {
        return fn(repo)
    }

    return repo.db.InTransaction(ctx, func(db *DB) error {
        txRepo := &Repository[T]{db: db, td: repo.td}
        return fn(txRepo)
    })
}
```

`Repository[T].InsertMany` and `UpsertMany` build on this same method: each batches its rows into multi-row statements internally and calls `InTransaction` so a later batch's failure rolls back every earlier batch from the same call.

pg.InTransaction and Typed Wrappers

A real application usually has more than one `Repository[T]`, and code often groups several of them into a small struct so a service layer can depend on one thing instead of five:

```go
type Repositories struct {
    Customers *pg.Repository[Customer]
    Orders    *pg.Repository[Order]
}

func NewRepositories(db *pg.DB) *Repositories {
    return &Repositories{
        Customers: pg.NewRepository[Customer](db),
        Orders:    pg.NewRepository[Order](db),
    }
```

```
}
```

Without help, giving `*Repositories` its own transactional method means hand-writing the same three lines every such wrapper needs: open a transaction on the underlying `*DB`, reconstruct the wrapper type around that transactional `*DB`, and call the caller's function with it. `pg.InTransaction[R]` is exactly that boilerplate, generalized once:

```go
func InTransaction[R any](
    ctx context.Context, db *DB, wrap func(*DB) R, fn func(R) error,
) error {
    return db.InTransaction(ctx, func(tx *DB) error { return fn(wrap(tx)) })
}
```

Using it, `Repositories` needs one line instead of a hand-written method:

```go
func (r *Repositories) InTransaction(
    ctx context.Context, fn func(*Repositories) error,
) error {
    return pg.InTransaction(ctx, r.Customers.DB(), NewRepositories, fn)
}

// Usage:
err := repos.InTransaction(ctx, func(tx *Repositories) error {
    if err := tx.Customers.Insert(ctx, customer); err != nil {
        return err
    }

    return tx.Orders.Insert(ctx, order)
})
```

`wrap` is typically the wrapper's own constructor, exactly as `NewRepositories` is above: `pg.InTransaction` calls `db.InTransaction` once, calls `wrap(tx)` to rebuild `*Repositories` (or whatever `R` is) around the now-transactional `*DB`, then calls `fn` with that rebuilt value. If `db` is already inside a transaction, the nesting rule from Nesting: Join Versus Savepoint still applies through the inner `db.InTransaction` call: `fn` joins the existing transaction, and `wrap` still runs so `fn` receives a wrapper built from that same, already-transactional `*DB`.

Concurrent Access to One Transaction

A single PostgreSQL connection processes one statement at a time. A `pgx.Tx` is not safe to call from multiple goroutines concurrently, because each call sends bytes on the same connection and reads back a response before the next call may begin. `BeginConcurrent` and `ConcurrentTx` exist for the case where several goroutines need to issue statements against the same transaction without each hand-rolling a mutex:

```go
tx, err := db.BeginConcurrent(ctx)
if err != nil {
    return err
}
defer tx.Rollback(ctx)

var wg sync.WaitGroup
for _, id := range ids {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        tx.Exec(ctx, "UPDATE items SET touched = true WHERE id = $1", id)
    }(id)
}
wg.Wait()

return tx.Commit(ctx)
```

`BeginConcurrent` follows the same join/fresh rule as `Begin` (a nested call on an already-transactional `*DB` opens a savepoint via `db.tx.Begin`), but when it starts a brand new transaction, it does so through `NewConcurrentTx`, which wraps the returned `pgx.Tx` in a `*ConcurrentTx`: a `sync.Mutex`-guarded decorator implementing every `pgx.Tx` method by locking, delegating to the embedded `pgx.Tx`, and unlocking. This makes concurrent use **safe, not parallel**: goroutines calling methods on the same `*ConcurrentTx` never corrupt the connection or race each other, but they still execute one at a time, serialized by the mutex, exactly as a single PostgreSQL connection requires. It solves the “conn busy” panic pgx raises when two goroutines write to the same connection at once; it does not make your queries run any faster than a single connection can. Its `LargeObjects()` and `Conn()` methods are the one exception: each returns a value that is itself unsynchronized, so a `pgx.LargeObjects` or `*pgx.Conn` obtained through a `*ConcurrentTx` needs its own locking if shared further.

Retrying Transient Failures

Under SERIALIZABLE isolation, PostgreSQL sometimes cannot allow a transaction to commit even though every individual statement in it succeeded, because doing so would produce a result inconsistent with any one-at-a-time ordering of the transactions that ran concurrently with it. When PostgreSQL detects this, it reports SQLSTATE `40001` (`serialization_failure`), and the correct response is not to inspect the error, but to retry the whole transaction from scratch. A related code, `40P01` (`deadlock_detected`), is reported when PostgreSQL picks one of two mutually blocked transactions as the victim to break a deadlock; it too is safe to retry.

`IsErrRetryableTx(err error) bool` recognizes both:

```go
func IsErrRetryableTx(err error) bool {
    pgErr, ok := asPgError(err)
    if !ok {
        return false
    }

    switch pgErr.Code {
    case "40001", "40P01":
        return true
    default:
        return false
    }
}
```

It checks the driver's structured `*pgconn.PgError.Code` (via `errors.As`, so it still matches an error wrapped several layers deep, including one that surfaced from `Commit` rather than from one of the transaction's own statements), never English message text. This is the same SQLSTATE-first approach Chapter 10 covers for every other error classifier in the package. No other member of PostgreSQL's `40` (`transaction_rollback`) class is treated as retryable; retrying `40000` or `40003` blindly is not generally safe, so a caller who wants to retry on more conditions supplies their own classifier that also calls `IsErrRetryableTx`, rather than this function growing an ever-expanding list.

RetryOptions

`RetryOptions` configures `InTransactionRetry`. Every field's zero value falls back to a documented default:

Field	Type	Zero-value default
`MaxAttempts`	`int`	3. A caller-supplied negative value (not zero) is treated as 1, an explicit "no retries," rather than silently upgraded to 3.
`BaseDelay`	`time.Duration`	50ms. The upper bound for the first retry's jittered wait.
`MaxDelay`	`time.Duration`	1s. The ceiling the doubling backoff bound saturates at.
`TxOptions`	`pgx.TxOptions`	zero value (read committed). Reused unchanged on every attempt.
`IsRetryable`	`func(error) bool`	`IsErrRetryableTx`. A non-nil value fully replaces the default; it is not combined with it.

The `TxOptions` field deserves a specific callout: PostgreSQL's default isolation level is read committed, under which SQLSTATE `40001` mostly cannot occur in the first place, since read committed does not perform the snapshot-based conflict detection SERIALIZABLE does. If the

whole point of calling `InTransactionRetry` is to retry around serialization failures, set `TxOptions: pgx.TxOptions{IsoLevel: pgx.Serializable}`, or the classifier will rarely have anything to classify:

```go
opts := pg.RetryOptions{
    MaxAttempts: 5,
    TxOptions:   pgx.TxOptions{IsoLevel: pgx.Serializable},
}
```

InTransactionRetry

`DB.InTransactionRetry(ctx, opts RetryOptions, fn func(*DB) error) error` and its typed counterpart `Repository[T].InTransactionRetry(ctx, opts RetryOptions, fn func(*Repository[T]) error) error` run `fn` in a fresh transaction, and if it fails with an error `opts.IsRetryable` (or `IsErrRetryableTx` by default) accepts, retry the whole thing, waiting out a backoff between attempts:

```go
err := db.InTransactionRetry(ctx, pg.RetryOptions{
    TxOptions: pgx.TxOptions{IsoLevel: pgx.Serializable},
}, func(tx *pg.DB) error {
    // Re-read whatever this needs on every attempt: nothing from a
    // failed attempt carries over except what fn re-derives here.
    return transferFunds(ctx, tx, from, to, amount)
})
```

Every attempt opens a brand new transaction and runs `fn` from scratch, internally reusing the exact commit/rollback/panic machinery `InTransaction` uses (including the same named-return trick from Why Commit Errors Must Propagate, reimplemented in `runInTransactionOnce` specifically because `InTransactionRetry` needs `RetryOptions.TxOptions` to reach `BeginTx` on each attempt, which the plain `Begin` / `InTransaction` path has no way to accept). Nothing a failed attempt wrote to variables `fn` closes over survives into the next attempt except what `fn` itself re-reads from the database, so `fn` must not assume it only runs once. `fn` returning `ErrIntentionalRollback` is never retried, the same as for `InTransaction`: the transaction rolls back and the call returns `nil` (or the rollback error) immediately.

If the context is canceled or expires while waiting out a backoff between attempts, `InTransactionRetry` returns promptly with `errors.Join(ctx.Err(), lastAttemptErr)` instead of continuing to retry against a context nobody wants to wait on anymore.

Full Jitter Backoff

Between a failed, retryable attempt and the next one, `InTransactionRetry` waits a randomized duration following the "full jitter" algorithm from AWS's exponential backoff article: the bound doubles with each failed attempt, capped at `MaxDelay`, and the actual wait is chosen uniformly at random between zero and that bound.

```
bound = min(MaxDelay, BaseDelay << (attempt-1))
delay = random value in [0, bound)
```

With the defaults (`BaseDelay` 50ms, `MaxDelay` 1s), the bound for the wait before the second attempt is 50ms, before the third is 100ms, and so on, doubling until it saturates at 1s. Choosing uniformly at random within the bound, rather than always waiting the full bound, is the "jitter" part: it keeps several clients that failed at the same instant (a common case, since a serialization failure often means they were genuinely contending for the same rows) from retrying in lockstep and immediately colliding again. The left shift that computes the doubling bound is guarded against overflowing `time.Duration`'s underlying `int64`, saturating at `math.MaxInt64` nanoseconds rather than wrapping around to a small or negative duration after enough attempts.

Nested Retry Calls Run Once

`InTransactionRetry` applies the same `if db.IsTransaction() { return fn(db) }` nesting short-circuit `InTransaction` uses, but for a stronger reason than mere convenience: if `db.IsTransaction()` is already true, `InTransactionRetry` runs `fn` exactly once, with no retry, full stop. A transaction that is already a subtransaction of some outer transaction cannot be meaningfully retried in isolation: rolling it back and reopening it (there is no "sub-BEGIN" a nested call could redo) would still leave the outer transaction, and everything it already did, exactly where it was. Retrying only makes sense for a transaction that owns its own `BEGIN` / `COMMIT`, so nesting `InTransactionRetry` calls is not an optimization that happens to skip redundant retries: it would be wrong to retry here, and the short-circuit exists to make that impossible rather than merely unlikely. To get real retries, call `InTransactionRetry` on a `*DB` that is not already inside a transaction.

`Repository[T].InTransactionRetry` does not gate on `repo.IsReadOnly()` the way the write-capable `Insert` / `Update` / `Delete` family does, because `fn` may legitimately be read-only: SQLSTATE `40001` is raised for read-write conflicts under SERIALIZABLE isolation, not exclusively for writes, so a read-only, repeatable-read- sensitive query is just as valid a candidate for retry.

ExecMany

`DB.ExecMany(ctx context.Context, queries ...string) error` runs each statement in `queries`, in order, inside a single transaction (joining the current one if `db` is already transactional, per the usual `InTransaction` nesting rule):

```go
err := db.ExecMany(ctx,
    `ALTER TABLE orders ADD COLUMN priority int DEFAULT 0`,
    `UPDATE orders SET priority = 1 WHERE rush = true`,
)
```

Statements are sent one `Exec` call at a time, not concatenated into one string, because pgx's extended query protocol cannot prepare a string containing multiple statements. If any statement fails, the whole transaction rolls back (or, if `ExecMany` joined an already-open transaction, that transaction is left aborted for whoever started it to roll back, the same as any other error surfaced from a function passed to `InTransaction`), and none of the earlier statements in the same call persist. Given zero queries, `ExecMany` does nothing and returns `nil` without opening a transaction at all.

SetConstraintsDeferred

`DB.SetConstraintsDeferred(ctx context.Context, constraints ...string) error` issues PostgreSQL's `SET CONSTRAINTS ... DEFERRED`, which postpones checking the named deferrable constraints (or every deferrable constraint, with no arguments) until the transaction commits instead of checking them after each statement. This matters when a batch of statements is only individually valid at the end: for example, swapping two rows' unique keys requires, for one moment mid-batch, that both rows hold the other's key, which a normal per-statement unique check would reject.

```go
const swapCode = `UPDATE items SET code = $1 WHERE id = $2`

err := db.InTransaction(ctx, func(tx *pg.DB) error {
    if err := tx.SetConstraintsDeferred(ctx); err != nil {
        return err
    }

    // Both updates would violate the unique constraint if checked
    // individually; deferring lets PostgreSQL check it once, at
    // COMMIT, after both have run.
    if _, err := tx.Exec(ctx, swapCode, codeB, idA); err != nil {
        return err
```

```
    }

    _, err := tx.Exec(ctx, swapCode, codeA, idB)
    return err
})
```

`SetConstraintsDeferred` checks `DB.IsTransaction()` itself and returns a descriptive error, `set constraints deferred: not inside a transaction`, before issuing any SQL if `db` is not transactional, since PostgreSQL would otherwise reject the statement anyway (there is no "remainder of the current transaction" outside one) with a less specific error. Naming a constraint that exists but is not itself declared `DEFERRABLE`, or one that does not exist at all, is still rejected by PostgreSQL: `SetConstraintsDeferred` does not cross-check constraint names against the registered `Schema`, since constraints are not modeled there the way tables and columns are. Every constraint name passed is quoted with `QuoteIdentifier` before being embedded in the generated statement.

Summary

- `DB.InTransaction` commits on `nil`, rolls back and returns `nil` on `ErrIntentionalRollback`, rolls back and returns the error otherwise (joined with any rollback error), and rolls back and re-panics on a panic. A commit failure is returned to the caller even when `fn` succeeded.
- The commit-error propagation depends on `InTransaction`'s named `(err error)` result: a deferred closure can only override what the caller receives when it is writing to the same variable the function returns, not a copy already frozen by an unnamed `return err`.
- Calling `InTransaction` again on an already-transactional `*DB` joins the existing transaction (no savepoint); calling `Begin` on one opens a real, independently committable/rollback-able savepoint-backed subtransaction via pgx's nested `Tx.Begin`.
- `Repository[T].InTransaction` mirrors `DB.InTransaction` at the typed level; `pg.InTransaction[R]` generalizes the "open a transaction, rebuild my typed wrapper around it" pattern so a multi-repository aggregate needs one line instead of a hand-written method.
- `BeginConcurrent` / `ConcurrentTx` make a transaction safe to touch from several goroutines by serializing every call through a mutex. Safe, not parallel: PostgreSQL connections process one statement at a time regardless.
- `IsErrRetryableTx` recognizes SQLSTATE `40001` (`serialization_failure`) and `40P01` (`deadlock_detected`). `InTransactionRetry` retries a fresh transaction per attempt with full-jitter exponential backoff (`RetryOptions`), because a serialization failure is frequently only detectable when the transaction tries to commit. A nested `InTransactionRetry` call runs `fn`

exactly once: retrying a subtransaction in isolation cannot undo what its outer transaction already did.

- `ExecMany` runs several statements in one transaction; `SetConstraintsDeferred` postpones deferrable constraint checks to commit time, for batches that are only valid once every statement in them has run.

Further Reading

- PostgreSQL: Transaction Isolation: read committed, repeatable read and serializable, and the anomalies each one permits or forbids.
- PostgreSQL: Explicit Locking, Advisory Locks: background for the advisory lock `DB.Migrate` takes, covered in Chapter 11.
- PostgreSQL: SAVEPOINT: the statement pgx issues under the hood for a nested `Begin`.
- PostgreSQL: SET CONSTRAINTS: the statement `SetConstraintsDeferred` builds.
- PostgreSQL Error Codes: Class 40: the full `transaction_rollback` class `IsErrRetryableTx` picks two members out of.
- AWS Architecture Blog: Exponential Backoff and Jitter: the “full jitter” algorithm `backoffDelay` implements.
- Go: Defer, Panic and Recover: the mechanics behind the named-result fix in this chapter.

CHAPTER 10

Errors

PostgreSQL does not report failures as English sentences you are meant to parse. Every error the server sends back carries a five-character SQLSTATE code, defined by the SQL standard and extended by PostgreSQL, and that code is the one thing about an error PostgreSQL guarantees will not change. The message text attached to it can, and does: it changes with `lc_messages`, with the server's language settings, and occasionally between PostgreSQL versions, so code that matches on a substring of the English message is one `LC_MESSAGES=de_DE` deployment away from silently breaking. This chapter covers how pg classifies PostgreSQL errors: `ErrNoRows`, the constraint-violation classifiers (`IsErrDuplicate`, `IsErrForeignKey`, `IsErrInputSyntax`, `IsErrColumnNotExists`), the typed `ConstraintError` / `AsConstraintError` pair, and the package's other exported sentinel errors. Every function signature below is read from `errors.go`, `db.go`, `listener.go` and `repository.go`.

Background

If you already know what SQLSTATE is and why matching on error message text is fragile, skip to ErrNoRows. A SQLSTATE is a five-character code (letters and digits, e.g. `23505`) that PostgreSQL attaches to every error and notice it produces. The first two characters name a broad class (`23` is integrity constraint violation, `40` is transaction rollback, `42` is syntax error or access rule violation), and the full five characters name a specific condition within that class. Client drivers, including pgx, expose this code on the structured error value they return (`*pgconn.PgError.Code` in pgx's case) alongside a human-readable `Message` meant for logs and terminals, not for `if strings.Contains(err.Error(), "...")` checks. The code is part of PostgreSQL's wire protocol contract and is stable across locales, versions and even other PostgreSQL-compatible databases that implement the same protocol; the message text is not

part of that contract at all. Every classifier in this chapter checks the code first, and its fallback to message-text checks exists purely for errors that never made it through the driver's typed error value in the first place.

ErrNoRows

```go
var ErrNoRows = pgx.ErrNoRows
```

`ErrNoRows` is a direct alias for pgx's own sentinel, returned by `QueryRow(...).Scan(...)` and by every `Repository[T]` / `DB` method that expects exactly one row and found none. `IsErrNoRows(err error) bool` wraps the standard-library check:

```go
func IsErrNoRows(err error) bool {
    return errors.Is(err, ErrNoRows)
}
```

Because it is `errors.Is`, `IsErrNoRows` still recognizes `ErrNoRows` under any number of layers of `fmt.Errorf("...: %w", err)` wrapping, so a repository method that adds context to the error before returning it does not break callers checking for "no rows" further up the stack. `DB.Count` swallows `ErrNoRows` itself and returns `(0, nil)` instead, since a `COUNT` wrapped in a `GROUP BY` that matches nothing is a legitimate zero, not a failure a caller should have to special-case.

The Classifiers

Four package-level functions turn a PostgreSQL error into a specific, actionable answer instead of an opaque `error`. All four follow the same shape: check whether `err` wraps a `*pgconn.PgError` (via the unexported `asPgError` helper, itself `errors.As` under the hood, so it matches an error wrapped several layers deep); if so, classify by SQLSTATE `Code`; if not, fall back to the original substring checks against `err.Error()`, kept only for compatibility with errors that never carried a structured `*pgconn.PgError` in the first place (already-formatted strings, or an older driver).

`IsErrDuplicate(err error) (string, bool)` reports whether an insert or update failed a unique constraint, returning the constraint's name:

```go
if name, ok := pg.IsErrDuplicate(err); ok {
    log.Printf("unique constraint %q violated", name)
}
```

SQLSTATE path: delegates to `AsConstraintError` and reports true only when the resulting `Kind` is `ConstraintUnique` (`23505`), returning its `ConstraintName` field directly.

`IsErrForeignKey(err error) (string, bool)` reports whether an insert or update referenced a row that does not exist, or deleted/updated a row still referenced elsewhere, returning the constraint's name. SQLSTATE path: `AsConstraintError` again, checking for `ConstraintForeignKey` (`23503`).

`IsErrInputSyntax(err error) (string, bool)` reports whether a value could not be parsed as its column's PostgreSQL type, returning the offending value (extracted from the quoted substring in the server's message) or the generic string `"invalid input syntax"` when no quoted value is present. SQLSTATE path: `22P02` (`invalid_text_representation`) is the primary signal, but PostgreSQL reports a malformed `tsquery` under the much more generic `42601` (`syntax_error`), which does not distinguish a tsquery problem from any other syntax error by code alone; for that case, `IsErrInputSyntax` also checks the structured `Message` field for `"syntax error in tsquery"` or `"no operand in tsquery"` even when a `*pgconn.PgError` is available, since the code alone is not specific enough here.

`IsErrColumnNotExists(err error, col string) bool` reports whether a query referenced a column, named `col`, that does not exist. SQLSTATE path: `42703` (`undefined_column`), cross-checked against the structured `Message` containing `column "<col>" does not exist` so the function only matches the specific column asked about, not any undefined-column error.

```go
if pg.IsErrColumnNotExists(err, "email") {
    // the query referenced a column that was renamed or dropped.
}
```

ConstraintKind

`ConstraintKind` is a `string` type classifying which family of integrity-constraint violation (SQLSTATE class `23`) occurred:

Constant	SQLSTATE	Meaning
`ConstraintUnique`	`23505` (`unique_violation`)	a row would duplicate an existing value in a unique index or constraint.

`ConstraintForeignKey`	`23503` (`foreign_key_violation`)	a row references a value absent from the referenced table, or a referenced row was deleted/updated while still referenced.
`ConstraintNotNull`	`23502` (`not_null_violation`)	a `NOT NULL` column was given a null value.
`ConstraintCheck`	`23514` (`check_violation`)	a row failed a `CHECK` constraint.
`ConstraintExclusion`	`23P01` (`exclusion_violation`)	a row conflicted with an existing row under an `EXCLUDE` constraint.

`Kind` is left empty (`""`) for a class-`23` SQLSTATE that is not one of these five, such as the generic `23000` (`integrity_constraint_violation`); `AsConstraintError` still reports `ok=true` for it, since it is still, unambiguously, an integrity violation, just not one of the five PostgreSQL commonly names more specifically.

ConstraintError

`ConstraintError` is a typed view over a PostgreSQL class-`23` error, built from the driver's `*pgconn.PgError` so a caller stops parsing `DETAIL` text by hand:

```go
type ConstraintError struct {
    Kind           ConstraintKind
    ConstraintName string
    TableName      string
    ColumnName     string
    Detail         string
    Code           string
    // unexported cause *pgconn.PgError
}
```

- `Kind`: one of the five `ConstraintKind` constants, or empty.
- `ConstraintName`: the violated constraint or index name, e.g. `"customers_email_key"`. May be empty if PostgreSQL did not report one.
- `TableName`: the table the violation occurred on. May be empty.
- `ColumnName`: the offending column. PostgreSQL only populates this for not-null violations (`ConstraintNotNull`); it is empty for every other kind.
- `Detail`: the server's `DETAIL` line, e.g. `Key (email)=(x@y.com) already exists.` for a unique violation. May be empty.
- `Code`: the raw SQLSTATE, e.g. `"23505"`.

`(*ConstraintError).Error() string` renders a compact one-line summary, e.g. `unique constraint "customers_email_key" on table "customers": Key (email)=(x@y.com) already exists.`, and `(*ConstraintError).Unwrap() error` returns the underlying `*pgconn.PgError`, so `errors.As(err, &pgErr)` still reaches it through a returned `*ConstraintError`.

AsConstraintError Is Extraction-Only

```go
func AsConstraintError(err error) (*ConstraintError, bool)
```

`AsConstraintError` reports whether `err` (or anything it wraps, per `errors.As`) is a PostgreSQL integrity-constraint violation, a `*pgconn.PgError` whose `Code` begins with `"23"`, and if so returns its typed form.

The important design decision is what `AsConstraintError` is *not*: pg never wraps the errors it returns from `Insert`, `Update`, `Delete` and friends in a `ConstraintError` itself. A failed `Insert` still returns the plain, driver-level error, exactly as pgx produced it. `AsConstraintError` is purely an extraction function you call yourself, at whichever layer in your application actually needs to turn a database failure into a decision: an HTTP handler choosing a status code, a GraphQL resolver choosing an error type, a background job deciding whether to retry. This keeps the typed view out of the data path entirely: a repository method's returned `error` is always just `error`, checkable with the plain classifiers above or unwrapped manually, and only the layer that actually maps errors to responses pays for constructing a `ConstraintError`.

```go
err := customers.Insert(ctx, newCustomer)
if err != nil {
    if constraintErr, ok := pg.AsConstraintError(err); ok {
        // constraintErr.Kind, .ConstraintName, .TableName, .Detail
        // are all populated here; err itself is untouched.
    }
}
```

Mapping Errors to an HTTP Response

A handler layer typically wants one switch that turns any error a repository call might return into the right HTTP status and a message safe to show a client. `AsConstraintError` plus the classifiers above are enough to build it without ever inspecting `err.Error()` text:

```go
func writeError(w http.ResponseWriter, err error) {
    switch {
```

```
    case pg.IsErrNoRows(err):
        http.Error(w, "not found", http.StatusNotFound)
    case func() bool {
        ce, ok := pg.AsConstraintError(err)
        return ok && ce.Kind == pg.ConstraintUnique
    }():
        http.Error(w, "already exists", http.StatusConflict)
    case func() bool {
        ce, ok := pg.AsConstraintError(err)
        return ok && ce.Kind == pg.ConstraintForeignKey
    }():
        http.Error(w, "invalid reference", http.StatusBadRequest)
    case func() bool {
        _, ok := pg.IsErrInputSyntax(err)
        return ok
    }():
        http.Error(w, "malformed input", http.StatusBadRequest)
    default:
        log.Printf("unhandled db error: %v", err) // detail stays server-side.
        http.Error(w, "internal error", http.StatusInternalServerError)
    }
}
```

The inline closures above exist only to keep the `switch` flat; a real handler would more naturally call `AsConstraintError` once up front and branch on `constraintErr.Kind`:

```go
if constraintErr, ok := pg.AsConstraintError(err); ok {
    switch constraintErr.Kind {
    case pg.ConstraintUnique:
        http.Error(w, "already exists", http.StatusConflict)
    case pg.ConstraintForeignKey:
        http.Error(w, "invalid reference", http.StatusBadRequest)
    case pg.ConstraintNotNull, pg.ConstraintCheck:
        http.Error(w, "invalid input", http.StatusBadRequest)
    default:
        http.Error(w, "constraint violation", http.StatusConflict)
    }
    return
}
```

Either way, the client only ever sees a stable status code and a generic message; `constraintErr.Detail` and `constraintErr.TableName` (which can reveal schema shape) stay in the server log, never in the response body.

Other Exported Sentinels

Three more exported sentinel errors live outside `errors.go`, each tied to the specific feature it signals:

Sentinel	Declared in	Returned when

`ErrIntentionalRollback`	`db.go`	returned from the function passed to `InTransaction` / `InTransactionRetry` to force a rollback without treating it as a failure; see Chapter 9.
`ErrIsReadOnly`	`repository.go`	`Repository[T].Insert` / `InsertSingle` is called against a table registered as read-only (e.g. a view).
`ErrEmptyPayload`	`listener.go`	`Listener.Accept` receives a notification whose payload is empty; see Chapter 12.

All three are plain `errors.New` values, compared with `errors.Is` like any other sentinel; none of them carries a SQLSTATE, since none of them represents a PostgreSQL-side condition. `ErrIntentionalRollback` and `ErrIsReadOnly` are pg's own control-flow signals, and `ErrEmptyPayload` reports a client-side observation about a notification payload, not something the server itself rejected.

SQLSTATE Lookup Table

Every SQLSTATE this chapter (and Chapter 9's retry classifier) checks by code, gathered in one place:

SQLSTATE	Name	Recognized by
`22P02`	invalid_text_representation	`IsErrInputSyntax`
`23502`	not_null_violation	`AsConstraintError` (`ConstraintNotNull`)
`23503`	foreign_key_violation	`IsErrForeignKey`, `AsConstraintError` (`ConstraintForeignKey`)
`23505`	unique_violation	`IsErrDuplicate`, `AsConstraintError` (`ConstraintUnique`)
`23514`	check_violation	`AsConstraintError` (`ConstraintCheck`)
`23P01`	exclusion_violation	`AsConstraintError` (`ConstraintExclusion`)
`40001`	serialization_failure	`IsErrRetryableTx` (Chapter 9)
`40P01`	deadlock_detected	`IsErrRetryableTx` (Chapter 9)
`42601`	syntax_error	`IsErrInputSyntax` (tsquery fallback, message-text)
`42703`	undefined_column	`IsErrColumnNotExists`

Any other class-`23` code (e.g. the generic `23000`) still resolves through `AsConstraintError` with an empty `Kind`; every other SQLSTATE outside this table has no dedicated classifier in pg and surfaces as a plain error carrying a `*pgconn.PgError` you can inspect with `errors.As` yourself.

Summary

- PostgreSQL errors carry a stable SQLSTATE code; matching on English message text breaks under a different `lc_messages` or a driver upgrade. Every classifier in this chapter checks the code first and only falls back to message text for errors without a structured `*pgconn.PgError`.
- `ErrNoRows` (`= pgx.ErrNoRows`) and `IsErrNoRows` cover the no-rows case; `DB.Count` swallows it and reports zero instead.
- `IsErrDuplicate`, `IsErrForeignKey`, `IsErrInputSyntax` and `IsErrColumnNotExists` classify the common cases and return the relevant name/value alongside a `bool`.
- `ConstraintKind` names five class-`23` violations by SQLSTATE; `ConstraintError` is the typed, extraction-only view `AsConstraintError` builds from a `*pgconn.PgError`. pg never wraps its own returned errors in a `ConstraintError`; you call `AsConstraintError` at the layer that turns a database error into a response.
- `ErrIntentionalRollback`, `ErrIsReadOnly` and `ErrEmptyPayload` are pg's own control-flow sentinels, unrelated to any SQLSTATE.

Further Reading

- PostgreSQL: Error Codes: the full, authoritative SQLSTATE table this chapter's lookup table is a subset of.
- PostgreSQL: Reporting Errors and Messages (PL/pgSQL): how SQLSTATE and message text relate on the server side that produces the errors this chapter classifies.
- pgconn.PgError: the driver type every classifier in this chapter reads `Code`, `Message`, `ConstraintName`, `TableName`, `ColumnName` and `Detail` from.
- Go: errors.Is and errors.As: the wrapping-aware comparisons `IsErrNoRows` and every classifier in this chapter build on.
- RFC 9110: HTTP Semantics, Status Codes: background for choosing a status code in the HTTP mapping example.

CHAPTER 11

Schema Management and Migrations

A `Schema` built with `MustRegister` (Chapter 2) exists only in Go memory until something turns it into DDL a live PostgreSQL database can execute. `DB.CreateSchema` does that: it reads every registered table and emits the `CREATE SCHEMA`, extension, table, foreign key, function and trigger statements needed to stand up a fresh database that matches your structs, all inside one transaction. `DB.CheckSchema` does the reverse comparison, reading a live database back and reporting where it disagrees with the registered Go structs. This chapter covers both, along with `DeleteSchema`, the extensions the library creates on demand, and the `set_timestamp` trigger convention for `updated_at` columns. The second half covers `DB.Migrate`, a deliberately small, forward-only migration runner for `.sql` files, and is honest about the ground it does not cover: no down migrations, no checksums, no out-of-order detection. Every function and behavior described here is read from `db_information.go` and `migrate.go`.

Background

If you already know what DDL (Data Definition Language) is and have run a migration tool before, skip to CreateSchema. SQL splits into DML (Data Manipulation Language: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, the statements that read and write rows) and DDL (`CREATE TABLE`, `ALTER TABLE`, `CREATE INDEX`, the statements that define the shape a table's rows must take). PostgreSQL's DDL is transactional: unlike some databases, `CREATE TABLE` and `ALTER TABLE` inside a `BEGIN` / `COMMIT` block roll back cleanly on error, exactly like any `INSERT`. This is what makes it safe for `CreateSchema` and `Migrate` (below) to run a whole batch of DDL statements in one transaction and trust that a failure partway through undoes everything, rather than leaving the database in a half-built state. A "migration" is simply a versioned, ordered unit of DDL (and sometimes DML, e.g. a data backfill) applied once to move a database from one known schema state to the next; a migration runner's job is tracking which ones have already run so it never applies the same one twice.

CreateSchema

`DB.CreateSchema(ctx context.Context) error` builds the full DDL for every registered table and runs it in one transaction:

```go
schema := pg.NewSchema()
schema.MustRegister("customers", Customer{})

db, err := pg.Open(ctx, schema, connString)
if err != nil {
    log.Fatal(err)
}

if err := db.CreateSchema(ctx); err != nil {
    log.Fatal(err)
}
```

Internally it calls `CreateSchemaDumpSQL` to build the full SQL text, then runs that text inside `db.InTransaction` (see Chapter 9): if anything fails, the whole batch rolls back and the error is returned wrapped with the generated SQL attached, `%w:\n%s`, so a failed `CreateSchema` call tells you exactly what it tried to run.

CreateSchemaDumpSQL

`DB.CreateSchemaDumpSQL(ctx context.Context) (string, error)` builds the same SQL `CreateSchema` executes, but returns it as a string instead of running it, useful for reviewing the generated DDL, keeping it as a reference file, or handing it to a DBA who applies schema changes by hand. It runs four dumpers in order, each appending to the same `strings.Builder`:

1. `CREATE SCHEMA IF NOT EXISTS <search_path>;` (the search path itself, e.g. `public`, validated as a bare identifier before being interpolated, since it cannot be defended with `QuoteIdentifier` the way other identifiers in this package are: it is deliberately emitted unquoted so PostgreSQL's normal case-folding still applies for existing callers).
2. The extensions the schema needs (below).
3. `CREATE TABLE` for every registered, non-read-only table, followed by a second pass adding every table's foreign keys, so that table creation order never has to match reference order.
4. The `set_timestamp` function and per-table triggers (below).

Tables registered as read-only (views, presenters; see `pg.View` and `pg.Presenter` in Chapter 2) are skipped by both the table and foreign key passes: pg assumes you create and maintain a view's defining query yourself, and only ever reads from it.

Extensions Created on Demand

`CreateSchemaDumpSQL` inspects the registered schema and emits `CREATE EXTENSION IF NOT EXISTS ...` for exactly the extensions your column types need, never unconditionally:

Extension	Emitted when
`pgcrypto`	any registered column uses the `desc.UUID` data type, or any table has a password-hashed column (`pg:"password"`).
`citext`	any registered column uses `desc.CIText` (case-insensitive text).
`hstore`	any registered column uses `desc.HStore`.

`pgcrypto` backs both `gen_random_uuid()` (for a UUID primary key's default) and `crypt()` / `gen_salt()` (for password columns, used by `SelectByUsernameAndPassword` in Chapter 4). If none of your tables use any of these three types, none of the three `CREATE EXTENSION` statements is emitted at all.

The set_timestamp Trigger Convention

If a table has a column named `updated_at` (configurable via `Schema.UpdatedAtColumnName`) whose type is a time type (`column.Type.IsTime()`), `CreateSchemaDumpSQL` automatically wires up a trigger that stamps it with `NOW()` on every `UPDATE`, so application code never has to remember to set it by hand. The function is created once, shared by every table's trigger:

```sql
CREATE OR REPLACE FUNCTION trigger_set_timestamp()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = NOW();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

and each qualifying table gets its own trigger naming that shared function:

```sql
CREATE TRIGGER set_timestamp
BEFORE UPDATE ON customers
```

```
FOR EACH ROW
EXECUTE PROCEDURE trigger_set_timestamp();
```

Both names come from `Schema` fields: `SetTimestampTriggerName` (default `"set_timestamp"`, used for both the trigger and, prefixed with `trigger_`, the function) and `UpdatedAtColumnName` (default `"updated_at"`). Setting either to an empty string disables the whole feature: no function or trigger is registered for any table. `CreateSchemaDumpSQL` also calls `DB.ListTriggers` first and skips a table that already has a trigger of that name, so calling `CreateSchema` again against a database that already has the triggers installed does not fail or duplicate them (the statement itself is `CREATE OR REPLACE`, but the skip check also avoids the round trip).

CheckSchema

`DB.CheckSchema(ctx context.Context) error` reads the live database back through `ListTables` and compares it, column by column, against the registered `Schema`. It exists to catch drift: a database that was altered by hand, a migration that never ran, a struct tag someone edited without updating the database to match.

What it detects:

- **A missing or extra registered table.** It asks `ListTables` for exactly the table names in the schema; if PostgreSQL returns a different count, `CheckSchema` fails immediately with `expected %d tables, got %d` before checking a single column.
- **A database column with no matching code column.** For each table, it walks the columns PostgreSQL actually reports and looks each one up by name in the registered `Table`; a database column that has no code counterpart fails with `column %q in table %q not found in schema`.
- **A mismatched column definition.** For every column present on both sides, it renders each one's full `pg` struct tag representation (name, type, nullable/default, primary key, unique, indexes, foreign key reference, check constraint, and more, via `Column.FieldTagString(false)`) and compares them case-insensitively; a difference anywhere in that tag fails with the two tag strings shown side by side.

What it does not detect:

- **A code column with no database counterpart.** `CheckSchema` walks the columns PostgreSQL reports, not the columns the Go struct declares, so a field you added to a struct but never migrated into the actual table passes silently: nothing about the extra Go field is visited by

the comparison. Running `CreateSchema` / `Migrate` before `CheckSchema`, not after, is what actually catches this case, since the missing column becomes DDL that either runs or fails.

- **Anything not folded into the struct tag string:** trigger presence, table-level comments beyond `Description`, row-level security policies, table or column privileges, and any index or constraint not modeled by pg's own tag vocabulary.
- **Presenter tables.** `desc.TableTypePresenter` (custom hand-written `SELECT` queries with no backing relation) is excluded from `desc.DatabaseTableTypes`, the type set `CheckSchema` compares against, since a presenter has no real table for PostgreSQL to report back.

`CheckSchema` also tolerates one specific, common false positive: a column that participates in a composite or hand-made unique index (commonly `<table>_<column>_fkey` on a junction table) is not required to have that index declared explicitly in code, so long as the code side has no conflicting index declaration of its own.

DeleteSchema

`DB.DeleteSchema(ctx context.Context) error` drops the whole schema:

```go
query := `DROP SCHEMA IF EXISTS ` + QuoteIdentifier(db.searchPath) + ` CASCADE;`
```

`CASCADE` means every object inside the schema (tables, views, sequences, functions, triggers) is dropped along with it, in one statement, regardless of foreign key or dependency ordering. This is appropriate for tearing down a throwaway database in a test suite (`pgtest` uses exactly this to clean up after itself) and is not something to reach for against a database holding data you care about; there is no confirmation step or dry-run mode.

DB.Migrate

Where `CreateSchema` re-derives the schema from your current Go structs every time, `Migrate` applies a fixed sequence of `.sql` files exactly once each, in ascending lexical order of their filenames:

```go
applied, err := db.Migrate(ctx, migrationsFS, &pg.MigrateOptions{})
if err != nil {
    log.Fatal(err)
}

for _, name := range applied {
    log.Printf("applied migration %s", name)
```

```
}
```

Lexical order is why migrations are conventionally named with a zero-padded numeric prefix, `0001_init.sql`, `0002_add_users.sql`, and so on: that convention keeps lexical order and intended order identical, since `"0002_..." > "0001_..."` as plain strings the same way `2 > 1` does as numbers.

`MigrateOptions` has two fields, both optional:

Field	Default	Purpose
`TableName`	`"schema_migrations"`	the tracking table, created on demand. Must be a bare identifier; validated before any SQL runs, since it is interpolated into DDL rather than bound as a parameter.
`Pattern`	`"*.sql"`	the `fs.Glob` pattern selecting migration filenames within `fsys`.

The whole call runs inside a single `db.InTransaction`: if any pending file fails, the transaction rolls back and every earlier file this same call already applied is undone along with it, so a failed `Migrate` call never leaves the tracking table or the schema half-updated. A file's contents run as-is via `Exec` (the same way `ExecFiles` runs a file, covered in Chapter 9's neighborhood in `db.go`), so a file containing several `;`-separated statements runs as one simple-protocol `Exec` covering all of them.

The Advisory Lock

Before touching any file, `Migrate` takes a PostgreSQL advisory lock:

```
db.Exec(ctx, "SELECT pg_advisory_xact_lock($1)", migrateLockKey)
```
GO

`migrateLockKey` is computed once at package init time from the FNV-1a hash of the fixed string `"kataras/pg/migrate"`, so it is the same value every time the package is compiled, across processes and restarts, with no configuration needed. This is what makes it safe for several instances of an application, for example replicas starting up concurrently, to call `Migrate` against the same database at the same moment: only one of them actually acquires the lock and runs the pending files, while the rest block on `pg_advisory_xact_lock` until the first one commits or rolls back. Once unblocked, each of the others reads the tracking table, sees the versions the first instance already recorded, and applies nothing. The lock is transaction-scoped

(`_xact_lock`, not a plain session-scoped advisory lock), so it releases automatically the instant the transaction ends, commit or rollback, and a crashed or killed process can never leave the lock stuck for the next deployment to hang on.

The schema_migrations Ledger

Inside the same transaction, after the lock, `Migrate` ensures the tracking table exists:

```sql
CREATE TABLE IF NOT EXISTS schema_migrations (
    version text PRIMARY KEY,
    applied_at timestamptz NOT NULL DEFAULT now()
);
```

then reads every `version` already recorded there, and for each pending file not already present, executes its contents (if any) and records its bare filename, exactly as matched by `Pattern` (e.g. `"0001_init.sql"`, never a full path), as a new row. A file with empty contents after reading is not executed, since there is nothing to run, but is still recorded as applied: it counts toward "already applied" on the next call and is never retried, so an intentionally empty placeholder file behaves exactly like any other applied migration.

Embedding Migrations

`Migrate` accepts any `fs.FS`, and the natural choice for a compiled binary is `embed.FS`, so migrations ship inside the binary instead of as loose files a deployment has to remember to copy alongside it:

```go
// migrations/0001_init.sql
// migrations/0002_add_orders.sql

//go:embed migrations/*.sql
var migrationsFS embed.FS

func main() {
    ctx := context.Background()

    db, err := pg.Open(ctx, schema, connString)
    if err != nil {
        log.Fatal(err)
    }

    fsys, err := fs.Sub(migrationsFS, "migrations")
    if err != nil {
        log.Fatal(err)
    }
```

```
    if _, err := db.Migrate(ctx, fsys, nil); err != nil {
        log.Fatal(err)
    }
}
```

`fs.Sub` rooted at the embedded directory matters here: without it, `fsys.Glob("*.sql")` would need to match `"migrations/0001_init.sql"` against the pattern `"*.sql"`, which it does not (`*` does not cross a path separator), so every file would look unmatched. Passing `fs.Sub` strips the leading `migrations/` so `Pattern`'s default matches the filenames directly. `nil` for `opts` in the example above applies both documented defaults, `"schema_migrations"` and `"*.sql"`.

What Migrate Deliberately Does Not Do

`Migrate` is, in its own doc comment's words, meant to be the smallest honest migration runner, not a full migration framework. Three things are left out on purpose, not by oversight:

- **No down/rollback direction.** There is no `.down.sql` counterpart and no API to reverse an applied migration. Reverting a mistake means writing and applying a new forward migration that undoes it, the same as any other schema change.
- **No checksum recorded or verified for an already-applied file.** Editing the contents of a file after it has already run has no effect on any future `Migrate` call: the ledger only remembers the filename, not a hash of its contents, so a silently edited already-applied file is not detected as drift.
- **No detection of, or special handling for, out-of-order files.** A file that lands lexically before one already applied (for example, adding `0001_forgotten.sql` after `0002_...` already ran) just applies wherever its filename sorts on the next call, with no warning that it arrived "late."

When to Reach for a Dedicated Migration Tool

`Migrate` is a genuinely reasonable choice for a small-to-medium application with one deployment pipeline and a team that reviews every migration file in the same pull request as the code that needs it: you already depend on pg, there is no second tool to install or CI step to configure, and the advisory-lock behavior alone solves the concurrent-replica problem correctly. It stops being enough the moment you need any of the three exclusions above as a real feature: a team large enough that migrations sometimes land out of order across branches and needs a tool that detects and refuses that, a compliance requirement to prove a migration was not edited after review (checksums), or an operational need to roll a specific migration back

without hand-writing its inverse. At that point, reach for golang-migrate/migrate or pressly/goose, both of which implement exactly those three features pg's `Migrate` leaves out, on top of the same idea of an ordered sequence of `.sql` files.

Summary

- `CreateSchema` (via `CreateSchemaDumpSQL`) emits `CREATE SCHEMA`, needed extensions (`pgcrypto`, `citext`, `hstore`, each only when a registered column type actually needs it), tables, foreign keys, and the `set_timestamp` trigger family, all in one transaction.
- The `set_timestamp` trigger convention fires on any table with a time-typed `updated_at` column (both names configurable on `Schema`), sharing one `trigger_set_timestamp()` function across every table.
- `CheckSchema` compares a live database's columns, rendered as `pg` struct tags, against the registered Go structs. It catches a missing/extra table, a database column absent from code, and a mismatched column definition; it does not catch a code column never migrated into the database, or anything not folded into the tag string (triggers, RLS, privileges).
- `DeleteSchema` issues `DROP SCHEMA ... CASCADE`, with no confirmation step.
- `Migrate` applies `.sql` files from an `fs.FS` in lexical filename order, all inside one transaction, guarded by a fixed-key `pg_advisory_xact_lock` that lets concurrent replicas start up against the same database safely, recording each applied filename in a `schema_migrations` ledger.
- `Migrate` deliberately excludes down migrations, checksums and out-of-order detection; reach for golang-migrate or goose once a team's process actually needs one of those.

Further Reading

- PostgreSQL: CREATE EXTENSION: the statement behind `pgcrypto` / `citext` / `hstore` provisioning.
- PostgreSQL: pgcrypto: `gen_random_uuid()` and `crypt()`, the two functions this extension provides that pg relies on.
- PostgreSQL: Trigger Behavior: `BEFORE UPDATE ... FOR EACH ROW`, the exact trigger shape `set_timestamp` installs.
- PostgreSQL: Advisory Locks: `pg_advisory_xact_lock` and how a transaction-scoped advisory lock differs from a session-scoped one.

- PostgreSQL: information_schema and DDL transactionality: background for why `CREATE TABLE` / `ALTER TABLE` can safely run inside the same transaction as any other statement.
- golang-migrate/migrate: a full-featured migration tool with down migrations and multiple database backends.
- pressly/goose: a migration tool supporting both `.sql` and Go-function migrations, with out-of-order detection.

CHAPTER 12

LISTEN and NOTIFY

PostgreSQL has a built-in publish/subscribe channel, separate from tables and rows: a connection issues `LISTEN channel`, and from then on the server pushes it an asynchronous message every time any connection runs `NOTIFY channel, 'payload'` (or the equivalent `pg_notify` function), for as long as that connection stays open. pg wraps this at two levels. The low-level API, `DB.Listen`, `DB.Notify`, `DB.Unlisten` and the `Listener` type, is a thin, direct mapping onto those SQL statements. The higher-level table-change API, `PrepareListenTable` and `DB.ListenTable` (plus `Repository[T].ListenTable`), installs a trigger and a shared notify function so INSERT/UPDATE/DELETE on a table arrives as a typed `TableNotification` instead of a hand-built payload string. This chapter covers both layers and is honest about what LISTEN/NOTIFY is not: a durable queue. A listener that is not connected when a notification fires never sees it; there is no replay, no persistence, and no delivery guarantee beyond "delivered to whoever was listening at the time." Every signature below is read from `db.go`, `listener.go` and `db_table_listener.go`.

Background

If you already know PostgreSQL's `LISTEN` / `NOTIFY` and why it is not a message queue, skip to DB.Listen and Notification Delivery. `LISTEN channel_name` tells the current connection's backend process to start delivering any notification sent to `channel_name`. `NOTIFY channel_name, 'payload'` (or `SELECT pg_notify('channel_name', 'payload')`, the same operation as a callable function instead of a statement) sends one such notification, containing an optional text payload, to every connection currently listening on that channel, at the moment `NOTIFY` runs. `UNLISTEN` stops delivery. The mechanism is intentionally simple and has no memory: PostgreSQL does not store notifications anywhere, does not know or care whether anyone received one, and drops it immediately for any connection that was not listening at that exact moment, including one that will reconnect and issue `LISTEN` again a second later. This is the opposite trade-off from a message broker like RabbitMQ or a durable log like Kafka, which

persist messages so a consumer that was offline can catch up. LISTEN/NOTIFY is closer to a radio broadcast: useful for waking up application code that is already listening ("a row changed, go check it"), unsuitable for anything that must never lose a message.

DB.Listen and Notification Delivery

```go
func (db *DB) Listen(ctx context.Context, channel string) (*Listener, error)
```

`Listen` acquires a dedicated connection from the pool, issues `LISTEN <channel>` on it, and returns a `*Listener` bound to that connection:

```go
conn, err := db.Listen(context.Background(), "chat_db")
if err != nil {
    log.Fatal(err)
}
defer conn.Close(context.Background())

for {
    notification, err := conn.Accept(context.Background())
    if err != nil {
        log.Println(err)
        return
    }

    fmt.Printf("channel: %s, payload: %s\n",
        notification.Channel, notification.Payload)
}
```

The acquired connection is not returned to the pool until the `Listener` is closed; see Operational Caveats for what that costs.

DB.Notify

```go
func (db *DB) Notify(ctx context.Context, channel string, payload any) error
```

`Notify` sends a notification via `pg_notify`. What it does with `payload` depends on its Go type:

- `string` or `[]byte`: sent as-is, unmodified, as the raw payload.
- anything else: marshaled with `encoding/json` first, then sent as the JSON text.

```go
err := db.Notify(ctx, "chat_db", "hello") // sent as-is.
```

```
type Message struct {
    Sender string `json:"sender"`
    Body   string `json:"body"`
}

err = db.Notify(ctx, "chat_json", Message{Sender: "kataras", Body: "hi"})
// sent as: {"sender":"kataras","body":"hi"}
```

`Notify` always runs on `db.Pool` directly (not through `db.tx`, even if `db` is transactional), since a notification sent from inside a transaction that later rolls back would otherwise need PostgreSQL's own deferred-delivery behavior (real `NOTIFY` only delivers at commit if issued inside a transaction) to avoid notifying about work that never happened; sending outside the transaction sidesteps that question entirely by notifying immediately, regardless of whether any surrounding transaction eventually commits.

DB.Unlisten

```
func (db *DB) Unlisten(ctx context.Context, channel string) error
```
GO

`Unlisten` issues `UNLISTEN <channel>` on whichever connection the pool happens to hand out for the call, which is not necessarily the same connection a prior `Listen` call is subscribed on: pooled connections are not addressable individually through `DB.Exec`. To stop a specific `Listener`, call `Listener.Close` instead, which issues `UNLISTEN` on that exact connection before releasing it. The special channel name `"*"` unsubscribes the connection from every channel it is currently listening on; `Unlisten` recognizes it and emits it unquoted (as the keyword-like wildcard it is), rather than quoting it as a literal channel named `*`.

The Listener Type

```
type Listener struct { /* unexported */ }

func (l *Listener) Accept(ctx context.Context) (*Notification, error)
func (l *Listener) Close(ctx context.Context) error
```
GO

`Accept` blocks until a notification arrives on the subscribed channel (or `ctx` is done), and returns it as a `*Notification` (`pgconn.Notification` under a package-level alias, carrying `Channel`, `Payload` and `PID`, the sending backend's process ID). If the payload is empty,

`Accept` returns `ErrEmptyPayload` instead of an empty `*Notification`, since an empty payload is rarely meaningful and callers otherwise have to remember to check `len(payload) == 0` themselves on every notification.

`Close` unsubscribes and releases the underlying pooled connection. It is safe to call more than once, and safe to call concurrently: only the first call does any work (guarded by an atomic compare-and-swap), every later call is a no-op returning `nil`. If the `UNLISTEN` statement itself fails, `Close` closes the raw connection outright (rather than releasing it back to the pool to be recycled), so pgxpool destroys it instead of handing a connection that is still subscribed to some channel to the next caller who acquires it for something unrelated.

Notification, ErrEmptyPayload and UnmarshalNotification

```go
type Notification = pgconn.Notification // Channel, Payload, PID.

var ErrEmptyPayload = errors.New("empty payload")

func UnmarshalNotification[T any](n *Notification) (T, error)
```

`UnmarshalNotification` is the counterpart to `Notify`'s JSON-encoding branch: it JSON-decodes `n.Payload` into `T` and returns it.

```go
type Message struct {
    Sender string `json:"sender"`
    Body   string `json:"body"`
}

notification, err := conn.Accept(ctx)
if err != nil {
    log.Fatal(err)
}

payload, err := pg.UnmarshalNotification[Message](notification)
if err != nil {
    log.Fatal(err)
}

fmt.Println(payload.Sender, payload.Body)
```

Nothing enforces that a payload was actually produced by `Notify`'s JSON branch. `UnmarshalNotification` against a plain string payload (sent with `Notify(ctx, channel, "hello")`, or by any other client issuing raw `NOTIFY`) fails with a JSON decode error unless `T` is itself `string`, since `"hello"` alone is not valid JSON for a struct.

Quoted Channel Identifiers

Both `Listen` and `Notify` / `Unlisten` quote the channel name before embedding it in SQL, via `QuoteIdentifier`. This is not only an injection guard; it also keeps `LISTEN` and `pg_notify` consistent with each other for a mixed-case channel name. An unquoted identifier in a `LISTEN` statement is folded to lowercase by PostgreSQL's normal identifier rules, but `pg_notify`'s channel argument is an ordinary string parameter, never folded at all. Without quoting on the `LISTEN` side, `db.Listen(ctx, "ChatDB")` would subscribe to `chatdb` (lowercased), while `db.Notify(ctx, "ChatDB", ...)` sends to the literal channel `ChatDB`, and the two would simply never meet. Quoting `LISTEN "ChatDB"` preserves the case exactly, so a caller who only ever uses this package's own `Listen` / `Notify` pair (rather than issuing raw `LISTEN` / `NOTIFY` SQL by hand) gets consistent behavior regardless of the casing chosen for a channel name.

The Table-Change Layer

Hand-writing a trigger, a notify function and a payload format for every table you want to watch is repetitive. `PrepareListenTable` and `DB.ListenTable` do it once, generically, for any set of registered tables:

```go
func (db *DB) PrepareListenTable(
    ctx context.Context, opts *ListenTableOptions,
) error

func (db *DB) ListenTable(ctx context.Context, opts *ListenTableOptions,
    callback func(TableNotificationJSON, error) error) (Closer, error)
```

`ListenTable` calls `PrepareListenTable` internally (creating whatever triggers and functions are missing), then calls `Listen` on the resolved channel, then starts a goroutine that loops on `Accept`, JSON-decodes each notification into a `TableNotificationJSON`, and calls `callback` with it:

```go
opts := &pg.ListenTableOptions{
    Tables: map[string][]pg.TableChangeType{
        "customers": {pg.TableChangeTypeInsert, pg.TableChangeTypeUpdate},
    },
}

closer, err := db.ListenTable(ctx, opts,
    func(evt pg.TableNotificationJSON, err error) error {
        if err != nil {
            log.Println("listen table error:", err)
```

```
            return err // non-nil stops the listener.
        }

        log.Printf("table: %s, change: %s\n", evt.Table, evt.Change)
        return nil // nil keeps listening.
    })
if err != nil {
    log.Fatal(err)
}
defer closer.Close(ctx)
```

`callback` returning any non-nil error stops the listener (the goroutine returns and the underlying `Listener` is closed via its own `defer`); returning `nil` keeps it running. `PrepareListenTable` can also be called on its own, ahead of time (for example during deployment, alongside `CreateSchema`), to install the triggers without immediately starting to listen.

ListenTableOptions

```go
type ListenTableOptions struct {
    Tables   map[string][]TableChangeType
    Channel  string
    Function string
}
```

- `Tables`: which tables to watch, and which of `INSERT`/`UPDATE`/`DELETE` to notify on for each. The special key `"*"` means every base table registered in the schema, watching the changes named in its value; it defaults to `{"*": [INSERT, UPDATE, DELETE]}` when `Tables` is left empty entirely.
- `Channel`: the PostgreSQL channel to `LISTEN`/notify on. Defaults to `"table_change_notifications"`.
- `Function`: the base name for the shared PL/pgSQL notify function. Each table's trigger is named `<table>_<Function>`. Defaults to `"table_change_notify"`.

The Trigger and Function pg Installs

`prepareListenTable` (the unexported worker `PrepareListenTable` calls once per table) creates one shared function:

```sql
CREATE OR REPLACE FUNCTION table_change_notify() RETURNS trigger AS $$
    DECLARE
```

```
    payload text;
    channel text := 'table_change_notifications';

    BEGIN
    SELECT json_build_object('table', TG_TABLE_NAME, 'change', TG_OP,
                             'old', OLD, 'new', NEW)::text
    INTO payload;
    PERFORM pg_notify(channel, payload);
    IF (TG_OP = 'DELETE') THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$
LANGUAGE plpgsql;
```

and, per watched table, a trigger naming it:

```sql
CREATE OR REPLACE TRIGGER customers_table_change_notify
AFTER INSERT OR UPDATE
ON customers
FOR EACH ROW
EXECUTE FUNCTION table_change_notify();
```

Both the function name and the notify channel are baked into the generated SQL text as literals, not parameters, which is exactly why `Channel` and `Function` are restricted to safe identifiers (see The Identifier Restriction). Creation is tracked per `*DB` (shared across every `*DB` cloned from the same root, including transaction-scoped clones, via a mutex-guarded `tableNotifyState`) so calling `ListenTable`, or `PrepareListenTable`, more than once for the same function or table does not re-issue the `CREATE OR REPLACE` statements needlessly, even from concurrent callers.

TableNotification and Change Kinds

```go
type TableNotification[T any] struct {
    Table  string
    Change TableChangeType
    New    T
    Old    T
}

type TableNotificationJSON = TableNotification[json.RawMessage]
```

`Change` is one of three `TableChangeType` string constants: `TableChangeTypeInsert` (`"INSERT"`), `TableChangeTypeUpdate` (`"UPDATE"`) and `TableChangeTypeDelete` (`"DELETE"`), matching PostgreSQL's own `TG_OP` trigger variable verbatim. `New` is populated for `INSERT` and `UPDATE`

and is the zero value of `T` for `DELETE`; `Old` is populated for `UPDATE` and `DELETE` and is the zero value of `T` for `INSERT`. `TableNotificationJSON` is the generic instantiation `DB.ListenTable`'s callback receives directly: `New` / `Old` stay as raw `json.RawMessage` since the low-level API has no registered Go type to decode them into, leaving that decision to the caller (or, for a typed table, to `Repository[T].ListenTable` below). `(TableNotification[T]).GetPayload() string` returns the notification's raw, undecoded text, useful for debugging a decode failure.

Repository ListenTable

```go
func (repo *Repository[T]) ListenTable(ctx context.Context,
    callback func(TableNotification[T], error) error) (Closer, error)
```

`Repository[T].ListenTable` is `DB.ListenTable` fixed to exactly one table, the repository's own, watching all three change types, and decoding `New` / `Old` into `T` instead of leaving them as raw JSON:

```go
customers := pg.NewRepository[Customer](db)

closer, err := customers.ListenTable(ctx,
    func(evt pg.TableNotification[Customer], err error) error {
        if err != nil {
            return err
        }

        fmt.Printf("change: %s, old name: %s, new name: %s\n",
            evt.Change, evt.Old.Name, evt.New.Name)
        return nil
    })
```

Internally it calls `db.ListenTable` with `Tables: map[string][]TableChangeType{ repo.td.Name: {Insert, Update, Delete}}`, and its callback unmarshals the raw `Old` / `New` JSON fields into `T` itself before calling the caller's typed callback, so `evt.New` and `evt.Old` above arrive already decoded as `Customer` values, not `json.RawMessage`.

The Identifier Restriction

`ListenTableOptions.Channel`, `Function`, and every non-wildcard key in `Tables`, must match `^[A-Za-z_][A-Za-z0-9_$]*$`: a bare identifier, starting with a letter or underscore, no quotes, dots or spaces. `PrepareListenTable` validates every one of them before touching the database

at all. The restriction exists because these three values are not passed as bind parameters anywhere in this layer: `Channel` is embedded both as a PL/pgSQL single-quoted string literal (`channel text := '<Channel>';`) inside the generated function body and as a raw `LISTEN` target; `Function` and each table name are embedded as raw SQL identifiers in `CREATE FUNCTION` / `CREATE TRIGGER` statements. A `Channel` value containing an unescaped single quote, or a `Function` value containing a semicolon, would otherwise let a caller-controlled string alter the generated DDL. This is stricter than the low-level `DB.Listen` / `Notify`, which quote (rather than reject) an arbitrary channel name via `QuoteIdentifier`; the table-change layer rejects instead of quoting because these values are interpolated into more than one syntactic position (a string literal and an identifier), and no single quoting strategy is safe for both at once.

Operational Caveats

Three things are worth knowing before relying on LISTEN/NOTIFY in production, none of them specific to pg, all of them inherited directly from PostgreSQL's own design:

- **Payload size.** PostgreSQL caps a `NOTIFY` payload at approximately 8000 bytes; a larger payload is rejected by the server with an error. `Notify`'s JSON-encoding branch makes it easy to exceed this without noticing, since a struct with a few nested slices grows quickly. Keep the payload to an identifier (a row's primary key) and have the receiver query for the rest, rather than trying to carry a whole row image through the channel, especially for wide tables.
- **The connection is held for the listener's lifetime.** `Listen` acquires one pooled connection and does not release it until `Close` is called; that connection is unavailable to every other caller of `db.Pool` for as long as the `Listener` (or the goroutine behind `ListenTable`) is alive. A pool sized for request-serving traffic, with several long-lived listeners layered on top, can run out of connections faster than expected. Size the pool with the listener count in mind, or use a separate, dedicated pool for listening connections.
- **A disconnected listener misses messages, and pg does not reconnect for you.** If the connection underlying a `Listener` is dropped (a network blip, a server restart, the connection being killed), any notification sent while it was down is gone; there is nothing to replay it from. `ListenTable`'s internal loop recognizes `io.ErrUnexpectedEOF` and `net.ErrClosed` as an intentional close and simply ends the goroutine, but any other error is handed to `callback`, and it is `callback`'s decision whether to stop (returning non-nil) or keep looping (returning `nil`). Since `Accept` on an already-broken connection tends to fail immediately rather than block, a callback that always returns `nil` on error risks a tight loop of instant failures instead of a clean stop. There is no automatic re-subscription: a caller that

wants resilience against dropped connections needs to detect the callback's error, return non-nil to stop the listener, and call `Listen` / `ListenTable` again to obtain a fresh connection and resume, accepting that whatever happened on the channel during the gap is unrecoverable.

Summary

- LISTEN/NOTIFY is PostgreSQL's built-in pub/sub channel, not a durable queue: nothing is persisted, and a listener that is not connected at the moment of a `NOTIFY` never receives it.
- `DB.Listen` acquires a dedicated pooled connection and returns a `*Listener`; `Accept` blocks for the next notification, `ErrEmptyPayload` flags an empty one, and `Close` unsubscribes and releases the connection, safely callable more than once.
- `DB.Notify` sends a `string` / `[]byte` payload as-is or JSON-encodes anything else; `UnmarshalNotification[T]` decodes a JSON payload back into `T`. Channel names are quoted with `QuoteIdentifier` so `LISTEN` and `pg_notify` agree on a mixed-case channel.
- `PrepareListenTable` / `DB.ListenTable` install one shared notify function and a per-table trigger, delivering typed `TableNotification` values over a background goroutine; `Repository[T].ListenTable` narrows that to one table and decodes `Old` / `New` into the repository's own struct type.
- `Channel`, `Function` and table names in `ListenTableOptions` are restricted to bare SQL identifiers, rejected rather than quoted, because they are interpolated into both a PL/pgSQL string literal and raw SQL identifiers.
- Watch payload size (about 8000 bytes), the pooled connection each listener holds for its entire lifetime, and the lack of automatic reconnection: a dropped listener must be re-established by the caller, and whatever happened on the channel during the gap is gone.

Further Reading

- PostgreSQL: NOTIFY: the statement, its payload size limit, and delivery semantics (including delivery timing relative to a wrapping transaction).
- PostgreSQL: LISTEN: channel identifier folding, the behavior this chapter's quoting section works around.
- PostgreSQL: Asynchronous Notification: the underlying libpq-level mechanism pgx's `WaitForNotification` wraps.

- PostgreSQL: CREATE TRIGGER: `AFTER`, `FOR EACH ROW`, and the `TG_OP` / `TG_TABLE_NAME` variables the generated notify function reads.
- pgconn.Notification: the driver type `Notification` aliases, with `PID`, `Channel` and `Payload`.

CHAPTER 13

Introspection and Code Generation

Every chapter so far started from a Go struct and asked pg to produce SQL. This chapter runs the library in the other direction: reading a live PostgreSQL database and asking pg to describe it, then, from that description, to write Go source. `db_information.go` exposes the introspection API that DB.CreateSchema and DB.CheckSchema already use internally to talk to `information_schema` and the `pg_catalog` system tables. The `gen` package builds on that same API to reverse-engineer entity structs from an existing database (`GenerateSchemaFromDatabase`) or to emit typed column-name constants from a schema you already registered in code (`GenerateColumnsFromSchema`). Every function and type named below is read directly from `db_information.go` and the `gen` package's source.

Background

If you have used `pg_dump --schema-only` or a framework's "introspect the database" command before, skip to Listing Tables.

Two catalogs, one merged result. PostgreSQL keeps a full description of every object it manages (tables, columns, constraints, indexes, triggers) in two places: the SQL-standard `information_schema` views, which are portable across databases but coarse, and the `pg_catalog` system tables (`pg_class`, `pg_constraint`, `pg_index`, and so on), which are PostgreSQL-specific but expose detail `information_schema` does not, such as an index's access method or a constraint's exact definition text. pg's introspection layer queries both: `information_schema` for the basic column shape, `pg_catalog` for constraints, unique indexes and triggers. The result is assembled into the same `*desc.Table` and `*desc.Column` types that `Schema.MustRegister` builds from a Go struct, so code that reads a live database and code that reads your struct tags end up looking at the same shape.

Listing Tables

`DB.ListTables` is the entry point. It queries the columns for every table in the connection's search path (or a subset you name), groups them back into tables, and returns `[]*desc.Table`:

```go
tables, err := db.ListTables(ctx, pg.ListTablesOptions{})
if err != nil {
    log.Fatal(err)
}

for _, t := range tables {
    fmt.Println(t.Name, t.StructName, len(t.Columns))
}
```

`ListTablesOptions` has two fields:

Field	Purpose
`TableNames []string`	Restrict the result to these table names; empty lists every table in the search path.
`Filter desc.TableFilter`	Customize or reject tables and columns after they are read (see below).

The returned tables are sorted so that "parent" tables sort before tables whose name contains an underscore (typically junction or child tables), and read-only tables (views) always sort last. This ordering is what `gen.GenerateSchemaFromDatabase` relies on when it writes files in dependency-friendly order, though it does not otherwise affect correctness: foreign keys are still applied in a second pass regardless of table order.

Filtering Tables

A raw database scan usually needs adjustment: a `jsonb` column should decode into a specific Go struct, or a legacy table should be skipped from generation entirely. `ListTablesOptions.Filter` takes anything implementing the `desc.TableFilter` interface:

```go
type TableFilter interface {
    FilterTable(*Table) bool
}
```

Returning `false` drops the table from the result. `desc.TableFilterFunc` adapts a plain function to that interface, useful for one-off filtering logic:

```
        }
        return true
    }),
}
```

For overriding column types rather than excluding tables, `pg.MapTypeFilter` is the ready-made implementation you will reach for most often. It maps a dotted expression (`"table.column"`, or `"table.column.jsonb"` to force the JSONB decode path) to a Go value whose type becomes the column's `FieldType`:

```go
opts := pg.ListTablesOptions{
    Filter: pg.MapTypeFilter{
        "customer_profiles.fields.jsonb": entity.Fields{},
    },
}
```

`MapTypeFilter.FilterTable` never rejects a table; it only rewrites column types. Internally it panics on a malformed expression string, since the map is developer-authored configuration, not user input (the same reasoning `Conditions` applies to a fragment's SQL text in Chapter 6). `ListTables` catches that panic for you: it calls the filter through an unexported `safeFilterTable` helper that recovers and turns the panic back into a returned `error`, so a typo in a filter expression fails the call instead of crashing the process at connect time.

Listing Columns

`DB.ListColumns` is what `ListTables` calls internally, and it is also useful directly when you only need column data:

```go
columns, err := db.ListColumns(ctx, "customers", "blogs")
```

It composes three lower-level queries and merges their results into `[]*desc.Column`:

- `DB.ListColumnsInformationSchema` reads `information_schema.columns` joined with `pg_catalog.pg_attribute`, returning `[]*desc.ColumnBasicInfo`: table name, table description, table type, column name, ordinal position, description, default expression, parsed data type, nullability, identity and generated status.
- `DB.ListConstraints` reads `pg_constraint` and `pg_indexes`, returning `[]*desc.Constraint`, then folds primary key, unique, check and foreign key information onto the matching column.
- `DB.ListUniqueIndexes` reads `pg_index` for unique, non-primary, non-constraint-backed indexes, returning `[]*desc.UniqueIndex` (table name, index name, and the ordered column

list), so a composite unique index shows up correctly on every column it covers instead of only the first.

The merge order matters for one detail worth knowing if you ever compare `ListColumns` output against `CheckSchema`'s expectations: a column that is both a primary key or a plain `Unique` column and also carries a same-column index gets its `Index` field cleared, because PostgreSQL already manages that index implicitly; `UniqueIndex` is left alone even in that case, since a composite unique index spanning other columns is still meaningful information.

Constraints and Unique Indexes

`DB.ListConstraints` and `DB.ListUniqueIndexes` are also useful on their own, for example to audit a schema outside of the struct-driven workflow:

```go
constraints, err := db.ListConstraints(ctx, "orders")
for _, c := range constraints {
    fmt.Println(c.ConstraintName, c.ConstraintType, c.ColumnName)
}
```

`desc.Constraint` reports `TableName`, `ColumnName` (filled in from the constraint's definition text for the common single-column case), `ConstraintName`, `ConstraintType`, and `IndexType` (the access method, e.g. `btree` or `gin`, when the constraint is backed by an index). `desc.UniqueIndex` is simpler: `TableName`, `IndexName`, and the ordered `Columns` slice. Both are populated from the same query `ListColumns` uses, so there is no separate code path to keep in sync.

Triggers

`DB.ListTriggers` reads `information_schema.triggers`, scoped to the database catalog and the tables registered on `db`'s schema, and returns `[]*desc.Trigger`: `Catalog`, `SearchPath`, `Name`, `Manipulation` (`INSERT`, `UPDATE`, `DELETE`), `TableName`, `ActionStatement`, `ActionOrientation` (`ROW` or `STATEMENT`), and `ActionTiming` (`BEFORE` or `AFTER`). `DB.CreateSchema` calls this before writing the `updated_at` trigger function so that reapplying `CreateSchema` against an already-provisioned database does not try to create the same trigger twice.

Table and Database Size

`DB.ListTableSizes` and `DB.GetSize` both return a `SizeInfo` (`SizePretty`, `Size` in bytes, `SizeTotalPretty`, `SizeTotal` in bytes, where "total" adds the size of every index on the table). `ListTableSizes` returns one `TableSizeInfo` (which embeds `SizeInfo` plus `TableName`) per table in the search path, largest first, regardless of whether the table is registered in your `*pg.Schema`. `GetSize` sums the same figures across every table in the search path into a single `SizeInfo`:

```go
sizes, err := db.ListTableSizes(ctx)
for _, s := range sizes {
    fmt.Printf("%-20s %10s (total %s)\n",
        s.TableName, s.SizePretty, s.SizeTotalPretty)
}

total, err := db.GetSize(ctx)
fmt.Println("database:", total.SizeTotalPretty)
```

These are operational, not schema, tools: they are worth wiring into a periodic job that watches for a table growing unexpectedly, which is usually the first sign of a missing index or a runaway retention policy. Chapter 15 revisits them alongside `PoolStat` and `DB.Health`.

Server Version

`DB.GetVersion` runs `SELECT version();` and parses out just the version number, e.g. `"16.1"` from `"PostgreSQL 16.1 on x86_64-pc-linux-gnu, compiled by gcc..."`. It also trims a trailing comma that some builds append before the compiler information (for example `"PostgreSQL 16.1,"`), so the returned string is always a bare version number, ready to compare or log without further cleanup.

Autovacuum Controls

Autovacuum reclaims dead tuples left behind by updates and deletes; if it falls behind, table bloat and stale query-planner statistics follow. Three methods let you inspect and, in narrow cases, disable it:

```go
enabled, err := db.IsAutoVacuumEnabled(ctx) // SHOW autovacuum;

err = db.DisableAutoVacuum(ctx)             // whole database
err = db.DisableTableAutoVacuum(ctx, "events")
```

`DisableAutoVacuum` runs `ALTER DATABASE <db> SET autovacuum = off;` and `DisableTableAutoVacuum` runs `ALTER TABLE <table> SET (autovacuum_enabled = false);`; both quote their identifier arguments with `QuoteIdentifier` before interpolating them (see Chapter 14 for why that quoting, rather than a bind parameter, is the correct tool here). Disabling autovacuum on a live table is rarely the right long-term answer: it is most often reached for temporarily around a large bulk load, then turned back on immediately afterward with a manual `VACUUM ANALYZE`.

Reverse-Engineering Structs From a Database

`gen.GenerateSchemaFromDatabase` is the code-generation counterpart to everything above: it opens a connection, calls `DB.ListTables` under the hood, and writes one Go source file per table plus a `schema.go` that registers all of them under a `*pg.Schema`:

```go
package main

import (
    "context"
    "log"

    "github.com/kataras/pg"
    "github.com/kataras/pg/gen"
)

func main() {
    i := gen.ImportOptions{
        ConnString: "postgres://postgres:pass@localhost:5432/app" +
            "?sslmode=disable",
        ListTables: pg.ListTablesOptions{
            Filter: pg.MapTypeFilter{
                "customers.metadata.jsonb": map[string]any{},
            },
        },
    }

    e := gen.ExportOptions{
        RootDir: "./internal/entities",
    }

    err := gen.GenerateSchemaFromDatabase(context.Background(), i, e)
    if err != nil {
        log.Fatal(err)
    }
}
```

`ImportOptions` is small on purpose: `ConnString` (accepted in any format `pg.Open` accepts) and `ListTables`, passed straight through to `DB.ListTables`, so every filtering technique from earlier in this chapter applies here too. If a column's PostgreSQL type has no obvious Go equivalent (a

custom enum, a domain type), `GenerateSchemaFromDatabase` prints a diagnostic listing every `table.column.type` it could not resolve, together with a ready-to-paste `MapTypeFilter` snippet naming the first one, before it fails to compile anything meaningful for that column.

`ExportOptions` controls where and how the generated files land; see the next section for its file-naming strategies. After writing every file, `GenerateSchemaFromDatabase` looks for `goimports` on `PATH` (the executable name is the package variable `gen.GoImportsTool`, default `"goimports"`) and runs it over `RootDir` if found, cleaning up import grouping that `go/format.Source` alone does not handle. `GoImportsTool` is a trusted, developer-set knob: it names a local executable to run, never a value derived from the database or from network input.

File Layout Strategies

By default, `ExportOptions.GetFileName` places every table's struct in its own file named after the table, all in one flat package under `RootDir`. Two ready-made strategies change that:

`EachTableToItsOwnPackage` gives every table its own subpackage, named after its singular form: a `customers` table becomes `RootDir/customer/customer.go`, package `customer`. This is the strategy to reach for when tables are otherwise unrelated and you want each entity importable on its own. It is itself a `GetFileName`-shaped function (`func(rootDir, tableName string) string`), so assign it directly, with no call: `GetFileName: gen.EachTableToItsOwnPackage`.

`EachTableGroupToItsOwnPackage` groups related tables together: the first table seen with a given prefix establishes a package, and any later table whose singular name starts with `<group>_` joins that same package instead of getting one of its own. A `customer` table followed by a `customer_address` table both land in package `customer`, as `customer.go` and `customer_address.go`. This is the strategy the library's own example test (`gen.ExampleGenerateSchemaFromDatabase`) uses, and it is the better default when your schema has genuine parent/child table families such as `orders` and `order_items`.

```go
e := gen.ExportOptions{
    RootDir:    "./internal/entities",
    GetFileName: gen.EachTableGroupToItsOwnPackage(),
}
```

Note that `EachTableGroupToItsOwnPackage` returns a stateful closure (it tracks which groups it has already seen), so call it once per `GenerateSchemaFromDatabase` invocation rather than sharing one instance across calls with different table sets. This makes it a factory, not a `GetFileName` value itself: unlike `EachTableToItsOwnPackage` above, which you assign directly, you must call `EachTableGroupToItsOwnPackage()` to obtain the closure. Leaving off the

parentheses assigns the factory function itself, whose type is `func() func(rootDir, tableName string) string`, not the `GetFileName`-shaped value `ExportOptions.GetFileName` expects, so it fails to compile.

`ExportOptions.ToSingular` and `ExportOptions.GetPackageName` are the lower-level hooks both strategies are built from, in case neither grouping rule fits: write your own `func(rootDir, tableName string) string` and assign it to `GetFileName` directly.

Formatting Generated Code

Every generated file is round-tripped through `go/format.Source` before it is written, so the output is always valid, gofmt-clean Go, independent of whether `goimports` happens to be installed. `ExportOptions.FileMode` controls the permission bits the files are written with; it defaults to `0o644` (owner read-write, group and other read-only) specifically so that generated source is never left world-writable by accident. Override it explicitly if your build pipeline needs something stricter.

Typed Column-Name Constants

`gen.GenerateColumnsFromSchema` solves a different problem: you already have a `*pg.Schema` built from hand-written structs (as in every earlier chapter), and you want compile-time-checked names for its columns instead of typing `"created_at"` as a bare string wherever a query needs one.

```go
schema := pg.NewSchema()
schema.MustRegister("companies", Company{})
schema.MustRegister("customers", Customer{})

opts := gen.ExportOptions{
    RootDir: "./definition",
}

if err := gen.GenerateColumnsFromSchema(schema, opts); err != nil {
    log.Fatal(err)
}
```

The generated package exposes one `Column`-valued field per table column, plus a `PG_TableName` string constant, and every `Column` implements `fmt.Stringer` by returning its underlying name:

```
definition.Customer.Email.String()    // "email"
definition.Customer.PG_TableName      // "customers"
```

This is where the payoff from Chapter 6 shows up. `Conditions`, `Repository.OrderBy` and `desc.Table.OrderBy` all take plain strings for column names, and both explicitly document that the string should come from a validated source, never straight from a client request. Generated `Column` constants are exactly that validated source: they can only ever name a column that genuinely exists on the table at generation time, so a typo becomes a Go compile error instead of a runtime "column does not exist":

```go
where := pg.Where(definition.Customer.Email.String()+" = $1", email)

orderBy, err := repo.OrderBy(definition.Customer.CreatedAt.String(), true)
if err != nil {
    log.Fatal(err)
}
```

Regenerate whenever the schema changes (a new migration, a renamed column) and the constants stay in lockstep with the database, the same way regenerating protobuf bindings keeps generated code in lockstep with a `.proto` file.

Safety Guards in the Generator

Both generators write files whose names are derived from table names, and table names for `GenerateSchemaFromDatabase` come from a live database rather than from your own source code. Before deriving a path from any table name, both generators call an internal `validateTableFileName` check that rejects an empty name, a name that is not `filepath.IsLocal` (so no absolute path and no `..` segment that could escape `RootDir`), and a name containing a path separator. A table literally named `../../etc/passwd` or `a/b` therefore fails generation with a descriptive error rather than being joined, unchecked, onto `RootDir` and written outside it. This mirrors the same identifier-validation discipline the rest of the library applies to table and column names read from struct tags (see Chapter 14): a name arriving from an external system (here, whatever the connected database happens to contain) is validated before it is trusted to shape a filesystem path, exactly as it is validated before being trusted to shape SQL.

Summary

- `DB.ListTables`, `DB.ListColumns`, `DB.ListColumnsInformationSchema`, `DB.ListConstraints` and `DB.ListUniqueIndexes` read `information_schema` and `pg_catalog` and assemble the same `*desc.Table` / `*desc.Column` shape the struct-tag path produces.
- `ListTablesOptions.Filter` accepts any `desc.TableFilter`; `pg.MapTypeFilter` overrides column Go types by dotted expression, and a filter panic is recovered into a returned error by `ListTables` itself.
- `DB.ListTriggers`, `DB.ListTableSizes`, `DB.GetSize`, `DB.GetVersion`, `DB.IsAutoVacuumEnabled`, `DB.DisableAutoVacuum` and `DB.DisableTableAutoVacuum` round out the operational introspection surface.
- `gen.GenerateSchemaFromDatabase` reverse-engineers Go structs and a registering `schema.go` from a live database, driven by `ImportOptions` (what to read) and `ExportOptions` (where and how to write it), with `EachTableToItsOwnPackage` and `EachTableGroupToItsOwnPackage` as ready-made layout strategies.
- `gen.GenerateColumnsFromSchema` emits typed `Column` constants from an already-registered `*pg.Schema`, meant to feed `Conditions`, `Repository.OrderBy` and `desc.Table.OrderBy` instead of hand-typed column-name strings.
- Both generators validate a table name against filesystem-escape characters (`validateTableFileName`) before deriving a path from it, and write files at a safe, non-world-writable default `FileMode`.

Further Reading

- PostgreSQL: The Information Schema: the portable `information_schema` views this chapter's basic column queries read from.
- PostgreSQL: System Catalogs: `pg_class`, `pg_constraint`, `pg_index` and the rest of the PostgreSQL-specific catalog this chapter's constraint and index queries read from.
- PostgreSQL: Routine Vacuuming: what autovacuum does and why disabling it is a narrow, temporary tool rather than a default.
- goimports: the tool `gen.GoImportsTool` names and runs over generated output when it is available on `PATH`.
- go/format: the standard-library formatter every generated file is round-tripped through before it is written to disk.

CHAPTER 14

Security

This chapter is not a checklist of things to enable. It is an account of the decisions already built into pg, so you know which threats they close, which ones remain your responsibility, and where the seams between the two run. It covers how SQL injection is prevented for values and, separately, for identifiers; the trust boundary between developer-controlled schema and request-controlled input, and exactly which knobs sit on the wrong side of it if you feed them user data; how passwords are hashed, on the server or in Go; what ends up in your logs at each log level; what each `sslmode` actually guarantees; and how `ConstraintError` lets a failure reach a client without leaking your schema. Every claim below is checked against the source in `errors.go`, `where.go`, `desc/insert_query.go`, `desc/on_conflict.go`, `desc/struct_table.go`, `db.go` and `db_information.go`.

Background

Every SQL injection defense in this library rests on one distinction: a **value** and an **identifier** are protected by entirely different mechanisms, because PostgreSQL's wire protocol only has a slot for one of them. A value (a string, a number, a UUID you are comparing a column against) can be sent as a bind parameter: the driver ships it to the server separately from the query text, tagged with its type, so there is no query text for it to corrupt no matter what bytes it contains. An identifier (a table name, a column name, a sort direction target) cannot: PostgreSQL only accepts `$1`, `$2`, and so on where a *value* is expected, never where a *name* is expected (see jackc/pgx#885). An identifier that needs to vary at runtime has no choice but to become part of the query text itself.

That single fact explains almost every API decision in this chapter. Wherever pg builds a query from data you control at compile time (a struct's field, a literal string in your own code), it does the concatenation for you and moves on. Wherever a name would have to come from something you do not fully control at compile time, either the library gives you a validating helper to call

first, or it explicitly documents that you are handling developer-authored SQL and must not let request data anywhere near it. Knowing which case you are in for any given API is the whole game.

Values Are Bind Parameters

Every `Insert`, `Update`, `Select` and `Delete` path in this library sends your struct's field values as `$1`, `$2`, … bind parameters, never as interpolated text. This is true of `Repository.InsertSingle`, of `Conditions`' fragment arguments, of `db.QueryRow(ctx, query, args...)`, and of every generated `WHERE`, `SET` and `VALUES` clause you have seen in earlier chapters. A customer's email address, a comment's body text, a page size someone typed into a search box: all of it is safe to pass as an argument, in whatever form it arrives in, because the driver keeps it out of the SQL text entirely. This is the part of injection defense you do not have to think about; it is also the part that is easy to accidentally bypass by hand-formatting a query string yourself instead of using placeholders, so the discipline is still worth stating rather than assuming.

Identifiers: Validate, Then Quote

Table names, column names and unique index names cannot travel as bind parameters, so pg defends them with a two-step discipline instead: validate the name against a strict allowlist pattern, then quote it before writing it into SQL.

Validation happens once, at `Schema.MustRegister` time, in `desc.ConvertStructToTable`. Every table name, every resolved column name (after tag or name-mapper resolution) and every unique index name is checked against:

```go
var identifierRegex = regexp.MustCompile(`^[A-Za-z_][A-Za-z0-9_$]*$`)
```

a bare, unquoted PostgreSQL identifier: a letter or underscore, then any number of letters, digits, underscores or dollar signs. Anything else, quotes, dots, whitespace, non-ASCII bytes, fails registration outright with a descriptive error naming the offending identifier. A struct with a column tagged to a name like `id; DROP TABLE users;--` never becomes a working `*pg.Schema` in the first place: registration fails before a single query is ever built from it.

Quoting happens every time a validated name is written into SQL, via `QuoteIdentifier`:

```go
func QuoteIdentifier(identifier string) string {
    return pgx.Identifier{identifier}.Sanitize()
}
```

`pgx.Identifier.Sanitize` double-quotes the identifier and doubles any embedded `"` character, which is the standard SQL escaping rule for a quoted identifier. `desc.Table.OrderBy`, the table-name CRUD in `db_crud.go`, `DB.DeleteSchema`, `DB.DisableAutoVacuum`, `DB.DisableTableAutoVacuum`, and `DB.SelectByUsernameAndPassword`'s column references all go through this same call (either directly or via the equivalent `pgx.Identifier{...}.Sanitize()`), so identifiers that reach SQL text are consistently escaped, not merely assumed safe because they passed validation once somewhere upstream.

Why Quoting Alone Is Not Enough

It is tempting to think quoting alone (skip the regex, just call `QuoteIdentifier` on whatever string shows up) would be sufficient: `Sanitize` doubles embedded quotes, so nothing can break out of the identifier position. That reasoning is incomplete, and the library's own source has one place that demonstrates exactly why: the schema search path.

`db.searchPath` is written, unquoted, into `CREATE SCHEMA IF NOT EXISTS <search path>;`. It is deliberately left unquoted so that PostgreSQL's normal identifier case-folding applies the same way it would for any other unquoted identifier a caller might have typed by hand elsewhere, matching how existing callers already expect the search path to behave. An unquoted identifier in PostgreSQL is folded to lowercase before it is resolved; a *quoted* identifier is taken literally, case and all. Wrapping the search path in `QuoteIdentifier` would make `MyApp` and `myapp` two different schemas instead of the same one, silently changing which schema every later query resolves against, for a caller who never asked for that distinction. Because quoting was not an option here, the library falls back to strict validation instead:

```go
// validateSearchPath reports an error if searchPath is not a safe,
// bare SQL identifier. It exists because db.searchPath is
// interpolated, unquoted, into "CREATE SCHEMA IF NOT EXISTS
// <search path>;". It is left unquoted so that Postgres' normal
// identifier case-folding applies for existing callers, which also
// means it cannot be defended with QuoteIdentifier the way the
// other identifier sinks in this package are.
func validateSearchPath(searchPath string) error {
    if !searchPathRegex.MatchString(searchPath) {
        return fmt.Errorf("invalid search path: %q", searchPath)
    }
    return nil
}
```

The lesson generalizes: quoting is an escaping transform, not a meaning-preserving one. It stops an identifier from breaking out of its syntactic slot, but it also changes what that identifier addresses whenever case sensitivity matters to the caller. Validation constrains which bytes are allowed through at all; quoting constrains how the allowed bytes are interpreted once they arrive. pg uses both together everywhere it safely can (registered table and column names), and falls back to validation alone in the one place quoting would change existing behavior (the search path). Keep that distinction in mind if you ever build your own identifier-accepting helper on top of this library: reach for `QuoteIdentifier` by default, and stop to ask whether case-folding matters before you do.

The Trust Boundary

Two categories of string end up inside a query pg builds for you, and they come from opposite sides of your application's trust boundary.

Developer-controlled. Struct tags, registered schema names, and literal strings in your own source code. These are written by you, reviewed in code review, and fixed at compile time (or at process start, for `Schema.MustRegister`). pg treats these as trusted: it validates their shape (see above) but does not, and cannot, second- guess their content.

Request-controlled. Anything that arrives over the wire: a query parameter, a JSON body field, a path segment, a header. This data must either arrive as a bind parameter (the normal case for values, see above) or pass through a validating helper before it can influence identifier position in a query. pg gives you exactly two such helpers:

- `Repository.OrderBy` (and its lower-level counterpart `desc.Table.OrderBy`), covered in Chapter 6. It takes a caller-supplied sort column, matches it case-insensitively against the table's own columns (plus an explicit `extraColumns` allowlist you provide), and returns a quoted `"column" ASC|DESC` fragment, or a descriptive error naming just the rejected column, never the full allowlist, so the error itself is safe to show a client.
- **The schema-validated table-name CRUD** in `db_crud.go` (`DB.DeleteByID`, `DB.DeleteBy`, `DB.ExistsBy`, `DB.CountBy`). These take a table name as a plain string, convenient for generic, table-agnostic code, and close the reintroduced risk the same way every time: the table name is resolved through `db.schema.GetByTableName` (an unregistered name fails before any SQL is built) and every column name in a `colValPairs` filter is resolved through that table's own `*desc.Table` (an unknown column fails the same way). Only names that already exist in your registered `*pg.Schema` ever reach a query, and even then they are quoted with `QuoteIdentifier`.

If a caller-chosen name needs to reach identifier position and neither of those two helpers fits your case, you are building your own allowlist-and-quote helper, not skipping the step. Never format a caller-supplied string directly into a query, “just this once”, even if the immediate context looks like it only accepts safe input; that assumption is exactly what regresses six months later when the caller changes.

Developer-Authored SQL in Tags and Builders

A handful of pg’s most flexible features work by taking a raw string and writing it into SQL verbatim, with no escaping, no validation beyond a bare syntax check, and no attempt to understand what it says. This is a deliberate, documented design: these strings are meant to be literal SQL fragments only you can write, not user input in disguise.

- The `default=`, `check=` and `generated=` struct tag options (`desc.Column.Default`, `CheckConstraint`, `GeneratedExpression`) are copied straight from the tag value into `CREATE TABLE`’s `DEFAULT`, `CHECK (...)` and `GENERATED ALWAYS AS (...) STORED` clauses.
- `desc.OnConflict.SetWhere` is appended, unescaped, as `" WHERE <SetWhere>"` after a generated `DO UPDATE SET` list; its own godoc says so directly: “developer-authored SQL, written verbatim with no escaping, never build it from end-user input.”
- Every fragment passed to `Where`, `And`, `AndIf` and the rest of `Conditions` (`where.go`, Chapter 6) is SQL text the library only renumbers placeholders in; it neither parses nor sanitizes it. The `column` and `elemType` arguments to helpers like `AndAnyOf` are the same kind of literal: `elemType` is even validated against a strict pattern and *panics* on a mismatch, the same fail-fast behavior `NewRepository` uses for a coding mistake, precisely because a malformed `elemType` here is a bug in your code, not a runtime condition to recover from.

The rule for all of these is the same, and it is worth internalizing rather than re-deriving each time: if the string is one you wrote and committed, it is fine, no matter how it reads. If any part of it is assembled from a request (`“status = ‘” + r.URL.Query().Get(“status”)

- ”’”`), you have reintroduced exactly the injection surface bind parameters exist to close, just one layer further from the query call site than usual, and consequently easier to miss in review.

Passwords

Marking a struct field `pg:"...,password"` gets you PostgreSQL-side hashing with no application code: on insert or update, pg wraps the value in `crypt($N, gen_salt('<PasswordAlg>'))`, a `pgcrypto` function that generates a random salt and returns a salted hash; on `Repository.SelectByUsernameAndPassword`, it compares the stored column against `crypt($plainPassword, storedColumn)`, which reuses the stored value's own salt to recompute the same hash and lets PostgreSQL itself do the comparison. The plaintext password is never returned from a successful lookup; only whether it matched.

`desc.PasswordAlg` (default `"bf"`, blowfish-based, PostgreSQL's recommended `crypt()` scheme) selects the algorithm `gen_salt` uses. Because this value is interpolated directly into the SQL fragment, every code path that emits it validates it first against a fixed allowlist:

```go
var passwordAlgAllowlist = map[string]bool{
    "bf": true, "md5": true, "xdes": true, "des": true,
}
```

`validatePasswordAlg` rejects anything outside that set, including a value engineered to break out of the surrounding SQL string literal. `PasswordAlg` is a package-level variable: set it once, in an `init` function or before your first query touches a password column, and never mutate it concurrently with query building.

For anything `pgcrypto` cannot express, or when hashing needs to happen application-side (to match an existing user store, to use a non-`crypt()` scheme such as bcrypt or Argon2 from Go), install a `desc.PasswordHandler` via `Schema.HandlePassword`:

```go
type PasswordHandler struct {
    Encrypt func(tableName, plainPassword string) (
        encryptedPassword string, err error)
    Decrypt func(tableName, encryptedPassword string) (
        plainPassword string, err error)
}
```

Setting only `Encrypt` is a meaningful, common configuration; the library's own example (`_examples/password/main.go`) recommends exactly that, leaving `Decrypt` unset unless a caller genuinely needs to read passwords back on select. Consider that recommendation carefully before you implement `Decrypt` at all. A `Decrypt` function only compiles if your `Encrypt` function is reversible, which means the stored value is not a one-way hash but a two-way encryption of the plaintext, recoverable by anyone who obtains your encryption key alongside your database. A proper password hash (bcrypt, Argon2, or PostgreSQL's own `crypt()` / `gen_salt()`) is one-way by design: verification recomputes the hash from a candidate password and compares, it never reconstructs the original. If you find yourself writing a working

`Decrypt`, that is a signal to reconsider whether the field should be a password at all, or a secret you are choosing to store reversibly for a different reason, in which case say so explicitly rather than routing it through the `password` tag.

Secrets in Logs

`WithLogger` installs a `pgx` `tracelog.TraceLog` tracer hardcoded to `tracelog.LogLevelTrace`, the most verbose level pgx has. At that level, pgx logs every SQL statement it executes together with every one of its bind arguments, including a plaintext password passed to `SelectByUsernameAndPassword`. The library's own godoc is explicit about this: "Do not use `WithLogger` against a production logger."

`WithLoggerLevel(logger, level)` is the production-safe alternative: it installs the same tracer, but lets you choose the `tracelog.LogLevel` instead of accepting the hardcoded `LogLevelTrace`. Lower it to `tracelog.LogLevelWarn` or `tracelog.LogLevelError` to log substantially less, or to `tracelog.LogLevelNone` to disable pgx's tracer logging entirely. One caveat worth keeping in mind at every level except `LogLevelNone`: pgx still logs the statement and its bind arguments for a *failed* query, down to and including `LogLevelError`, so a query that fails while carrying a password argument can still log it even under a level chosen to be quiet in the normal case. `LogLevelNone` is the only level that guarantees a sensitive bind argument never reaches the logger.

```go
db, err := pg.Open(ctx, schema, connString,
    pg.WithLoggerLevel(logger, tracelog.LogLevelWarn),
)
```

TLS

`pg.Open` accepts any connection string `pgxpool.ParseConfig` understands, including the standard `sslmode` parameter. Its own example connection string in `db.go`'s godoc uses `sslmode=disable` for a local walkthrough, and immediately notes: "For production connections, prefer `sslmode=verify-full` (or, at a minimum, `sslmode=require`)." Each value guarantees a different, and easy to conflate, subset of "secure":

sslmode	Encrypts the connection	Verifies the server's certificate	Verifies the hostname matches
`disable`	No	No	No
`allow`	Only if the server demands it	No	No

`prefer`	Yes, if available (falls back to plaintext)	No	No
`require`	Yes	No	No
`verify-ca`	Yes	Yes, against a trusted CA	No
`verify-full`	Yes	Yes	Yes

`require` stops a passive eavesdropper from reading traffic on the wire, but it does not confirm you are actually talking to your database server rather than an attacker positioned to intercept the connection: without certificate verification, an active man-in-the-middle can present its own certificate and `require` will still connect. `verify-ca` closes that gap for the certificate itself, but a certificate issued to the wrong hostname (by a CA you otherwise trust) still passes. `verify-full` is the only mode that defends against both, and it is the one to run in production. It needs a CA certificate available to the client (`sslrootcert` in the connection string, or the system trust store, depending on your provider); most managed PostgreSQL providers publish theirs specifically for this purpose.

Least-Privilege Database Roles

TLS protects the connection; the role your application connects as protects everything after that connection is established. Create a dedicated role per application (never a shared superuser, never the role you use for migrations) and grant it exactly the privileges the application needs: typically `SELECT`, `INSERT`, `UPDATE`, `DELETE` on its own tables, and nothing on tables it has no business touching. Schema-management operations, `DB.CreateSchema`, `DB.Migrate`, `DDL` you run by hand, belong to a separate, more privileged role used only at deploy time, not to the role your running service holds open in its connection pool around the clock. This is standard PostgreSQL role hygiene rather than anything pg enforces for you, but it composes directly with everything above: even a successful injection or a compromised credential is bounded by what the connected role is actually allowed to do.

Error Messages Without Leaking Schema

A raw PostgreSQL error is not safe to hand back to a client. Its message text can include table names, column names, constraint names and the literal offending value (`Key (email)=(x@y.com) already exists.`), all of which are internal details a client has no business seeing, and some of which (the value itself) may be the user's own input reflected back in a way that looks like a bug even when it is not.

`AsConstraintError` (`errors.go`, introduced in Chapter 10) exists to break that coupling. It extracts a typed `*ConstraintError` from a PostgreSQL SQLSTATE class-23 integrity-constraint violation, giving you a structured `Kind` (`ConstraintUnique`, `ConstraintForeignKey`, `ConstraintNotNull`, `ConstraintCheck`, `ConstraintExclusion`) to switch on, instead of the raw message text to parse:

```go
err := repo.InsertSingle(ctx, customer, &customer.ID)
if cerr, ok := pg.AsConstraintError(err); ok {
    switch cerr.Kind {
    case pg.ConstraintUnique:
        http.Error(w, "already exists", http.StatusConflict)
    case pg.ConstraintForeignKey:
        http.Error(w, "referenced row does not exist",
            http.StatusUnprocessableEntity)
    case pg.ConstraintNotNull, pg.ConstraintCheck:
        http.Error(w, "invalid request", http.StatusBadRequest)
    default:
        http.Error(w, "constraint violation", http.StatusConflict)
    }
    return
}
```

The pattern that keeps this safe is choosing the client-facing message from `Kind` (a small, closed set you wrote yourself) rather than echoing `cerr.Error()` or `cerr.Detail` back verbatim: those two still carry the constraint name and the offending value straight from PostgreSQL, exactly the internal detail you are trying to keep off the wire. Log the full `cerr` (or the original `err`) server-side, where that detail is genuinely useful for debugging, and return only the curated message to the caller. `AsConstraintError` is extraction-only: pg never wraps its own returned errors in a `ConstraintError` for you, so this mapping only happens where you explicitly call it, at whatever layer in your application is responsible for turning database failures into a response.

Summary

- Values always travel as bind parameters (`$1`, `$2`, ...); this is automatic everywhere in the library and is not something you opt into.
- Identifiers cannot be bind parameters, so pg validates every table, column and unique index name at registration (`^[A-Za-z_][A-Za-z0-9_$]*$`) and quotes it with `QuoteIdentifier` (`pgx.Identifier.Sanitize`) wherever it is written into SQL.
- Quoting and validating solve different problems: quoting escapes an identifier so it cannot break out of its syntactic slot, but it also makes the identifier case-sensitive, which is why `db.searchPath` is validated but deliberately never quoted.

- Struct tags and registered schema names are developer-controlled and trusted; anything from a request must arrive as a bind parameter or pass through `Repository.OrderBy` / `desc.Table.OrderBy` or the schema-validated table-name CRUD before it can influence identifier position.
- `default=`, `check=`, `generated=` tags, `OnConflict.SetWhere` and every `Conditions` fragment are developer-authored SQL written verbatim: never build any of them from user input.
- Password columns can be hashed server-side via `crypt()` / `gen_salt()` (algorithm restricted to an allowlist) or client-side via a `PasswordHandler`; a working `Decrypt` implies reversible storage, which is a design smell for anything called a password.
- `WithLogger` logs full SQL and bind arguments, including plaintext passwords, at trace level; use `WithLoggerLevel` in production, down to `LogLevelNone` if you need a hard guarantee against a leaked secret in a failed-query log line.
- Prefer `sslmode=verify-full` in production; `require` alone does not verify the server's identity.
- Run your application under a least-privilege database role, separate from whatever role applies migrations.
- Map database failures to client responses through `AsConstraintError` and `Kind`, not by echoing `Error()` or `Detail` back to the caller.

Further Reading

- PostgreSQL: SQL Injection Prevention: the lexical rules behind quoted versus unquoted identifiers and case-folding referenced in this chapter.
- PostgreSQL: pgcrypto: the extension behind `crypt()` and `gen_salt()`, including the algorithm identifiers `PasswordAlg` accepts.
- PostgreSQL: SSL Support: the full definition of every `sslmode` value in the table above.
- OWASP SQL Injection Prevention Cheat Sheet: the general defense-in-depth framing this chapter's identifier discipline fits into.
- OWASP Password Storage Cheat Sheet: current guidance on one-way password hashing, the property a reversible `Decrypt` gives up.
- pgx: tracelog: the `Logger` interface and `LogLevel` values `WithLogger` and `WithLoggerLevel` configure.

CHAPTER 15

Observability and Operations

Getting a query to run correctly is only part of running pg in production. This chapter covers the rest: telling your orchestrator whether the database is actually reachable, watching the connection pool for the counters that predict an outage before it happens, choosing how much of your SQL ends up in your logs, wiring in a tracer without pg ever depending on one, sizing the pool and picking an exec mode for the deployment you actually have (including the one where PgBouncer forces your hand), and reasoning about timeouts and retries in terms an on-call engineer can act on at 3 a.m. Every API named here is read directly from `db_health.go`, `db_stat.go`, `db_options.go`, `db.go`, `db_information.go` and `retry.go`.

Readiness and Liveness

`DB.Ping` acquires a connection from the pool (opening one if necessary) and reports whether the database answered:

```go
if err := db.Ping(ctx); err != nil {
    // not reachable.
}
```

`Ping` always goes through the pool directly, even when called on a transaction-scoped `*DB` (one returned inside `InTransaction`'s callback): `pgx.Tx` has no `Ping` of its own, and a liveness check is about whether the database is reachable at all, not about the specific connection a given transaction happens to be pinned to. Calling it inside a transaction therefore never touches, and cannot invalidate, that transaction.

`DB.Health` builds on `Ping` for a richer readiness response: it pings, then reports the server version and a pool snapshot together as a `Health` value:

```go
type Health struct {
    ServerVersion string   `json:"server_version"`
    Pool          PoolStat `json:"pool"`
```

```
}
```

```go
func healthHandler(db *pg.DB) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        health, err := db.Health(r.Context())
        if err != nil {
            http.Error(w, err.Error(), http.StatusServiceUnavailable)
            return
        }

        json.NewEncoder(w).Encode(health)
    }
}
```

Use `Ping` for a minimal liveness probe (is the process still able to talk to the database at all) and `Health` for a readiness endpoint where you also want to surface the server version and pool pressure to whatever is polling it, a deploy script, a dashboard, an on-call runbook. Like `Ping`, `Health` always checks the pool directly and never disturbs an in-flight transaction.

Pool Statistics

`DB.PoolStat` returns a point-in-time snapshot of the connection pool's counters as a plain, JSON-serializable `PoolStat` struct, built directly from `pgxpool.Pool.Stat()`:

Field	Meaning
`AcquireCount`	Cumulative count of successful acquires.
`AcquireDuration`	Cumulative time spent acquiring connections.
`AcquiredConns`	Connections currently checked out.
`CanceledAcquireCount`	Acquires canceled by a context before completing.
`ConstructingConns`	Connections currently being established.
`EmptyAcquireCount`	Acquires that had to wait because the pool was empty.
`IdleConns`	Connections currently idle in the pool.
`MaxConns`	The pool's configured maximum size.
`TotalConns`	`ConstructingConns + AcquiredConns + IdleConns`.

Two of these are worth alerting on directly rather than only eyeballing on a dashboard. `EmptyAcquireCount` climbing means requests are regularly waiting for a connection because none were free, the first sign your pool is undersized (or a slow query, or a leak, is holding connections longer than it should) for the load you are actually serving. `AcquiredConns` sitting at

or near `MaxConns` for sustained periods is the same symptom from a different angle: the pool has no headroom left, and the next burst of traffic queues instead of proceeding. `CanceledAcquireCount` rising tells you requests are timing out while waiting for a connection, which is what an undersized pool looks like from the caller's side, a context deadline expiring during `Acquire` rather than during the query itself.

```go
stat := db.PoolStat()
if stat.AcquiredConns >= stat.MaxConns {
    log.Warn("connection pool saturated", "max", stat.MaxConns)
}
```

Logging

`WithLogger` and `WithLoggerLevel` install a `pgx` `tracelog.TraceLog` tracer as a `ConnectionOption` passed to `pg.Open`. `WithLogger` is hardcoded to `tracelog.LogLevelTrace`, the most verbose level pgx has: every statement and every bind argument, useful while developing, unsafe as a default in production because it logs plaintext values, including passwords (see Chapter 14 for the full argument). `WithLoggerLevel(logger, level)` is the same tracer with a level you choose:

```go
db, err := pg.Open(ctx, schema, connString,
    pg.WithLoggerLevel(logger, tracelog.LogLevelWarn),
)
```

`logger` is anything satisfying `tracelog.Logger` (`Log(ctx, level, msg, data)`), so it adapts to whatever structured logging library your service already uses. Choose the level for the volume and sensitivity you are willing to accept: `LogLevelError` (or `LogLevelWarn`, one step more verbose) for routine production traffic, `LogLevelDebug` or `LogLevelTrace` for a short-lived, deliberately narrow debugging window, `LogLevelNone` to disable pgx's tracer logging entirely.

Query Tracers and OpenTelemetry

`WithQueryTracer` composes one or more `pgx.QueryTracer` implementations into the pool, using pgx's own `multitracer` package underneath. This is how pg supports distributed tracing without ever importing a tracing library itself: `pg`'s `go.mod` carries no OpenTelemetry dependency, and adding tracing is entirely something you opt into from your own module.

```go
import "github.com/exaring/otelpgx"
```

```
db, err := pg.Open(ctx, schema, connString,
    pg.WithQueryTracer(otelpgx.NewTracer()),
)
```

otelpgx is one such tracer, maintained outside this project, that turns every query into an OpenTelemetry span. `WithQueryTracer` composes rather than replaces: calling it more than once, or passing more than one tracer in a single call, wraps every tracer already installed (including one set by an earlier `ConnectionOption` in the same `Open` call) inside a single `multitracer.Tracer` that fans calls out to all of them, so a metrics tracer and a tracing tracer can coexist. Calling `WithQueryTracer` with zero tracers is a documented no-op: it never clears whatever tracer was already installed.

The WithLogger/WithQueryTracer Ordering Caveat

`WithLogger` and `WithLoggerLevel` do not compose the way `WithQueryTracer` does: both overwrite `poolConfig.ConnConfig.Tracer` outright, discarding anything already installed there. Combine logging with tracing by passing `WithLogger` / `WithLoggerLevel` *before* `WithQueryTracer` in the same `Open` call:

```
db, err := pg.Open(ctx, schema, connString,
    pg.WithLoggerLevel(logger, tracelog.LogLevelWarn), // first.
    pg.WithQueryTracer(otelpgx.NewTracer()),           // second.
)
```

GO

Written in the reverse order, `WithQueryTracer` installs its tracer first, and the following `WithLoggerLevel` call silently replaces it, so the otelpgx spans simply stop appearing with no error telling you why. `ConnectionOption` values run in the order you list them in `Open`'s call, so this is a plain ordering rule to hold onto, not a race condition: get the order right once, in one place, and it stays right.

Connection Pool Sizing

Every pool parameter `pgxpool.ParseConfig` recognizes can be set directly in the connection string passed to `pg.Open`, with no `ConnectionOption` required:

```
connString := fmt.Sprintf(
    "host=%s port=%d user=%s password=%s dbname=%s sslmode=%s "+
        "pool_max_conns=%d pool_min_conns=%d "+
        "pool_max_conn_lifetime=%s pool_max_conn_idle_time=%s "+
```

GO

```
        "pool_health_check_period=%s",
    host, port, user, password, dbname, sslMode,
    maxConns, minConns, maxConnLifetime, maxConnIdleTime, healthCheckPeriod,
)
```

`pool_max_conns` is the ceiling `PoolStat.MaxConns` reports and the number your `EmptyAcquireCount` / `AcquiredConns` alerting above is measured against; there is no universally correct value; it depends on how many concurrent queries your workload actually issues and how many connections PostgreSQL itself is configured to accept (`max_connections`) across every client sharing that server. `pool_min_conns` keeps that many connections warm even when idle, at the cost of holding them open against the server's own connection budget. `pool_max_conn_lifetime` and `pool_max_conn_idle_time` bound how long a connection can live or sit idle before the pool recycles it, useful for rotating connections behind a load balancer or a database that periodically restarts. `pool_health_check_period` sets how often the pool proactively checks idle connections rather than discovering a dead one only when a caller tries to use it.

Exec Modes and Prepared-Statement Caching

Three more `ConnectionOption` values, and their connection-string equivalents, control how pgx sends queries and how much it caches about them:

Option	Connection-string equivalent	Purpose
`WithDefaultQueryExecMode(mode)`	`default_query_exec_mode`	Selects pgx's query execution protocol, e.g. `pgx.QueryExecModeSimpleProtocol`.
`WithStatementCacheCapacity(n)`	`statement_cache_capacity`	Size of the automatic prepared-statement cache; `0` disables it.
`WithDescriptionCacheCapacity(n)`	`description_cache_capacity`	Size of the statement-description cache pgx's describe exec mode uses.

Under pgx's default exec mode, a query pgx has seen before is prepared once and reused, which saves a parse/plan round trip on every repeated query shape. That optimization assumes a stable, long-lived connection: the server remembers the prepared statement for the life of that specific connection.

PgBouncer and Transaction Pooling

That assumption breaks under PgBouncer configured for transaction pooling (the mode most deployments actually use, since it is the one that lets many application connections share a small number of real server connections): a prepared statement created on one physical connection may not exist on the next one your session gets handed after the transaction ends, because PgBouncer, not pgx, decides which underlying connection you get next. The fix is to stop relying on server-side prepared statements at all:

```go
db, err := pg.Open(ctx, schema, connString,
    pg.WithDefaultQueryExecMode(pgx.QueryExecModeSimpleProtocol),
    pg.WithStatementCacheCapacity(0),
    pg.WithDescriptionCacheCapacity(0),
)
```

or, equivalently, entirely as connection-string parameters, with no Go code involved:

```
postgres://user:pass@pgbouncer-host:6432/mydb?sslmode=verify-full&default_query_exec_mode=simple_protocol&statement_cache_capacity=0&description_cache_capacity=0
```

If your deployment fronts PostgreSQL with PgBouncer (or any pooler that multiplexes application connections onto a smaller set of server connections) in transaction-pooling mode, `WithDefaultQueryExecMode(pgx.QueryExecModeSimpleProtocol)` is not optional tuning, it is what makes the connection work at all; without it, queries intermittently fail with errors about a prepared statement that does not exist on the connection pgx happened to be handed. `WithStatementCacheCapacity(0)` and `WithDescriptionCacheCapacity(0)` are not strictly required for that default behavior, since pgx never populates either cache while the default exec mode stays `QueryExecModeSimpleProtocol`. Set them anyway: `*DB`'s `Query` / `Exec` / `QueryRow` pass their `args` straight through to pgx, which recognizes a `pgx.QueryExecMode` value among those args as a per-call override, so disabling both capacities makes a call that explicitly opts back into `QueryExecModeCacheStatement` or `QueryExecModeCacheDescribe` fail immediately with an explicit error instead of silently caching a statement handle across a pooled connection swap.

Vacuum and Table Sizes

Chapter 13 covers the introspection API in full; three of its methods are specifically operational tools worth revisiting here. `DB.IsAutoVacuumEnabled` reports whether autovacuum is running at all (`SHOW autovacuum;`). `DB.ListTableSizes` returns every table's on-disk size, largest first, as `[]TableSizeInfo`, and `DB.GetSize` sums the same figures across the whole search path into one `SizeInfo`. Wire these into a periodic job, not just a one-off diagnostic: a table growing

faster than expected, or an autovacuum setting that quietly drifted to `off`, is exactly the kind of slow-burn problem a dashboard catches long before it becomes an incident, and both are a handful of lines against an API you already have open for schema introspection.

Timeouts

There is exactly one timeout mechanism. Every `*DB` method that talks to PostgreSQL takes a `context.Context` as its first argument, and pg does not impose a timeout of its own on top of it: the context you pass is the only deadline in effect. A `context.WithTimeout` around a single query bounds that query; a shorter deadline passed into `InTransaction`'s outer `ctx` bounds the whole transaction, since every statement inside it shares that same context. Choose deadlines deliberately rather than opening every handler with a blanket five-second `context.WithTimeout` and hoping it fits every code path: a bulk `CopyFrom` and a single-row `SelectByID` have very different legitimate durations, and a timeout tuned for one starves the other.

A canceled or expired context surfaces as a plain Go error from whatever call was in flight, `context.DeadlineExceeded` or `context.Canceled`, wrapped the way pgx wraps it; nothing about pg changes that shape, so the same `errors.Is` checks you would already write against `context` apply unchanged.

Retrying Transient Failures

Some PostgreSQL failures are not really failures of your query, they are the server telling you a transaction cannot be allowed to complete as if it ran alone and must be retried from scratch: SQLSTATE `40001` (`serialization_failure`) and `40P01` (`deadlock_detected`). `DB.InTransactionRetry` (and its `Repository[T]` counterpart), introduced in Chapter 9, automates exactly that retry with exponential backoff and full jitter:

```go
err := db.InTransactionRetry(ctx, pg.RetryOptions{
    TxOptions: pgx.TxOptions{IsoLevel: pgx.Serializable},
}, func(tx *pg.DB) error {
    // statements that might race another transaction under SERIALIZABLE.
    return nil
})
```

Pair it with `pgx.Serializable` deliberately: under the default read committed isolation level, `40001` mostly cannot happen in the first place, so `RetryOptions.TxOptions` is where you opt into the isolation level that makes the retry logic meaningful. `RetryOptions.MaxAttempts` (default 3), `BaseDelay` (default 50ms) and `MaxDelay` (default 1s) bound how hard and how long it tries before giving up and returning the last attempt's error as-is. Every attempt runs in its own, brand-new transaction, so `fn` must re-read whatever state it needs from the database on each call rather than assuming it only runs once; nothing carries over between attempts except what `fn` itself re-derives.

Operationally, treat a rising count of retried transactions the same way you would treat rising `EmptyAcquireCount`: it is a leading indicator, here of contention on a hot row or a serialization conflict pattern in your workload, worth graphing even though `InTransactionRetry` is, by design, absorbing the symptom for you before it becomes a user-visible error.

Summary

- `DB.Ping` is a minimal liveness check; `DB.Health` adds server version and `PoolStat` for a richer readiness endpoint. Both always check the pool directly, never an in-flight transaction.
- `PoolStat`'s `EmptyAcquireCount`, `AcquiredConns` near `MaxConns`, and `CanceledAcquireCount` are the counters that predict pool exhaustion before it becomes an outage.
- `WithLogger` is trace-level and unsafe for production logs; `WithLoggerLevel` lets you choose the verbosity, down to `LogLevelNone` for a hard guarantee against logging sensitive bind arguments.
- `WithQueryTracer` composes `pgx.QueryTracer` implementations (such as otelpgx) with no OpenTelemetry dependency in pg itself; pass `WithLogger` / `WithLoggerLevel` before it in the same `Open` call, or it silently overwrites the tracer slot.
- Pool size, lifetime and health-check parameters are all available directly in the connection string (`pool_max_conns`, `pool_min_conns`, `pool_max_conn_lifetime`, `pool_max_conn_idle_time`, `pool_health_check_period`).
- `WithDefaultQueryExecMode`, `WithStatementCacheCapacity` and `WithDescriptionCacheCapacity` (or their connection-string equivalents) are required, not optional, behind PgBouncer in transaction-pooling mode.
- `DB.IsAutoVacuumEnabled`, `DB.ListTableSizes` and `DB.GetSize` from Chapter 13 double as operational monitoring tools.
- Timeouts are entirely `context.Context`-driven; pg adds no timeout of its own. `DB.InTransactionRetry` handles the one class of failure (`40001` / `40P01`) that is meant to be

retried rather than surfaced.

Further Reading

- pgxpool: Config: the full set of pool parameters, including every `pool_*` connection-string key referenced in this chapter.
- pgx: QueryExecMode: the exec modes `WithDefaultQueryExecMode` selects between.
- otelpgx: the OpenTelemetry tracer used as this chapter's `WithQueryTracer` example.
- PgBouncer: Documentation: pooling modes, including transaction pooling and its prepared statement caveat.
- PostgreSQL: Transaction Isolation: the SERIALIZABLE isolation level and the `40001` / `40P01` failure modes `InTransactionRetry` targets.
- Go: Context package: `WithTimeout`, `WithDeadline` and cancellation propagation, the mechanism behind every timeout in this chapter.

CHAPTER 16

Testing

pg's entire value proposition is the SQL it generates from your structs and tags: the quoting, the placeholder numbering, the `ON CONFLICT` clause, the generated column list. A mock of `*pg.DB` tests none of that; it tests that your code calls a method with the arguments you told the mock to expect, which is a fact about your code, not about whether the resulting query is correct PostgreSQL. This chapter is about testing against a real PostgreSQL server instead, and about the `pgtest` package this library ships specifically to make that fast and safe: `pgtest.ConnString` for skipping cleanly when no database is configured, and `pgtest.New` for an ephemeral, per-test schema created and torn down automatically. It then covers the patterns worth building on top of that foundation: table-driven tests over a repository, testing rollback with `ErrIntentionalRollback`, asserting on typed failures with `AsConstraintError`, and running all of it in CI against a service container, the same way this library's own test suite runs.

Background

If you are comfortable with Go's `testing` package and table-driven tests, skip to Why a Real Database, Not a Mock. A Go test is an ordinary function named `TestXxx(t *testing.T)` that `go test` discovers and runs; failing an expectation inside it (`t.Fatalf`, `t.Errorf`) marks that test failed without crashing the rest of the suite. `t.Run` opens a named subtest, most often used to drive one test function over a table of cases (a *table-driven test*): a slice of structs, one per scenario, looped over with `t.Run(tt.name, func(t *testing.T) { ... })` so each case reports independently and adding a new one is adding a row rather than a new function. `t.Cleanup` registers a function that runs when the current test (or subtest) finishes, success or failure, the mechanism `pgtest.New` uses to drop its schema and close its pool automatically. `testing.TB` is the common interface `*testing.T` and `*testing.B` both satisfy, which is why a helper written against `testing.TB` works from either a test or a benchmark.

Why a Real Database, Not a Mock

A mock proves your code called something; it cannot prove the SQL was correct. A mock of the query interface pg uses under the hood can only assert that your code issued *a* call; it cannot tell you whether the SQL that call would have produced is valid, whether the `ON CONFLICT` target you configured actually matches your unique constraint, whether a `CHECK` constraint you wrote in a struct tag is satisfied, or whether a generated column's expression even parses. All of that logic lives in `desc/insert_query.go`, `desc/on_conflict.go`, `desc/create_table_query.go`, and it only proves itself correct by actually running against PostgreSQL. Given that, a mock's only honest job in a pg-based test suite is standing in for something *other* than the database, an HTTP client to a third-party service, a clock, and even then the repository layer itself, the part that talks to PostgreSQL, is exactly the part worth exercising for real.

The practical objection to a real database in tests, that it is slow and stateful and tests interfere with each other, is what `pgtest` exists to remove: a schema per test, created and dropped automatically, so tests are as isolated as a mock would have been, and fast enough to run on every save against a local PostgreSQL or a CI service container.

The pgtest Package

`pgtest` provides two functions: `ConnString`, which reads where to connect from an environment variable and skips cleanly when it is unset, and `New`, which creates an isolated schema, materializes your registered tables in it, and cleans up automatically.

```go
import "github.com/kataras/pg/pgtest"
```

Import it from your own test files (`_test.go`), never from production code; it depends on `testing.TB` and is meant exclusively for test setup.

ConnString and Skipping Cleanly

```go
func TestSomething(t *testing.T) {
    connString := pgtest.ConnString(t)
    // ...
```

```
}
```

`ConnString` reads the `PG_CONNSTRING` environment variable. If it is unset or empty, it calls `t.Skipf` with an explanatory message instead of guessing a default, so the test reports as skipped, not failed, in an environment with no database configured. This is a deliberate difference from pg's own internal test suite (`getTestConnString` in `db_example_test.go`), which falls back to a hardcoded local connection string so its own CI job always has something to run against. `pgtest` has no such fallback, since it is meant to be imported by downstream projects whose database might live on a different host, user or password entirely; a hardcoded default there would be meaningless at best and silently wrong at worst.

New: an Ephemeral Schema per Test

```go
db := pgtest.New(t, schema, connString)
```

`New` takes the calling test, a `*pg.Schema` with your tables registered, and a connection string, and does the following, in order: generates a random schema name (`pgtest_<16 hex chars>`), opens a fresh `pgxpool.Pool` whose `search_path` defaults to that schema, issues `CREATE SCHEMA`, registers a `t.Cleanup` that drops the schema (`CASCADE`, so every table inside it goes too) and closes the pool, then, if `schema` has any tables registered, calls `DB.CreateSchema` to materialize them and returns a ready-to-use `*pg.DB`. Cleanup is registered immediately after the schema itself exists, specifically so that a later failure (a bad tag, an unsupported type, a foreign key to a table that does not exist yet while you are iterating) still drops the schema instead of leaking it into the target database.

An empty schema (`pg.NewSchema()`, nothing registered) is a valid argument too: `New` still creates and cleans up the isolated schema, leaving table creation to you, useful for tests that want a scratch schema for hand-written DDL rather than the struct-driven path.

The Shared-Schema Constraint

`New` mutates the `*pg.Schema` you pass it: it repoints every registered table's `SearchPath` field at the new ephemeral schema name, in place, because `desc.Table`'s query builders schema-qualify every `INSERT` / `UPSERT` using that field, captured once at `Schema.MustRegister` time,

rather than the live connection's `search_path`. Left alone, every insert would keep targeting whatever schema was in effect when you *registered* the table, `"public"` by default, not the ephemeral one `New` just created.

Because `schema.Tables()` returns the `Schema`'s own live `*desc.Table` pointers rather than copies, two `New` calls sharing one `*pg.Schema` at the same time would be an unsynchronized write to the same memory from two goroutines, a genuine data race, not merely a source of occasional flaky assertions. `New` detects this and fails fast: a second `New` call given a `*pg.Schema` that an earlier, still-active `New` call already holds calls `t.Fatalf` immediately with a message naming the reuse, rather than letting the two calls race silently.

The fix is simple and cheap: build a fresh `*pg.Schema` inside each test, as every example in this chapter does, rather than sharing one package-level `*pg.Schema` across tests. Registering the same struct types on a fresh `Schema` repeatedly costs nothing meaningful.

A Full Test

GO

```go
// widget_test.go
package store_test

import (
    "context"
    "testing"

    "github.com/kataras/pg"
    "github.com/kataras/pg/pgtest"
)

type widget struct {
    ID   string `pg:"type=uuid,primary"`
    Name string `pg:"type=varchar(64)"`
}

func widgetSchema() *pg.Schema {
    schema := pg.NewSchema()
    schema.MustRegister("widgets", widget{})
    return schema
}

func TestWidgetInsertAndSelect(t *testing.T) {
    connString := pgtest.ConnString(t)
    ctx := context.Background()

    db := pgtest.New(t, widgetSchema(), connString)
    repo := pg.NewRepository[widget](db)

    inserted := widget{Name: "gear"}
    var id string
    if err := repo.InsertSingle(ctx, inserted, &id); err != nil {
        t.Fatalf("insert single: %v", err)
    }
```

```
    if id == "" {
        t.Fatal("expected a generated uuid primary key")
    }

    got, err := repo.SelectByID(ctx, id)
    if err != nil {
        t.Fatalf("select by id: %v", err)
    }
    if got.Name != "gear" {
        t.Fatalf("expected name %q, got %q", "gear", got.Name)
    }
}
```

Run it with the environment variable set, against any reachable PostgreSQL:

```sh
PG_CONNSTRING="postgres://postgres:pass@localhost:5432/test_db?sslmode=disable" go test ./... -v
```

Run it with `PG_CONNSTRING` unset and the test reports `SKIP`, not `FAIL`, so a contributor without a local database still gets a clean `go test ./...` for everything else in the package.

Table-Driven Tests Over a Repository

The table-driven shape composes directly with `pgtest.New`: build one fresh schema and repository per subtest (or once per top-level test, if the cases do not interfere with each other's rows) and loop over the cases as usual.

```go
func TestWidgetValidation(t *testing.T) {
    connString := pgtest.ConnString(t)
    ctx := context.Background()

    tests := []struct {
        name    string
        widget  widget
        wantErr bool
    }{
        {"valid name", widget{Name: "gear"}, false},
        // No NOT NULL/CHECK constraint on Name in this table.
        {"empty name", widget{Name: ""}, false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            db := pgtest.New(t, widgetSchema(), connString)
            repo := pg.NewRepository[widget](db)

            var id string
            err := repo.InsertSingle(ctx, tt.widget, &id)
            if (err != nil) != tt.wantErr {
                t.Fatalf("InsertSingle() error = %v, wantErr %v",
                    err, tt.wantErr)
            }
```

```
        })
    }
}
```

Building a new schema per subtest costs one `CREATE SCHEMA` and, for this table, one `CREATE TABLE`, both cheap operations; it buys complete isolation between cases, so an earlier case's rows can never change a later case's assertions, and `t.Parallel()` is safe to add to each subtest if the suite grows large enough to want it. (`pgtest.New` itself has no built-in concurrency limit beyond the shared-schema constraint described above: two subtests running in parallel are fine as long as each builds its own `*pg.Schema`, which this pattern already does.)

Testing Transaction Rollback

`ErrIntentionalRollback` (Chapter 9) is a sentinel your `InTransaction` callback can return to roll back on purpose, useful in tests that want to verify a sequence of writes without persisting any of them, or that want to prove a rollback genuinely discards work rather than merely returning an error:

```go
func TestInsertRollsBackCleanly(t *testing.T) {
    connString := pgtest.ConnString(t)
    ctx := context.Background()

    db := pgtest.New(t, widgetSchema(), connString)

    err := db.InTransaction(ctx, func(tx *pg.DB) error {
        repo := pg.NewRepository[widget](tx)

        var id string
        toInsert := widget{Name: "should not persist"}
        if err := repo.InsertSingle(ctx, toInsert, &id); err != nil {
            return err
        }

        return pg.ErrIntentionalRollback
    })
    if err != nil {
        t.Fatalf("expected nil after an intentional rollback, got: %v", err)
    }

    repo := pg.NewRepository[widget](db)
    count, err := repo.Count(ctx, "SELECT COUNT(*) FROM widgets")
    if err != nil {
        t.Fatalf("count: %v", err)
    }
    if count != 0 {
        t.Fatalf("expected 0 rows after rollback, got %d", count)
    }
}
```

`InTransaction` returns `nil` when `fn` returns `pg.ErrIntentionalRollback`, since the rollback itself succeeded and was requested, not an error condition; asserting `err == nil` here is therefore the correct check, and the row count query is what actually proves the insert never committed. This is the same pattern the library's own suite uses to verify `InTransaction`'s rollback behavior end to end, against a real transaction rather than an assumption about what `ROLLBACK` does.

Asserting on Typed Errors

`AsConstraintError` (Chapter 10, and see Chapter 14 for why you should prefer it over parsing error text) turns a PostgreSQL constraint violation into a typed value you can assert on directly, instead of matching against a driver error string that could change wording between versions:

```go
type product struct {
    ID  string `pg:"type=uuid,primary"`
    SKU string `pg:"type=varchar(32),unique"`
}

func productSchema() *pg.Schema {
    schema := pg.NewSchema()
    schema.MustRegister("products", product{})
    return schema
}

func TestDuplicateSKURejected(t *testing.T) {
    connString := pgtest.ConnString(t)
    ctx := context.Background()

    db := pgtest.New(t, productSchema(), connString)
    repo := pg.NewRepository[product](db)

    var firstID string
    first := product{SKU: "WIDGET-1"}
    if err := repo.InsertSingle(ctx, first, &firstID); err != nil {
        t.Fatalf("first insert: %v", err)
    }

    var secondID string
    err := repo.InsertSingle(ctx, product{SKU: "WIDGET-1"}, &secondID)
    if err == nil {
        t.Fatal("expected a unique constraint violation on the duplicate SKU")
    }

    cerr, ok := pg.AsConstraintError(err)
    if !ok {
        t.Fatalf("expected a *pg.ConstraintError, got: %v", err)
    }
    if cerr.Kind != pg.ConstraintUnique {
        t.Fatalf("expected ConstraintUnique, got: %s", cerr.Kind)
    }
}
```

Asserting on `cerr.Kind` rather than on `err.Error()`'s text keeps the test tied to what PostgreSQL actually reports (the SQLSTATE class), not to incidental message wording, so it stays correct across PostgreSQL versions and across whether the violated constraint was declared `unique` or `unique_index` in the struct tag.

Running the Suite in CI

pg's own test suite is a working example of every pattern in this chapter, run against a real PostgreSQL service container rather than any form of mock. Its GitHub Actions workflow starts a `postgres:16-alpine` service alongside the test job:

```yaml
services:
  postgres:
    image: postgres:16-alpine
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin!123
      POSTGRES_DB: test_db
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5

steps:
  - uses: actions/setup-go@v6
    with:
      go-version: 1.26.x
  - uses: actions/checkout@v6
  - run: go test -v ./...
```

The service container's health check (`pg_isready`, polled every 10 seconds up to 5 times) ensures the job's `go test` step never starts racing a database that has not finished starting up. The library's own `getTestConnString` helper falls back to a hardcoded connection string matching these exact service credentials when `PG_CONNSTRING` is unset, which is why `go test -v ./...` works unmodified in this CI job with no extra environment configuration; a downstream project using `pgtest` instead sets `PG_CONNSTRING` explicitly as a workflow environment variable, pointing at its own service container, and gets the clean-skip behavior described earlier for any environment where that variable is absent (a contributor's laptop with no local PostgreSQL, for instance).

Summary

- Mock `*pg.DB` tests your code's calls, not the SQL pg generates from your structs; a real PostgreSQL is the only thing that proves the query itself is correct, which is the entire value this library provides.
- `pgtest.ConnString(t)` reads `PG_CONNSTRING` and skips cleanly (not a failure) when it is unset, with no hardcoded fallback, unlike the library's own internal test helper.
- `pgtest.New(t, schema, connString)` creates a randomly named, isolated schema, materializes the registered tables in it via `DB.CreateSchema`, and drops it (plus closes the pool) automatically on cleanup.
- `New` mutates the `*pg.Schema` you pass it (retargeting every table's `SearchPath`), so build a fresh `*pg.Schema` per test rather than sharing one; `New` detects and fails fast on a shared, still-active `*pg.Schema` instead of racing.
- `pg.ErrIntentionalRollback` lets a transaction test prove a rollback genuinely discarded its writes, not merely that an error was returned.
- `pg.AsConstraintError` and `ConstraintKind` let a test assert on the SQLSTATE class of a failure directly, immune to message wording changes across PostgreSQL versions.
- The library's own suite runs this same way in CI: a `postgres:16-alpine` service container, gated by a health check, with a connection string sourced from `PG_CONNSTRING` (or, for the library's own tests only, a matching hardcoded default).

Further Reading

- Go: Testing package: `*testing.T`, `t.Run` subtests, `t.Cleanup`, `t.Skipf`, and the `testing.TB` interface `pgtest` is written against.
- Go: Table-driven tests: the idiom behind every table-driven example in this chapter.
- GitHub Actions: Service containers: the mechanism behind the `postgres:16-alpine` service in this chapter's CI example.
- PostgreSQL: Error Codes: the full SQLSTATE table `ConstraintKind` and `AsConstraintError` classify against.
- PostgreSQL: CREATE SCHEMA: the statement `pgtest.New` issues to create each test's isolated namespace.

Epilogue

Sixteen chapters ago this book promised a path from an empty `go.mod` to a tested, production-shaped data layer. Here at the end, it is worth naming what you actually carry out of it, because it is more than a list of function signatures.

What You Can Do Now

You can take a set of Go structs, tag them with `pg:"..."`, register them in a `Schema`, and both create and verify the tables they describe against a live database, so a drifted schema fails loudly at startup instead of quietly at 2 a.m. You can read and write through a typed `Repository[T]` without hand-writing SQL for the common cases, and drop down to the `Where` builder, raw `Query`, or a hand-written statement the moment a case stops being common. You know when to reach for a multi-row `InsertMany` or `CopyFrom` instead of a loop of single-row inserts, and what each one costs you in return: `COPY` is faster, and it gives up per-row `DEFAULT` and per-row constraint errors. You can wrap a unit of work in `InTransaction`, tell a transient serialization failure apart from a real one, and retry only the former. You can subscribe to a table's changes with `ListenTable` instead of polling it, and you can point `gen` at a live database to get Go structs back, or at an already registered `Schema` to get typed column constants.

Just as importantly, you know what pg does *for* you: the prepared statement cache and description cache it maintains so repeated queries do not re-plan, the connection pool it hands you rather than a bare connection, and the specific `SQLSTATE` each constraint violation maps to so your error handling can branch on `ConstraintKind` instead of matching a driver's error string. Good libraries are measured by the footguns they remove; a fair amount of this book has quietly been about those.

The Habits Worth Keeping

If the details fade, keep the habits. They were the real curriculum:

- **Verify the schema, do not assume it.** `CheckSchema` costs one round trip at startup and catches the migration that never ran.
- **Branch on the constraint, not the message.** `AsConstraintError` and `ConstraintKind` survive a PostgreSQL upgrade that changes wording; a substring match on `err.Error()` does not.
- **Choose the write path on purpose.** A single `InsertSingle`, a batched `InsertMany`, and a streaming `CopyFrom` all insert rows; which one is correct depends on row count and on whether you need per-row defaults and per-row errors back.
- **Retry the failure that is actually transient.** `IsErrRetryableTx` exists because not every transaction error should be retried, and retrying the wrong one turns a bug into a silent duplicate write.
- **Let the pool outlive the request.** Open one `*pg.DB` per process, not per handler, and let its pool amortize the cost of a new connection across everything that runs after it.

None of these are pg-specific. They will serve you against any database library you use next. pg simply gives each one a direct, typed way to follow it.

What to Build Next

Reading builds understanding; building makes it stick. Three suggestions, in rising order of ambition:

1. **Rebuild one of the `_examples` programs from memory.** The `password` example wires up hashing and the `gen` package end to end; doing it again without looking is the fastest way to find the parts that have not settled yet.
2. **Wire `ListenTable` into something that watches.** The `live-table` example pushes row changes to a browser over a websocket; a Slack notification or a cache invalidation is a smaller version of the same idea.
3. **Port one table of a real project to pg.** Take a table you already query by hand and give it a struct, a registered schema and a `Repository[T]`. That is the moment the material becomes yours.

Staying Current

- The library lives at github.com/kataras/pg: releases, issues and discussions all happen there.

- API reference documentation is on pkg.go.dev/github.com/kataras/pg, generated from the same source you have been reading about.
- Runnable programs beyond this book's examples live in the repository's `_examples` directory, including the `logging` example, which wires pg's query tracing into a structured logger.

Contributing

Bug reports with a failing test, documentation fixes, and honest questions in the discussions all move the library forward, the same way they move any open source project forward. The same goes for this book: when you spot something that could be clearer, or wrong, say so in the repository's discussions. Corrections from readers are how a book stays trustworthy.

Thank You

A database library is a wager: that the hours spent getting the schema, the pool and the query builder right will save many more hours for everyone who writes application code on top of them. By reading this far, and by whatever you build next, you are the other side of that wager paying off.

Thank you for reading. Now go build something good.

- Gerasimos Maropoulos